



SCHOOL OF DESIGN, ENGINEERING & COMPUTING

BSc (Hons) Computing

BSc (Hons) Software Engineering Management

ARM: ASSEMBLY LANGUAGE PROGRAMMING

STEPHEN WELSH

PETER KNAGGS

December 22, 2003

Contents

Contents	v
List of Programs	viii
Preface	ix
1 Introduction	1
1.1 The Meaning of Instructions	1
1.1.1 Binary Instructions	1
1.2 A Computer Program	1
1.3 The Binary Programming Problem	2
1.4 Using Octal or Hexadecimal	2
1.5 Instruction Code Mnemonics	3
1.6 The Assembler Program	4
1.6.1 Additional Features of Assemblers	4
1.6.2 Choosing an Assembler	5
1.7 Disadvantages of Assembly Language	5
1.8 High-Level Languages	6
1.8.1 Advantages of High-Level Languages	6
1.8.2 Disadvantages of High-Level Languages	7
1.9 Which Level Should You Use?	8
1.9.1 Applications for Machine Language	8
1.9.2 Applications for Assembly Language	8
1.9.3 Applications for High-Level Language	8
1.9.4 Other Considerations	8
1.10 Why Learn Assembler?	9
2 Assemblers	11
2.1 Fields	11
2.1.1 Delimiters	11
2.1.2 Labels	12
2.2 Operation Codes (Mnemonics)	14
2.3 Directives	14
2.3.1 The DEFINE CONSTANT (Data) Directive	14
2.3.2 The EQUATE Directive	15
2.3.3 The AREA Directive	16
2.3.4 Housekeeping Directives	17
2.3.5 When to Use Labels	17
2.4 Operands and Addresses	17
2.4.1 Decimal Numbers	18
2.4.2 Other Number Systems	18
2.4.3 Names	18
2.4.4 Character Codes	18

2.4.5	Arithmetic and Logical Expressions	18
2.4.6	General Recommendations	19
2.5	Comments	19
2.6	Types of Assemblers	20
2.7	Errors	20
2.8	Loaders	21
3	ARM Architecture	23
3.1	Processor modes	23
3.2	Registers	25
3.2.1	The stack pointer, SP or R13	26
3.2.2	The Link Register, LR or R14	27
3.2.3	The program counter, PC or R15	27
3.2.4	Current Processor Status Registers: CPSR	28
3.3	Flags	28
3.4	Exceptions	29
3.5	Instruction Set	30
3.5.1	Conditional Execution: $\langle cc \rangle$	31
3.5.2	Data Processing Operands: $\langle op1 \rangle$	32
3.5.3	Memory Access Operands: $\langle op2 \rangle$	34
4	The ARM Instruction Set	37
4.1	Instruction set encoding	37
4.1.1	Multiplies and extra load/store instructions	38
4.1.2	Miscellaneous instructions	39
4.2	The condition field	39
4.2.1	Condition code 0b1111	40
4.3	Branch instructions	40
4.3.1	Examples	41
4.3.2	List of branch instructions	42
4.4	Data-processing instructions	42
4.4.1	Instruction encoding	42
4.4.2	List of data-processing instructions	43
4.5	Multiply instructions	43
4.5.1	Normal multiply	43
4.5.2	Long multiply	44
4.5.3	Examples	44
4.5.4	List of multiply instructions	44
4.6	Miscellaneous arithmetic instructions	44
4.6.1	Instruction encoding	45
4.6.2	List of miscellaneous arithmetic instructions	45
4.7	Status register access instructions	45
4.7.1	CPSR value	45
4.7.2	Examples	46
4.7.3	List of status register access instructions	46
4.8	Load and store instructions	46
4.8.1	Addressing modes	46
4.8.2	Load and Store word or unsigned byte instructions	47
4.8.3	Load and Store Halfword and Load Signed Byte	48
4.8.4	Examples	48
4.8.5	List of load and store instructions	49
4.9	Load and Store Multiple instructions	49
4.9.1	Examples	50
4.9.2	List of Load and Store Multiple instructions	50

4.10	Semaphore instructions	50
4.10.1	Examples	51
4.10.2	List of semaphore instructions	51
4.11	Exception-generating instructions	51
4.11.1	Instruction encodings	51
4.11.2	List of exception-generating instructions	52
4.12	Coprocessor instructions	52
4.12.1	Examples	52
4.12.2	List of coprocessor instructions	53
4.13	Extending the instruction set	53
4.13.1	Undefined instruction space	54
4.13.2	Arithmetic instruction extension space	55
4.13.3	Control instruction extension space	55
4.13.4	Load/store instruction extension space	56
4.13.5	Coprocessor instruction extension space	57
4.13.6	Unconditional instruction extension space	58
5	ARM Addressing Modes	59
5.1	Data-processing operands	59
5.1.1	The shifter operand	59
5.1.2	Immediate	60
5.1.3	Register	61
5.1.4	Logical Shift Left by Immediate	61
5.1.5	Logical Shift Left by Register	62
5.1.6	Logical Shift Right by Immediate	62
5.1.7	Logical Shift Right by Register	62
5.1.8	Arithmetic Shift Right by Immediate	63
5.1.9	Arithmetic Shift Right by Register	63
5.1.10	Rotate Right by Immediate	63
5.1.11	Rotate Right by Register	64
5.1.12	Rotate Right with Extend	64
5.2	Memory Access	64
5.2.1	Immediate Offset	64
5.2.2	Register Offset	65
5.2.3	Scaled Register Offset	65
5.2.4	Immediate Pre-indexed	66
5.2.5	Register Pre-indexed	66
5.2.6	Scaled Register Pre-indexed	66
5.2.7	Immediate Post-indexed	67
5.2.8	Register Post-indexed	67
5.2.9	Scaled Register Post-indexed	68
6	Beginning Programs	69
6.1	Example Programs	69
6.1.1	Program Listing Format	69
6.1.2	Guidelines for Examples	70
6.1.3	Trying the examples	71
6.1.4	Program Initialization	71
6.1.5	Special Conditions	72
6.2	Program Examples	72
6.2.1	16-Bit Data Transfer	72
6.2.2	One's Complement	73
6.2.3	16-Bit Addition	74
6.2.4	Shift Left One Bit	75

6.2.5	Byte Disassembly	76
6.2.6	Find Larger of Two Numbers	76
6.2.7	64-Bit Addition	78
6.2.8	Table of Factorials	79
6.3	Problems	80
6.3.1	64-Bit Data Transfer	80
6.3.2	32-Bit Subtraction	81
6.3.3	Shift Right Three Bits	81
6.3.4	Halfword Assembly	81
6.3.5	Find Smallest of Three Numbers	81
6.3.6	Sum of Squares	81
6.3.7	Shift Left n bits	82
7	Program Loops	83
7.1	Program Examples	84
7.2	Problems	88
7.2.1	Checksum of data	88
7.2.2	Number of Zero, Positive, and Negative numbers	88
7.2.3	Find Minimum	89
7.2.4	Count 1 Bits	89
7.2.5	Find element with most 1 bits	89
8	Character-Coded Data	91
8.1	Handling data in ASCII	91
8.2	A string of characters	92
8.2.1	Fixed Length Strings	93
8.2.2	Terminated Strings	93
8.2.3	Counted Strings	94
8.3	International Characters	94
8.4	Program Examples	94
8.5	Problems	99
8.5.1	Length of a Teletypewriter Message	99
8.5.2	Find Last Non-Blank Character	100
8.5.3	Truncate Decimal String to Integer Form	100
8.5.4	Check Even Parity and ASCII Characters	101
8.5.5	String Comparison	101
9	Code Conversion	103
9.1	Program Examples	103
9.2	Problems	107
9.2.1	ASCII to Hexadecimal	107
9.2.2	Seven-Segment to Decimal	108
9.2.3	Decimal to ASCII	108
9.2.4	Binary to Binary-Coded-Decimal	108
9.2.5	Packed Binary-Coded-Decimal to Binary String	109
9.2.6	ASCII string to Binary number	109
10	Arithmetic	111
10.1	Program Examples	111
10.2	Problems	114
10.2.1	Multiple precision Binary subtraction	114
10.2.2	Decimal Subtraction	115
10.2.3	32-Bit by 32-Bit Multiply	115

11 Tables and Lists	117
11.1 Program Examples	117
11.2 Problems	120
11.2.1 Remove Entry from List	120
11.2.2 Add Entry to Ordered List	121
11.2.3 Add Element to Queue	121
11.2.4 4-Byte Sort	121
11.2.5 Using a Jump Table with a Key	122
12 Subroutines	123
12.1 Types of Subroutines	123
12.2 Subroutine Documentation	124
12.3 Parameter Passing Techniques	124
12.3.1 Passing Parameters In Registers	124
12.3.2 Passing Parameters In A Parameter Block	125
12.3.3 Passing Parameters On The Stack	125
12.4 Types Of Parameters	126
12.5 Program Examples	126
12.6 Problems	133
12.6.1 ASCII Hex to Binary	133
12.6.2 ASCII Hex String to Binary Word	133
12.6.3 Test for Alphabetic Character	133
12.6.4 Scan to Next Non-alphabetic	133
12.6.5 Check Even Parity	134
12.6.6 Check the Checksum of a String	134
12.6.7 Compare Two Counted Strings	134
A ARM Instructions	135
A.1 Alphabetical list of ARM instructions	135
A.1.1 General notes	135
A.2 ADC: Add with Carry	136
A.3 ADD: Add	137
A.4 AND: Bitwise AND	137
A.5 B, BL: Branch, Branch and Link	137
A.6 CMP: Compare	138
A.7 EOR: Exclusive OR	138
A.8 LDM: Load Multiple	139
A.9 LDR: Load Register	139
A.10 LDRB: Load Register Byte	140
A.11 MOV: Move	140
A.12 MVN: Move Negative	141
A.13 ORR: Bitwise OR	141
A.14 SBC: Subtract with Carry	141
A.15 STM: Store Multiple	142
A.16 STR: Store Register	143
A.17 STRB: Store Register Byte	143
A.18 SUB: Subtract	144
A.19 SWI: Software Interrupt	144
A.20 SWP: Swap	145
A.21 SWPB: Swap Byte	145
B ARM Instruction Summary	147

List of Programs

6.1	16bitdatatrans.s — 16bit data transfer	72
6.2	onescomp.s — Find the one's compliment of a number	73
6.3a	16bitadd.s — Add two 16bit numbers	74
6.3b	16bitadd-2.s — Add two 16bit numbers and store the result	74
6.4	shiftright.s — Shift Right one bit	75
6.5	splitbyte.s — Disassemble a byte into its high and low order nibbles	76
6.6	comparenum.s — Find the larger of two numbers	77
6.7	64bitadd.s — 64 bit addition	78
6.8	factorial.s — Lookup the factorial from a table by using the address of the memory location	79
7.1a	Ch5Ex1.s — Add a series of 16 bit numbers by using a table address	84
7.1b	Ch5Ex2.s — Add a series of 16 bit numbers by using a table address look-up	84
7.1c	Ch5Ex3.s — Scan a series of 32 bit numbers to find how many are negative	85
7.1d	Ch5Ex4.s — Scan a series of 16 bit numbers to find how many are negative	86
7.1e	Ch5Ex5.s — Scan a series of 16 bit numbers to find the largest	86
7.1f	Ch5Ex6.s — Normalize a binary number	87
8.1a	Ch6Ex1.s — Find the length of a string	94
8.1b	Ch6Ex2.s — Find the length of a null terminated string	95
8.1c	Ch6Ex3.s — Find the length of a string	95
8.1d	Ch6Ex4.s — Suppress leading zeros in a string	96
8.1e	Ch6Ex5.s — Set the parity bit on a series of characters store the amended string in Result	97
8.1f	Ch6Ex6.s — Compare two counted strings for equality	98
8.1g	Ch6Ex7.s — Compare null terminated strings for equality assume that we have no knowledge of the data structure so we must assess the individual strings	98
9.1a	Ch7Ex1.s — Convert a single hex digit to its ASCII equivalent	103
9.1b	Ch7Ex2.s — Convert a 32 bit hexadecimal number to an ASCII string and output to the terminal	104
9.1c	Ch7Ex3.s — Convert a decimal number to seven segment binary	104
9.1d	Ch7Ex4.s — Convert an ASCII numeric character to decimal	105
9.1e	Ch7Ex5.s — Convert an unpacked BCD number to binary	105
9.1f	Ch7Ex6.s — Convert an unpacked BCD number to binary using MUL	106
9.1g	Ch7Ex7.s — Store a 16bit binary number as an ASCII string of '0's and '1's	107
10.1a	Ch8Ex1.s — 16 bit binary multiplication	111
10.1b	Ch8Ex2.s — Divide a 32 bit binary no by a 16 bit binary no store the quotient and remainder there is no 'DIV' instruction in ARM!	112
10.1c	Ch8Ex3.s — Add two packed BCD numbers to give a packed BCD result	112
10.1d	Ch8Ex4.s — Multiply two 32 bit number to give a 64 bit result (corrupts R0 and R1)	113

11.1a Ch9Ex1.s — <i>Examine a table for a match - store a new entry at the end if no match found</i>	117
11.1b Ch9Ex2.s — <i>Examine a table for a match - store a new entry if no match found extends Ch9Ex1</i>	118
11.1c Ch9Ex3.s — <i>Examine an ordered table for a match</i>	118
11.1d Ch9Ex4.s — <i>Remove the first element of a queue</i>	119
11.1e Ch9Ex5.s — <i>Sort a list of values - simple bubble sort</i>	120
12.1a Ch10Ex1.s — <i>Initiate a simple stack</i>	126
12.1b Ch10Ex2.s — <i>Initiate a simple stack</i>	127
12.1c Ch10Ex3.s — <i>Initiate a simple stack</i>	127
12.1d Ch10Ex3a.s — <i>Initiate a simple stack</i>	128
12.1e Ch10Ex4.s — <i>A simple subroutine example program passes a variable to the routine in a register</i>	128
12.1f Ch10Ex5.s — <i>A more complex subroutine example program passes variables to the routine using the stack</i>	129
12.1g Ch10Ex6.s — <i>A 64 bit addition subroutine</i>	131
12.1h Ch10Ex7.s — <i>A subroutine to find the factorial of a number</i>	132

Preface

Broadly speaking, you can divide the history of computers into four periods: the mainframe, the mini, the microprocessor, and the modern post-microprocessor. The *mainframe* era was characterized by computers that required large buildings and teams of technicians and operators to keep them going. More often than not, both academics and students had little direct contact with the mainframe—you handed a deck of punched cards to an operator and waited for the output to appear hours later. During the mainframe era, academics concentrated on languages and compilers, algorithms, and operating systems.

The *minicomputer* era put computers in the hands of students and academics, because university departments could now buy their own minis. As minicomputers were not as complex as mainframes and because students could get direct hands-on experience, many departments of computer science and electronic engineering taught students how to program in the native language of the computer—assembly language. In those days, the mid 1970s, assembly language programming was used to teach both the control of I/O devices, and the writing of programs (i.e., assembly language was taught rather like high level languages). The explosion of computer software had not taken place, and if you wanted software you had to write it yourself.

The late 1970s saw the introduction of the *microprocessor*. For the first time, each student was able to access a real computer. Unfortunately, microprocessors appeared before the introduction of low-cost memory (both primary and secondary). Students had to program microprocessors in assembly language because the only storage mechanism was often a ROM with just enough capacity to hold a simple single-pass assembler.

The advent of the low-cost microprocessor system (usually on a single board) ensured that virtually every student took a course on assembly language. Even today, most courses in computer science include a module on computer architecture and organization, and teaching students to write programs in assembly language forces them to understand the computer's architecture. However, some computer scientists who had been educated during the mainframe era were unhappy with the microprocessor, because they felt that the 8-bit microprocessor was a retrograde step—its architecture was far more primitive than the mainframes they had studied in the 1960s.

The 1990s is the *post-microprocessor* era. Today's personal computers have more power and storage capacity than many of yesterday's mainframes, and they have a range of powerful software tools that were undreamed of in the 1970s. Moreover, the computer science curriculum of the 1990s has exploded. In 1970 a student could be expected to be familiar with all field of computer science. Today, a student can be expected only to browse through the highlights.

The availability of high-performance hardware and the drive to include more and more new material in the curriculum, has put pressure on academics to justify what they teach. In particular, many are questioning the need for courses on assembly language.

If you regard computer science as being primarily concerned with the *use* of the computer, you can argue that assembly language is an irrelevance. Does the surgeon study metallurgy in order to understand how a scalpel operates? Does the pilot study thermodynamics to understand how a jet engine operates? Does the news reader study electronics to understand how the camera

operates? The answer to all these questions is “no”. So why should we inflict assembly language and computer architecture on the student?

First, *education* is not the same as *training*. The student of computer science is not simply being trained to use a number of computer packages. A university course leading to a degree should also cover the *history* and the *theoretical basis* for the subject. Without a knowledge of computer architecture, the computer scientist cannot understand how computers have developed and what they are capable of.

Is assembly language today the same as assembly language yesterday?

Two factors have influenced the way in which we teach assembly language—one is the way in which microprocessors have changed, and the other is the use to which assembly language teaching is put. Over the years microprocessors have become more and more complex, with the result that the architecture and assembly language of a modern state-of-the-art microprocessor is radically different to that of an 8-bit machine of the late 1970s. When we first taught assembly language in the 1970s and early 1980s, we did it to demonstrate how computers operated and to give students hands-on experience of a computer. Since all students either have their own computer or have access to a computer lab, this role of the single-board computer is now obsolete. Moreover, assembly language programming once attempted to ape high-level language programming—students were taught algorithms such as sorting and searching in assembly language, as if assembly language were no more than the (desperately) poor person’s C.

The argument for teaching assembly language programming today can be divided into two components: the underpinning of computer architecture and the underpinning of computer software.

Assembly language teaches how a computer works at the machine (i.e., register) level. It is therefore necessary to teach assembly language to all those who might later be involved in computer architecture—either by specifying computers for a particular application, or by designing new architectures. Moreover, the von Neumann machine’s sequential nature teaches students the limitation of conventional architectures and, indirectly, leads them on to unconventional architectures (parallel processors, Harvard architectures, data flow computers, and even neural networks).

It is probably in the realm of software that you can most easily build a case for the teaching of assembly language. During a student’s career, he or she will encounter a lot of *abstract* concepts in subjects ranging from programming languages, to operating systems, to real-time programming, to AI. The foundation of many of these concepts lies in assembly language programming and computer architecture. You might even say that assembly language provides *bottom-up* support for the *top-down* methodology we teach in high-level languages. Consider some of the following examples (taken from the teaching of Advanced RISC Machines Ltd (ARM) assembly language).

Data types Students come across data types in high-level languages and the effects of strong and weak data typing. Teaching an assembly language that can operate on bit, byte, word and long word operands helps students understand data types. Moreover, the ability to perform any type of assembly language operation on any type of data structure demonstrates the need for strong typing.

Addressing modes A vital component of assembly language teaching is addressing modes (literal, direct, and indirect). The student learns how pointers function and how pointers are manipulated. This aspect is particularly important if the student is to become a C programmer. Because an assembly language is unencumbered by data types, the students’ view of pointers is much simplified by an assembly language. The ARM has complex addressing modes that support direct and indirect addressing, generated jump tables and handling of unknown memory offsets.

The stack and subroutines How procedures are called, and parameters passed and returned from procedures. By using an assembly language you can readily teach the passing of parameters by *value* and by *reference*. The use of *local variables* and *re-entrant* programming can also be taught. This supports the teaching of task switching kernels in both operating systems and real-time programming.

Recursion The recursive calling of subroutines often causes a student problems. You can use an assembly language, together with a suitable system with a tracing facility, to demonstrate how recursion operates. The student can actually observe how the stack grows as procedures are called.

Run-time support for high-level languages A high-performance processor like the ARM provides facilities that support run-time checking in high-level languages. For example, the programming techniques document lists a series of programs that interface with 'C' and provide run-time checking for errors such as an attempt to divide a number by zero.

Protected-mode operation Members of the ARM family operate in either a *privilege mode* or a *user mode*. The operating system operates in the privilege mode and all user (applications) programs run in the user mode. This mechanism can be used to construct *secure* or *protected* environments in which the effects of an error in one application can be prevented from harming the operating system (or other applications).

Input-output Many high-level languages make it difficult to access I/O ports and devices directly. By using an assembly language we can teach students how to write device drivers and how to control interfaces. Most real interfaces are still programmed at the machine level by accessing registers within them.

All these topics can, of course, be taught in the appropriate courses (e.g., high-level languages, operating systems). However, by teaching them in an assembly language course, they pave the way for future studies, and also show the student exactly what is happening within the machine.

Conclusion

A strong case can be made for the continued teaching of assembly language within the computer science curriculum. However, an assembly language cannot be taught just as if it were another general-purpose programming language as it was once taught ten years ago. Perhaps more than any other component of the computer science curriculum, teaching an assembly language supports a wide range of topics at the heart of computer science. An assembly language should not be used just to illustrate algorithms, but to demonstrate what is actually happening inside the computer.

1 Introduction

A computer program is ultimately a series of numbers and therefore has very little meaning to a human being. In this chapter we will discuss the levels of human-like language in which a computer program may be expressed. We will also discuss the reasons for and uses of assembly language.

1.1 The Meaning of Instructions

The instruction set of a microprocessor is the set of binary inputs that produce defined actions during an instruction cycle. An instruction set is to a microprocessor what a function table is to a logic device such as a gate, adder, or shift register. Of course, the actions that the microprocessor performs in response to its instruction inputs are far more complex than the actions that logic devices perform in response to their inputs.

1.1.1 Binary Instructions

An instruction is a binary digit pattern — it must be available at the data inputs to the microprocessor at the proper time in order to be interpreted as an instruction. For example, when the ARM receives the binary pattern 111000000100 as the input during an instruction fetch operation, the pattern means subtract. Similarly the microinstruction 111000001000 means add. Thus the 32 bit pattern 11100000010011101100000000001111 means:

“Subtract R15 from R14 and put the answer in R12.”

The microprocessor (like any other computer) only recognises binary patterns as instructions or data; it does not recognise characters or octal, decimal, or hexadecimal numbers.

1.2 A Computer Program

A program is a series of instructions that causes a computer to perform a particular task.

Actually, a computer program includes more than instructions, it also contains the data and the memory addresses that the microprocessor needs to accomplish the tasks defined by the instructions. Clearly, if the microprocessor is to perform an addition, it must have two numbers to add and a place to put the result. The computer program must determine the sources of the data and the destination of the result as well as the operation to be performed.

All microprocessors execute instructions sequentially unless an instruction changes the order of execution or halts the processor. That is, the processor gets its next instruction from the next higher memory address unless the current instruction specifically directs it to do otherwise.

Ultimately, every program is a set of binary numbers. For example, this is a snippet of an ARM program that adds the contents of memory locations 8094₃₂ and 8098₃₂ and places the result in memory location 809C₃₂:

```

11100101100111110001000000010000
11100101100111110001000000001000
11100000100000010101000000000000
11100101100011110101000000001000

```

This is a machine language, or object, program. If this program were entered into the memory of an ARM-based microcomputer, the microcomputer would be able to execute it directly.

1.3 The Binary Programming Problem

There are many difficulties associated with creating programs as object, or binary machine language, programs. These are some of the problems:

- The programs are difficult to understand or debug. (Binary numbers all look the same, particularly after you have looked at them for a few hours.)
- The programs do not describe the task which you want the computer to perform in anything resembling a human-readable format.
- The programs are long and tiresome to write.
- The programmer often makes careless errors that are very difficult to locate and correct.

For example, the following version of the addition object program contains a single bit error. Try to find it:

```

11100101100111110001000000010000
11100101100111110001000000001000
11100000100000010101000000000000
11100110100011110101000000001000

```

Although the computer handles binary numbers with ease, people do not. People find binary programs long, tiresome, confusing, and meaningless. Eventually, a programmer may start remembering some of the binary codes, but such effort should be spent more productively.

1.4 Using Octal or Hexadecimal

We can improve the situation somewhat by writing instructions using octal or hexadecimal numbers, rather than binary. We will use hexadecimal numbers because they are shorter, and because they are the standard for the microprocessor industry. Table 1.1 defines the hexadecimal digits and their binary equivalents. The ARM program to add two numbers now becomes:

```

E59F1010
E59f0008
E0815000
E58F5008

```

At the very least, the hexadecimal version is shorter to write and not quite so tiring to examine.

Errors are somewhat easier to find in a sequence of hexadecimal digits. The erroneous version of the addition program, in hexadecimal form, becomes:

Hexadecimal Digit	Binary Equivalent	Decimal Equivalent
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15

Table 1.1: Hexadecimal Conversion Table

E59F1010
E59f0008
E0815000
E68F5008

The mistake is far more obvious.

The hexadecimal version of the program is still difficult to read or understand; for example, it does not distinguish operations from data or addresses, nor does the program listing provide any suggestion as to what the program does. What does 3038 or 31C0 mean? Memorising a card full of codes is hardly an appetising proposition. Furthermore, the codes will be entirely different for a different microprocessor and the program will require a large amount of documentation.

1.5 Instruction Code Mnemonics

An obvious programming improvement is to assign a name to each instruction code. The instruction code name is called a “*mnemonic*” or memory jogger.

In fact, all microprocessor manufacturers provide a set of mnemonics for the microprocessor instruction set (they cannot remember hexadecimal codes either). You do not have to abide by the manufacturer’s mnemonics; there is nothing sacred about them. However, they are standard for a given microprocessor, and therefore understood by all users. These are the instruction codes that you will find in manuals, cards, books, articles, and programs. The problem with selecting instruction mnemonics is that not all instructions have “obvious” names. Some instructions do (for example, ADD, AND, ORR), others have obvious contractions (such as SUB for subtraction, EOR for exclusive-OR), while still others have neither. The result is such mnemonics as BIC, STMIA, and even MRS. Most manufacturers come up with some reasonable names and some hopeless ones. However, users who devise their own mnemonics rarely do much better.

Along with the instruction mnemonics, the manufacturer will usually assign names to the CPU registers. As with the instruction names, some register names are obvious (such as A for Accumulator) while others may have only historical significance. Again, we will use the manufacturer’s suggestions simply to promote standardisation.

If we use standard ARM instruction and register mnemonics, as defined by Advanced RISC Machines, our ARM addition program becomes:

```

LDR  R1, num1
LDR  R0, num2
ADD  R5, R1, R0
STR  R5, num3

```

The program is still far from obvious, but at least some parts are comprehensible. `ADD` is a considerable improvement over `E59F`. The `LDR` mnemonic does suggest loading data into a register or memory location. We now see that some parts of the program are operations and others are addresses. Such a program is an assembly language program.

1.6 The Assembler Program

How do we get the assembly language program into the computer? We have to translate it, either into hexadecimal or into binary numbers. You can translate an assembly language program by hand, instruction by instruction. This is called hand assembly.

The following table illustrates the hand assembly of the addition program:

Instruction Mnemonic	Register/Memory Location	Hexadecimal Equivalent
<code>LDR</code>	<code>R1, num1</code>	<code>E59F1010</code>
<code>LDR</code>	<code>R0, num2</code>	<code>E59F0008</code>
<code>ADD</code>	<code>R5, R1, R0</code>	<code>E0815000</code>
<code>STR</code>	<code>R5, num3</code>	<code>E58F5008</code>

Hand assembly is a rote task which is uninteresting, repetitive, and subject to numerous minor errors. Picking the wrong line, transposing digits, omitting instructions, and misreading the codes are only a few of the mistakes that you may make. Most microprocessors complicate the task even further by having instructions with different lengths. Some instructions are one word long while others may be two or three. Some instructions require data in the second and third words; others require memory addresses, register numbers, or who knows what?

Assembly is a rote task that we can assign to the microcomputer. The microcomputer never makes any mistakes when translating codes; it always knows how many words and what format each instruction requires. The program that does this job is an “*assembler*.” The assembler program translates a user program, or “source” program written with mnemonics, into a machine language program, or “object” program, which the microcomputer can execute. The assembler’s input is a source program and its output is an object program.

Assemblers have their own rules that you must learn. These include the use of certain markers (such as spaces, commas, semicolons, or colons) in appropriate places, correct spelling, the proper control of information, and perhaps even the correct placement of names and numbers. These rules are usually simple and can be learned quickly.

1.6.1 Additional Features of Assemblers

Early assemblers did little more than translate the mnemonic names of instructions and registers into their binary equivalents. However, most assemblers now provide such additional features as:

- Allowing the user to assign names to memory locations, input and output devices, and even sequences of instructions
- Converting data or addresses from various number systems (for example, decimal or hexadecimal) to binary and converting characters into their ASCII or EBCDIC binary codes

- Performing some arithmetic as part of the assembly process
- Telling the loader program where in memory parts of the program or data should be placed
- Allowing the user to assign areas of memory as temporary data storage and to place fixed data in areas of program memory
- Providing the information required to include standard programs from program libraries, or programs written at some other time, in the current program
- Allowing the user to control the format of the program listing and the input and output devices employed

1.6.2 Choosing an Assembler

All of these features, of course, involve additional cost and memory. Microcomputers generally have much simpler assemblers than do larger computers, but the tendency is always for the size of assemblers to increase. You will often have a choice of assemblers. The important criterion is not how many off-beat features the assembler has, but rather how convenient it is to use in normal practice.

1.7 Disadvantages of Assembly Language

The assembler does not solve all the problems of programming. One problem is the tremendous gap between the microcomputer instruction set and the tasks which the microcomputer is to perform. Computer instructions tend to do things like add the contents of two registers, shift the contents of the Accumulator one bit, or place a new value in the Program Counter. On the other hand, a user generally wants a microcomputer to do something like print a number, look for and react to a particular command from a teletypewriter, or activate a relay at the proper time. An assembly language programmer must translate such tasks into a sequence of simple computer instructions. The translation can be a difficult, time-consuming job.

Furthermore, if you are programming in assembly language, you must have detailed knowledge of the particular microcomputer that you are using. You must know what registers and instructions the microcomputers has, precisely how the instructions affect the various registers, what addressing methods the computer uses, and a mass of other information. None of this information is relevant to the task which the microcomputer must ultimately perform.

In addition, assembly language programs are not portable. Each microcomputer has its own assembly language which reflects its own architecture. An assembly language program written for the ARM will not run on a 486, Pentium, or Z8000 microprocessor. For example, the addition program written for the Z8000 would be:

```
LD    R0,%6000
ADD   R0,%6002
LD    %6004,R0
```

The lack of portability not only means that you will not be able to use your assembly language program on a different microcomputer, but also that you will not be able to use any programs that were not specifically written for the microcomputer you are using. This is a particular drawback for new microcomputers, since few assembly language programs exist for them. The result, too frequently, is that you are on your own. If you need a program to perform a particular task, you are not likely to find it in the small program libraries that most manufacturers provide. Nor are you likely to find it in an archive, journal article, or someone's old program File. You will probably have to write it yourself.

1.8 High-Level Languages

The solution to many of the difficulties associated with assembly language programs is to use, instead, *high-level* or *procedure-oriented* languages. Such languages allow you to describe tasks in forms that are problem-oriented rather than computer-oriented. Each statement in a high-level language performs a recognisable function; it will generally correspond to many assembly language instructions. A program called a compiler translates the high-level language source program into object code or machine language instructions.

Many different high-level languages exist for different types of tasks. If, for example, you can express what you want the computer to do in algebraic notation, you can write your **FORTRAN** (*Formula Translation Language*), the oldest of the high-level languages. Now, if you want to add two numbers, you just tell the computer:

```
sum = num1 + num2;
```

That is a lot simpler (and shorter) than either the equivalent machine language program or the equivalent assembly language program. Other high-level languages include **COBOL** (for business applications), **BASIC** (a cut down version of **FORTRAN** designed to prototype ideas before coding them in full), **C** (a systems-programming language), **C++** and **JAVA** (object-orientated general development languages).

1.8.1 Advantages of High-Level Languages

Clearly, high-level languages make program easier and faster to write. A common estimate is that a programmer can write a program about ten times as fast in a high-level language as in assembly language. That is just writing the program; it does not include problem definition, program design, debugging testing or documentation, all of which become simpler and faster. The high-level language program is, for instance, partly self-documenting. Even if you do not know **FORTRAN**, you could probably tell what the statement illustrated above does.

Machine Independence

High-level languages solve many other problems associated with assembly language programming. The high-level language has its own syntax (usually defined by an international standard). The language does not mention the instruction set, registers, or other features of a particular computer. The compiler takes care of all such details. Programmers can concentrate on their own tasks; they do not need a detailed understanding of the underlying CPU architecture — for that matter, they do not need to know anything about the computer they are programming.

Portability

Programs written in a high-level language are portable — at least, in theory. They will run on any computer that has a standard compiler for that language.

At the same time, all previous programs written in a high-level language for prior computers and available to you when programming a new computer. This can mean thousands of programs in the case of a common language like **C**.

1.8.2 Disadvantages of High-Level Languages

If all the good things we have said about high-level languages are true — if you can write programs faster and make them portable besides — why bother with assembly languages? Who wants to worry about registers, instruction codes, mnemonics, and all that garbage! As usual, there are disadvantages that balance the advantages.

Syntax

One obvious problem is that, as with assembly language, you have to learn the “rules” or *syntax* of any high-level language you want to use. A high-level language has a fairly complicated set of rules. You will find that it takes a lot of time just to get a program that is syntactically correct (and even then it probably will not do what you want). A high-level computer language is like a foreign language. If you have talent, you will get used to the rules and be able to turn out programs that the compiler will accept. Still, learning the rules and trying to get the program accepted by the compiler does not contribute directly to doing your job.

Cost of Compilers

Another obvious problem is that you need a compiler to translate program written in a high-level language into machine language. Compilers are expensive and use a large amount of memory. While most assemblers occupy only a few KBytes of memory, compilers would occupy far larger amounts of memory. A compiler could easily require over four times as much memory as an assembler. So the amount of overhead involved in using the compiler is rather large.

Adapting Tasks to a Language

Furthermore, only some compilers will make the implementation of your task simpler. Each language has its own target problem area, for example, FORTRAN is well-suited to problems that can be expressed as algebraic formulas. If however, your problem is controlling a display terminal, editing a string of characters, or monitoring an alarm system, your problem cannot be easily expressed. In fact, formulating the solution in FORTRAN may be more awkward and more difficult than formulating it in assembly language. The answer is, of course, to use a more suitable high-level language. Languages specifically designed for tasks such as those mentioned above do exist — they are called system implementation languages. However, these languages are less widely used.

Inefficiency

High-level languages do not produce very efficient machine language program. The basic reason for this is that compilation is an automatic process which is riddled with compromises to allow for many ranges of possibilities. The compiler works much like a computerised language translator — sometimes the words are right but the sentence structures are awkward. A simpler compiler cannot know when a variable is no longer being used and can be discarded, when a register should be used rather than a memory location, or when variables have simple relationships. The experienced programmer can take advantage of shortcuts to shorten execution time or reduce memory usage. A few compiler (known as optimizing compilers) can also do this, but such compilers are much larger than regular compilers.

1.9 Which Level Should You Use?

Which language level you use depends on your particular application. Let us briefly note some of the factors which may favor particular levels:

1.9.1 Applications for Machine Language

- Virtually no one program in machine language because it wastes human time and is difficult to document. An assembler costs very little and greatly reduces programming time.

1.9.2 Applications for Assembly Language

- Short to moderate-sized programs
- Application where memory cost is a factor
- Real-Time control applications
- Limited data processing
- High-volume applications
- Applications involving more input/output or control than computation

1.9.3 Applications for High-Level Language

- Long programs
- Low-volume applications
- Applications where the amount of memory required is already very large
- Applications involving more computation than input/output or control
- Compatibility with similar applications using larger computers
- Availability of specific program in a high-level language which can be used in the application.
- Programs which are expected to undergo many changes

1.9.4 Other Considerations

Many other factors are also important, such as the availability of a large computer for use in development, experience with particular languages, and compatibility with other applications.

If hardware will ultimately be the largest cost in your application, or if speed is critical, you should favor assembly language. But be prepared to spend much extra time in software development in exchange for lower memory costs and higher execution speeds. If software will be the largest cost in your application, you should favor a high-level language. But be prepared to spend the extra money required for the supporting hardware and software.

Of course, no one except some theorists will object if you use both assembly and high-level languages. You can write the program originally in a high-level language and then patch some sections in assembly language. However, most users prefer not to do this because it can create havoc in debugging, testing, and documentation.

1.10 Why Learn Assembler?

Given the advance of high-level languages, why do you need to learn assembly language programming? The reasons are:

1. Most industrial microcomputer users program in assembly language.
2. Many microcomputer users will continue to program in assembly language since they need the detailed control that it provides.
3. No suitable high-level language has yet become widely available or standardised.
4. Many application require the efficiency of assembly language.
5. An understanding of assembly language can help in evaluating high-level languages.
6. Almost all microcomputer programmers ultimately find that they need some knowledge of assembly language, most often to debug programs, write I/O routines, speed up or shorten critical sections of programs written in high-level languages, utilize or modify operating system functions, and understand other people's programs.

The rest of these notes will deal exclusively with assembler and assembly language programming.

2 Assemblers

This chapter discusses the functions performed by assemblers, beginning with features common to most assemblers and proceeding through more elaborate capabilities such as macros and conditional assembly. You may wish to skim this chapter for the present and return to it when you feel more comfortable with the material.

As we mentioned, today's assemblers do much more than translate assembly language mnemonics into binary codes. But we will describe how an assembler handles the translation of mnemonics before describing additional assembler features. Finally we will explain how assemblers are used.

2.1 Fields

Assembly language instructions (or “statements”) are divided into a number of “*fields*”.

The operation code field is the only field which can never be empty; it always contains either an instruction mnemonic or a directive to the assembler, sometimes called a “pseudo-instruction,” “pseudo-operation,” or “pseudo-op.”

The operand or address field may contain an address or data, or it may be blank.

The comment and label fields are optional. A programmer will assign a label to a statement or add a comment as a personal convenience: namely, to make the program easier to read and use.

Of course, the assembler must have some way of telling where one field ends and another begins. Assemblers often require that each field start in a specific column. This is a “fixed format.” However, fixed formats are inconvenient when the input medium is paper tape; fixed formats are also a nuisance to programmers. The alternative is a “free format” where the fields may appear anywhere on the line.

2.1.1 Delimiters

If the assembler cannot use the position on the line to tell the fields apart, it must use something else. Most assemblers use a special symbol or “delimiter” at the beginning or end of each field.

Label Field	Operation Code or Mnemonic Field	Operand or Address Field	Comment Field
VALUE1	DCW	0x201E	;FIRST VALUE
VALUE2	DCW	0x0774	;SECOND VALUE
RESULT	DCW	1	;16-BIT STORAGE FOR ADDITION RESULT
START	MOV	RO, VALUE1	;GET FIRST VALUE
	ADD	RO, RO, VALUE2	;ADD SECOND VALUE TO FIRST VALUE
	STR	RESULT, RO	;STORE RESULT OF ADDITION
NEXT:	?	?	;NEXT INSTRUCTION

label *<whitespace>* instruction *<whitespace>* ; comment

whitespace	Between label and operation code, between operation code and address, and before an entry in the comment field
comma	Between operands in the address field
asterisk	Before an entire line of comment
semicolon	Marks the start of a comment on a line that contains preceding code

Table 2.1: Standard ARM Assembler Delimiters

The most common delimiter is the space character. Commas, periods, semicolons, colons, slashes, question marks, and other characters that would not otherwise be used in assembly language programs also may serve as delimiters. The general form of layout for the ARM assembler is:

You will have to exercise a little care with delimiters. Some assemblers are fussy about extra spaces or the appearance of delimiters in comments or labels. A well-written assembler will handle these minor problems, but many assemblers are not well-written. Our recommendation is simple: avoid potential problems if you can. The following rules will help:

- Do not use extra spaces, in particular, do not put spaces after commas that separate operands, even though the ARM assembler allows you to do this.
- Do not use delimiter characters in names or labels.
- Include standard delimiters even if your assembler does not require them. Then it will be more likely that your programs are in correct form for another assembler.

2.1.2 Labels

The label field is the first field in an assembly language instruction; it may be blank. If a label is present, the assembler defines the label as equivalent to the address into which the first byte of the object code generated for that instruction will be loaded. You may subsequently use the label as an address or as data in another instruction's address field. The assembler will replace the label with the assigned value when creating an object program.

The ARM assembler requires labels to start at the first character of a line. However, some other assemblers also allow you to have the label start anywhere along a line, in which case you must use a colon (:) as the delimiter to terminate the label field. Colon delimiters are not used by the ARM assembler.

Labels are most frequently used in Branch or SWI instructions. These instructions place a new value in the program counter and so alter the normal sequential execution of instructions. B 150₁₆ means “*place the value 150₁₆ in the program counter.*” The next instruction to be executed will be the one in memory location 150₁₆. The instruction B **START** means “*place the value assigned to the label **START** in the program counter.*” The next instruction to be executed will be the one at the address corresponding to the label **START**. Figure 2.1 contains an example.

Why use a label? Here are some reasons:

- A label makes a program location easier to find and remember.
- The label can easily be moved, if required, to change or correct a program. The assembler will automatically change all instructions that use the label when the program is reassembled.

Assembly language Program

```

START  MOV   RO, VALUE1
        .
        .      (Main Program)
        .
        BAL   START

```

When the machine language version of this program is executed, the instruction **B START** causes the address of the instruction labeled **START** to be placed in the program counter. That instruction will then be executed.

Figure 2.1: Assigning and Using a Label

- The assembler can relocate the whole program by adding a constant (a “relocation constant”) to each address in which a label was used. Thus we can move the program to allow for the insertion of other programs or simply to rearrange memory.
- The program is easier to use as a library program; that is, it is easier for someone else to take your program and add it to some totally different program.
- You do not have to figure out memory addresses. Figuring out memory addresses is particularly difficult with microprocessors which have instructions that vary in length.

You should assign a label to any instruction that you might want to refer to later.

The next question is how to choose a label. The assembler often places some restrictions on the number of characters (usually 5 or 6), the leading character (often must be a letter), and the trailing characters (often must be letters, numbers, or one of a few special characters). Beyond these restrictions, the choice is up to you.

Our own preference is to use labels that suggest their purpose, i.e., mnemonic labels. Typical examples are **ADDW** in a routine that adds one word into a sum, **SRCHETX** in a routine that searches for the ASCII character **ETX**, or **NKEYS** for a location in data memory that contains the number of key entries. Meaningful labels are easier to remember and contribute to program documentation. Some programmers use a standard format for labels, such as starting with **L0000**. These labels are self-sequencing (you can skip a few numbers to permit insertions), but they do not help document the program.

Some label selection rules will keep you out of trouble. We recommend the following:

- Do not use labels that are the same as operation codes or other mnemonics. Most assemblers will not allow this usage; others will, but it is confusing.
- Do not use labels that are longer than the assembler recognises. Assemblers have various rules, and often ignore some of the characters at the end of a long label.
- Avoid special characters (non-alphabetic and non-numeric) and lower-case letters. Some assemblers will not permit them; others allow only certain ones. The simplest practice is to stick to capital letters and numbers.
- Start each label with a letter. Such labels are always acceptable.
- Do not use labels that could be confused with each other. Avoid the letters **I**, **O**, and **Z** and the numbers **0**, **1**, and **2**. Also avoid things like **XXXX** and **XXXXX**. Assembly programming is difficult enough without tempting fate or Murphy’s Law.
- When you are not sure if a label is legal, do not use it. You will not get any real benefit from discovering exactly what the assembler will accept.

These are recommendations, not rules. You do not have to follow them but don't blame us if you waste time on unnecessary problems.

2.2 Operation Codes (Mnemonics)

One main task of the assembler is the translation of mnemonic operation codes into their binary equivalents. The assembler performs this task using a fixed table much as you would if you were doing the assembly by hand.

The assembler must, however, do more than just translate the operation codes. It must also somehow determine how many operands the instruction requires and what type they are. This may be rather complex — some instructions (like a Stop) have no operands, others (like a Jump instruction) have one, while still others (like a transfer between registers or a multiple-bit shift) require two. Some instructions may even allow alternatives; for example, some computers have instructions (like Shift or Clear) which can either apply to a register in the CPU or to a memory location. We will not discuss how the assembler makes these distinctions; we will just note that it must do so.

2.3 Directives

Some assembly language instructions are not directly translated into machine language instructions. These instructions are directives to the assembler; they assign the program to certain areas in memory, define symbols, designate areas of memory for data storage, place tables or other fixed data in memory, allow references to other programs, and perform minor housekeeping functions.

To use these assembler directives or pseudo-operations a programmer places the directive's mnemonic in the operation code field, and, if the specified directive requires it, an address or data in the address field.

The most common directives are:

```
DEFINE CONSTANT (Data)
EQUATE (Define)
AREA
DEFINE STORAGE (Reserve)
```

Different assemblers use different names for those operations but their functions are the same. Housekeeping directives include:

```
END          LIST          FORMAT          TTL          PAGE          INCLUDE
```

We will discuss these pseudo-operations briefly, although their functions are usually obvious.

2.3.1 The DEFINE CONSTANT (Data) Directive

The DEFINE CONSTANT directive allows the programmer to enter fixed data into program memory. This data may include:

- Names
- Messages
- Commands
- Tax tables
- Thresholds
- Test patterns
- Lookup tables
- Standard forms
- Masking patterns
- Weighting factors
- Conversion factors
- Key identifications
- Subroutine addresses
- Code conversion tables
- Identification patterns
- State transition tables
- Synchronisation patterns
- Coefficients for equations
- Character generation patterns
- Characteristic times or frequencies

The define constant directive treats the data as a permanent part of the program.

The format of a define constant directive is usually quite simple. An instruction like:

```
DZCON    DCW    12
```

will place the number 12 in the next available memory location and assign that location the name DZCON. Every DC directive usually has a label, unless it is one of a series. The data and label may take any form that the assembler permits.

More elaborate define constant directives that handle a large amount of data at one time are provided, for example:

```
EMESS    DCB    'ERROR'
SQRS     DCW    1,4,9,16,25
```

A single directive may fill many bytes of program memory, limited perhaps by the length of a line or by the restrictions of a particular assembler. Of course, you can always overcome any restrictions by following one define constant directive with another:

```
MESSG    DCB    "NOW IS THE "
          DCB    "TIME FOR ALL "
          DCB    "GOOD MEN "
          DCB    "TO COME TO THE "
          DCB    "AID OF THEIR "
          DCB    "COUNTRY", 0 ;note the '0' terminating the string
```

Microprocessor assemblers typically have some variations of standard define constant directives. Define Byte or DCB handles 8-bit numbers; Define Word or DCW handles 32-bit numbers or addresses. Other special directives may handle character-coded data. The ARM assembler also defines DCD to (Define Constant Data) which may be used in place of DCW.

2.3.2 The EQUATE Directive

The EQUATE directive allows the programmer to equate names with addresses or data. This pseudo-operation is almost always given the mnemonic EQU. The names may refer to device addresses, numeric data, starting addresses, fixed addresses, etc.

The EQUATE directive assigns the numeric value in its operand field to the label in its label field. Here are two examples:

```
TTY      EQU    5
LAST     EQU    5000
```

Most assemblers will allow you to define one label in terms of another, for example:

```

LAST      EQU      FINAL
ST1       EQU      START+1

```

The label in the operand field must, of course, have been previously defined. Often, the operand field may contain more complex expressions, as we shall see later. Double name assignments (two names for the same data or address) may be useful in patching together programs that use different names for the same variable (or different spellings of what was supposed to be the same name).

Note that an EQU directive does not cause the assembler to place anything in memory. The assembler simply enters an additional name into a table (called a “symbol table”) which the assembler maintains.

When do you use a name? The answer is: whenever you have a parameter that you might want to change or that has some meaning besides its ordinary numeric value. We typically assign names to time constants, device addresses, masking patterns, conversion factors, and the like. A name like DELAY, TTY, KBD, KROW, or OPEN not only makes the parameter easier to change, but it also adds to program documentation. We also assign names to memory locations that have special purposes; they may hold data, mark the start of the program, or be available for intermediate storage.

What name do you use? The best rules are much the same as in the case of labels, except that here meaningful names really count. Why not call the teletypewriter TTY instead of X15, a bit time delay BTIME or BTDLY rather than WW, the number of the “GO” key on a keyboard GOKEY rather than HORSE? This advice seems straightforward, but a surprising number of programmers do not follow it.

Where do you place the EQUATE directives? The best place is at the start of the program, under appropriate comment headings such as I/O ADDRESSES, TEMPORARY STORAGE, TIME CONSTANTS, or PROGRAM LOCATIONS. This makes the definitions easy to find if you want to change them. Furthermore, another user will be able to look up all the definitions in one centralised place. Clearly this practice improves documentation and makes the program easier to use.

Definitions used only in a specific subroutine should appear at the start of the subroutine.

2.3.3 The AREA Directive

The AREA directive allows the programmer to specify the memory locations where programs, subroutines, or data will reside. Programs and data may be located in different areas of memory depending on the memory configuration. Startup routines interrupt service routines, and other required programs may be scattered around memory at fixed or convenient addresses.

The assembler maintains a location counter (comparable to the computer’s program counter) which contains the location in memory of the instruction or data item being processed. An area directive causes the assembler to place a new value in the location counter, much as a Jump instruction causes the CPU to place a new value in the program counter. The output from the assembler must not only contain instructions and data, but must also indicate to the loader program where in memory it should place the instructions and data.

Microprocessor programs often contain several AREA statements for the following purposes:

- Reset (startup) address
- Interrupt service addresses
- Trap (software interrupt) addresses
- RAM storage
- Stack
- Main program
- Subroutines
- Input/Output

Still other origin statements may allow room for later insertions, place tables or data in memory, or assign vacant memory space for data buffers. Program and data memory in microcomputers may occupy widely separate addresses to simplify the hardware. Typical origin statements are:

```
AREA    RESET
AREA    $1000
AREA    INT3
```

The assembler will assume a fake address if the programmer does not put in an **AREA** statement. The **AREA** statement at the start of an ARM program is required, and its absence will cause the assembly to fail.

2.3.4 Housekeeping Directives

There are various assembler directives that affect the operation of the assembler and its program listing rather than the object program itself. Common directives include:

END, marks the end of the assembly language source program. This must appear in the file or a “missing **END** directive” error will occur.

INCLUDE will include the contents of a named file into the current file. When the included file has been processed the assembler will continue with the next line in the original file. For example the following line

```
INCLUDE MATH.S
```

will include the content of the file **math.s** at that point of the file.

You should never use a label with an include directive. Any labels defined in the included file will be defined in the current file, hence an error will be reported if the same label appears in both the source and include file.

An include file may itself include other files, which in turn could include other files, and so on, however, the level of includes the assembler will accept is limited. It is not recommended you go beyond three levels for even the most complex of software.

2.3.5 When to Use Labels

Users often wonder if or when they can assign a label to an assembler directive. These are our recommendations:

1. All **EQU** directives must have labels; they are useless otherwise, since the purpose of an **EQU** is to define its label.
2. Define Constant and Define Storage directives usually have labels. The label identifies the first memory location used or assigned.
3. Other directives should not have labels.

2.4 Operands and Addresses

The assembler allow the programmer a lot of freedom in describing the contents of the operand or address field. But remember that the assembler has built-in names for registers and instructions and may have other built-in names. We will now describe some common options for the operand field.

2.4.1 Decimal Numbers

The assembler assume all numbers to be decimal unless they are marked otherwise. So:

```
ADD    100
```

means “add the contents of memory location 100₁₀ to the contents of the Accumulator.”

2.4.2 Other Number Systems

The assembler will also accept hexadecimal entries. But you must identify these number systems in some way: for example, by preceding the number with an identifying character.

2_	Binary	Base 2
8_	Octal	Base 8
	Decimal	Base 10
0x	Hexadecimal	Base 16

It is good practice to enter numbers in the base in which their meaning is the clearest: that is, decimal constants in decimal; addresses and BCD numbers in hexadecimal; masking patterns or bit outputs in hexadecimal.

2.4.3 Names

Names can appear in the operand field; they will be treated as the data that they represent. Remember, however, that there is a difference between operands and addresses. In an ARM assembly language program the sequence:

```
FIVE    EQU    5
        ADD    R2, #FIVE
```

will add the contents of memory location FIVE (not necessarily the number 5) to the contents of data register R2.

2.4.4 Character Codes

The assembler allows text to be entered as ASCII strings. Such strings must be surrounded with double quotation marks, unless a single ASCII character is quoted, when single quotes may be used exactly as in 'C'. We recommend that you use character strings for all text. It improves the clarity and readability of the program.

2.4.5 Arithmetic and Logical Expressions

Assemblers permit combinations of the data forms described above, connected by arithmetic, logical, or special operators. These combinations are called expressions. Almost all assemblers allow simple arithmetic expressions such as `START+1`. Some assemblers also permit multiplication, division, logical functions, shifts, etc. Note that the assembler evaluates expressions at assembly time; if a symbol appears in an expression, the address is used (i.e., the location counter or EQUATE value).

Assemblers vary in what expressions they accept and how they interpret them. Complex expressions make a program difficult to read and understand.

2.4.6 General Recommendations

We have made some recommendations during this section but will repeat them and add others here. In general, the user should strive for clarity and simplicity. There is no payoff for being an expert in the intricacies of an assembler or in having the most complex expression on the block. We suggest the following approach:

- Use the clearest number system or character code for data.
- Masks and BCD numbers in decimal, ASCII characters in octal, or ordinary numerical constants in hexadecimal serve no purpose and therefore should not be used.
- Remember to distinguish data from addresses.
- Don't use offsets from the location counter.
- Keep expressions simple and obvious. Don't rely on obscure features of the assembler.

2.5 Comments

All assemblers allow you to place comments in a source program. Comments have no effect on the object code, but they help you to read, understand, and document the program. Good commenting is an essential part of writing computer programs, programs without comments are very difficult to understand.

We will discuss commenting along with documentation in a later chapter, but here are some guidelines:

- Use comments to tell what application task the program is performing, not how the micro-computer executes the instructions.
- Comments should say things like “is temperature above limit?”, “linefeed to TTY,” or “examine load switch.”
- Comments should not say things like “add 1 to Accumulator,” “jump to Start,” or “look at carry.” You should describe how the program is affecting the system; internal effects on the CPU should be obvious from the code.
- Keep comments brief and to the point. Details should be available elsewhere in the documentation.
- Comment all key points.
- Do not comment standard instructions or sequences that change counters or pointers; pay special attention to instructions that may not have an obvious meaning.
- Do not use obscure abbreviations.
- Make the comments neat and readable.
- Comment all definitions, describing their purposes. Also mark all tables and data storage areas.
- Comment sections of the program as well as individual instructions.
- Be consistent in your terminology. You can (should) be repetitive, you need not consult a thesaurus.

- Leave yourself notes at points that you find confusing: for example, “remember carry was set by last instruction.” If such points get cleared up later in program development, you may drop these comments in the final documentation.

A well-commented program is easy to use. You will recover the time spent in commenting many times over. We will try to show good commenting style in the programming examples, although we often over-comment for instructional purposes.

2.6 Types of Assemblers

Although all assemblers perform the same tasks, their implementations vary greatly. We will not try to describe all the existing types of assemblers, we will merely define the terms and indicate some of the choices.

A *cross-assembler* is an assembler that runs on a computer other than the one for which it assembles object programs. The computer on which the cross-assembler runs is typically a large computer with extensive software support and fast peripherals. The computer for which the cross-assembler assembles programs is typically a micro like the 6809 or MC68000.

When a new microcomputer is introduced, a cross-assembler is often provided to run on existing development systems. For example, ARM provide the 'Armulator' cross-assembler that will run on a PC development system.

A *self-assembler* or *resident assembler* is an assembler that runs on the computer for which it assembles programs. The self-assembler will require some memory and peripherals, and it may run quite slowly compared to a cross-assembler.

A *macroassembler* is an assembler that allows you to define sequences of instructions as macros.

A *microassembler* is an assembler used to write the microprograms which define the instruction set of a computer. Microprogramming has nothing specifically to do with programming microcomputers, but has to do with the internal operation of the computer.

A *meta-assembler* is an assembler that can handle many different instruction sets. The user must define the particular instruction set being used.

A *one-pass assembler* is an assembler that goes through the assembly language program only once. Such an assembler must have some way of resolving forward references, for example, Jump instructions which use labels that have not yet been defined.

A *two-pass assembler* is an assembler that goes through the assembly language source program twice. The first time the assembler simply collects and defines all the symbols; the second time it replaces the references with the actual definitions. A two-pass assembler has no problems with forward references but may be quite slow if no backup storage (like a floppy disk) is available; then the assembler must physically read the program twice from a slow input medium (like a teletypewriter paper tape reader). Most microprocessor-based assemblers require two passes.

2.7 Errors

Assemblers normally provide error messages, often consisting of an error code number. Some typical errors are:

Undefined name often a misspelling or an omitted definition

Illegal character such as a 2 in a binary number

Illegal format wrong delimiter or incorrect operands

Invalid expression for example, two operators in a row

Illegal value usually too large

Missing operand

Double definition two different values assigned to one name

Illegal label such as a label on a pseudo-operation that cannot have one

Missing label

Undefined operation code

In interpreting assembler errors, you must remember that the assembler may get on the wrong track if it finds a stray letter, an extra space, or incorrect punctuation. The assembler will then proceed to misinterpret the succeeding instructions and produce meaningless error messages. Always look at the first error very carefully; subsequent ones may depend on it. Caution and consistent adherence to standard formats will eliminate many annoying mistakes.

2.8 Loaders

The loader is the program which actually takes the output (object code) from the assembler and places it in memory. Loaders range from the very simple to the very complex. We will describe a few different types.

A *bootstrap loader* is a program that uses its own first few instructions to load the rest of itself or another loader program into memory. The bootstrap loader may be in ROM, or you may have to enter it into the computer memory using front panel switches. The assembler may place a bootstrap loader at the start of the object program that it produces.

A *relocating loader* can load programs anywhere in memory. It typically loads each program into the memory space immediately following that used by the previous program. The programs, however, must themselves be capable of being moved around in this way; that is, they must be relocatable. An *absolute loader*, in contrast, will always place the programs in the same area of memory.

A *linking loader* loads programs and subroutines that have been assembled separately; it resolves cross-references — that is, instructions in one program that refer to a label in another program. Object programs loaded by a linking loader must be created by an assembler that allows external references. An alternative approach is to separate the linking and loading functions and have the linking performed by a program called a *link editor* and the loading done by a loader.

3 *ARM Architecture*

This chapter outlines the ARM processor's architecture and describes the syntax rules of the ARM assembler. Later chapters of this book describe the ARM's stack and exception processing system in more detail.

Figure 3.1 on the following page shows the internal structure of the ARM processor. The ARM is a *Reduced Instruction Set Computer* (RISC) system and includes the attributes typical to that type of system:

- A large array of uniform registers.
- A load/store model of data-processing where operations can only operate on registers and not directly on memory. This requires that all data be loaded into registers before an operation can be performed, the result can then be used for further processing or stored back into memory.
- A small number of addressing modes with all load/store addresses being determined from registers and instruction fields only.
- A uniform fixed length instruction (32-bit).

In addition to these traditional features of a RISC system the ARM provides a number of additional features:

- Separate *Arithmetic Logic Unit* (ALU) and shifter giving additional control over data processing to maximize execution speed.
- Auto-increment and Auto-decrement addressing modes to improve the operation of program loops.
- Conditional execution of instructions to reduce pipeline flushing and thus increase execution speed.

3.1 Processor modes

The ARM supports the seven processor modes shown in table 3.1.

Mode changes can be made under software control, or can be caused by external interrupts or exception processing.

Most application programs execute in User mode. While the processor is in User mode, the program being executed is unable to access some protected system resources or to change mode, other than by causing an exception to occur (see 3.4 on page 29). This allows a suitably written operating system to control the use of system resources.

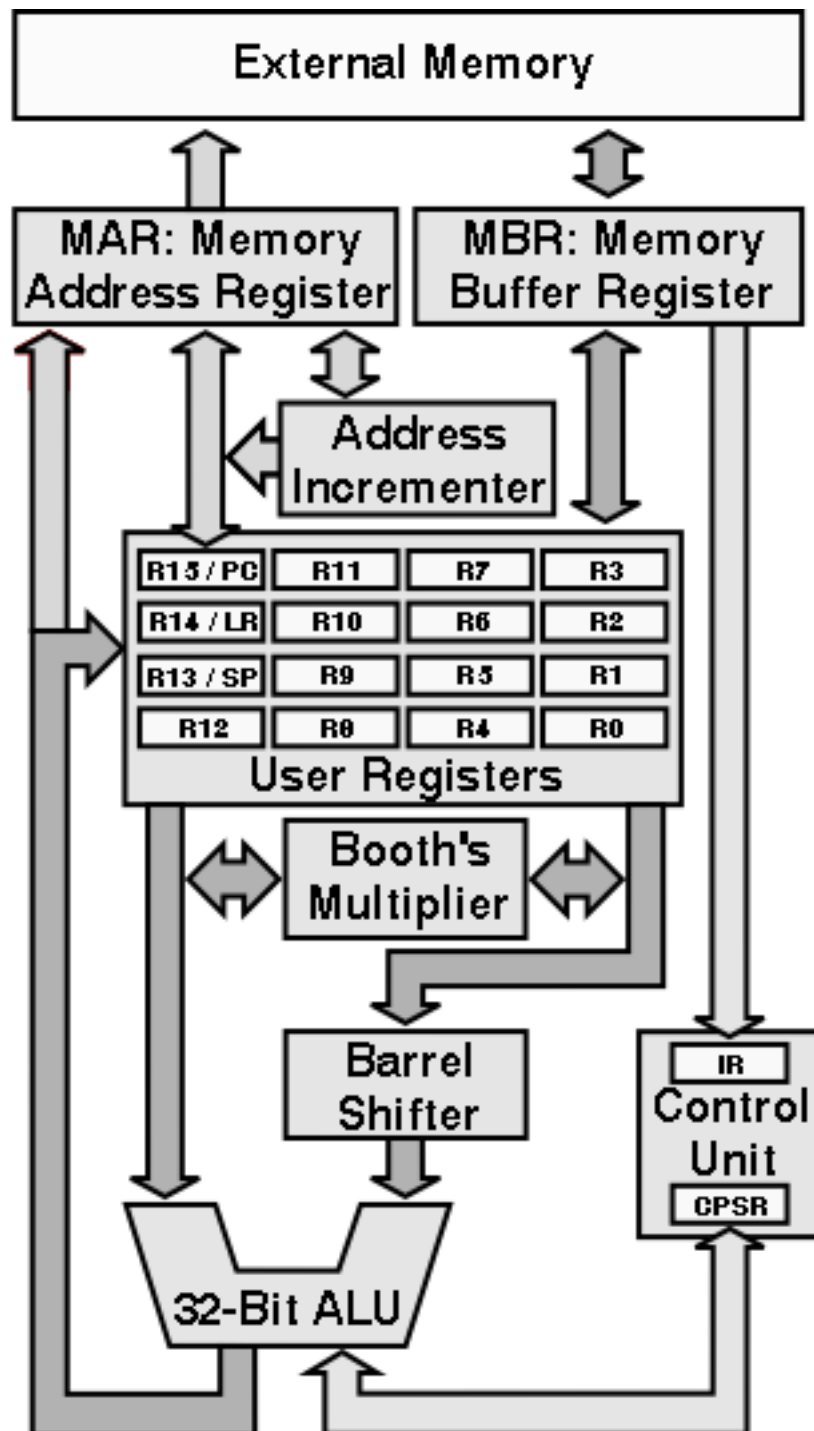


Figure 3.1: ARM Block Diagram

Processor mode		Description
User	usr	Normal program execution mode
FIQ	fiq	Fast Interrupt for high-speed data transfer
IRQ	irq	Used for general-purpose interrupt handling
Supervisor	svc	A protected mode for the operating system
Abort	abt	Implements virtual memory and/or memory protection
Undefined	und	Supports software emulation of hardware coprocessors
System	sys	Runs privileged operating system tasks

Table 3.1: ARM processor modes

The modes other than User mode are known as *privileged modes*. They have full access to system resources and can change mode freely. Five of them are known as *exception modes*: FIQ (Fast Interrupt), IRQ (Interrupt), Supervisor, Abort, and Undefined. These are entered when specific exceptions occur. Each of them has some additional registers to avoid corrupting User mode state when the exception occurs (see 3.2 for details).

The remaining mode is System mode, it is not entered by any exception and has exactly the same registers available as User mode. However, it is a privileged mode and is therefore not subject to the User mode restrictions. It is intended for use by operating system tasks which need access to system resources, but wish to avoid using the additional registers associated with the exception modes. Avoiding such use ensures that the task state is not corrupted by the occurrence of any exception.

3.2 Registers

The ARM has a total of 37 registers. These comprise 30 general purpose registers, 6 status registers and a program counter. Figure 3.2 illustrates the registers of the ARM. Only fifteen of the general purpose registers are available at any one time depending on the processor mode.

There are a standard set of eight general purpose registers that are always available (R0 – R7) no matter which mode the processor is in. These registers are truly general-purpose, with no special uses being placed on them by the processors’ architecture.

A few registers (R8 – reg12) are common to all processor modes with the exception of the fiq mode. This means that to all intent and purpose these are general registers and have no special use. However, when the processor is in the fast interrupt mode these registers are replaced with different set of registers (R8_fiq - R12_fiq). Although the processor does not give any special purpose to these registers they can be used to hold information between fast interrupts. You can consider they to be **static** registers. The idea is that you can make a fast interrupt even faster by holding information in these registers.

The general purpose registers can be used to handle 8-bit bytes, 16-bit half-words¹, or 32-bit words. When we use a 32-bit register in a byte instruction only the least significant 8 bits are used. In a half-word instruction only the least significant 16 bits are used. Figure 3.3 demonstrates this.

The remaining registers (R13 – R15) are special purpose registers and have very specific roles: R13 is also known as the Stack Pointer, while R14 is known as the Link Register, and R15 is the Program Counter. The “user” (usr) and “System” (sys) modes share the same registers. The exception modes all have their own version of these registers. Making a reference to register R14 will assume you are referring to the register for the current processor mode. If you wish to refer to

¹ Although the ARM does allow for Half-Word instructions, the emulator we are using does not.

Modes						
Privileged Modes						
Exception Modes						
User	System	Supervisor	Abort	Undefined	Interrupt	Fast Interrupt
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
PC	PC	PC	PC	PC	PC	PC

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

Figure 3.2: Register Organization

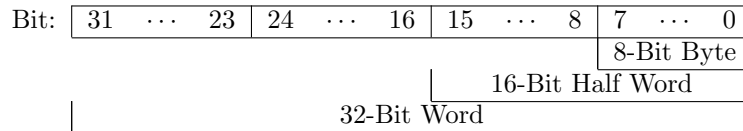


Figure 3.3: Byte/Half Word/Word

the user mode version of this register you have refer to the *R14_usr* register. You may only refer to register from other modes when the processor is in one of the privileged modes, i.e., any mode other than user mode.

There are also one or two status registers depending on which mode the processor is in. The Current Processor Status Register (CPSR) holds information about the current status of the processor (including its current mode). In the exception modes there is an additional Saved Processor Status Register (SPSR) which holds information on the processors state before the system changed into this mode, i.e., the processor status just before an exception.

3.2.1 The stack pointer, SP or R13

Register *R13* is used as a stack pointer and is also known as the *SP* register. Each exception mode has its own version of *R13*, which points to a stack dedicated to that exception mode.

The stack is typically used to store temporary values. It is normal to store the contents of any registers a function is going to use on the stack on entry to a subroutine. This leaves the register free for use during the function. The routine can then recover the register values from the stack

on exit from the subroutine. In this way the subroutine can preserve the value of the register and not corrupt the value as would otherwise be the case.

See Chapter 12 for more information on using the stack.

3.2.2 The Link Register, LR or R14

Register *R14* is also known as the *Link Register* or LR.

It is used to hold the return address for a subroutine. When a subroutine call is performed via a BL instruction, *R14* is set to the address of the next instruction. To return from a subroutine you need to copy the Link Register into the Program Counter. This is typically done in one of the two ways:

- Execute either of these instructions:

```
MOV    PC, LR
```

or

```
B      LR
```

- On entry to the subroutine store *R14* to the stack with an instruction of the form:

```
STMFD   SP!, {<registers>, LR}
```

and use a matching instruction to return from the subroutine:

```
LDMFD   SP!, {<registers>, PC}
```

This saves the Link Register on the stack at the start of the subroutine. On exit from the subroutine it collects all the values it placed on the stack, including the return address that was in the Link Register, except it returns this address directly into the Program Counter instead.

See Chapter 12 for further details on subroutines and using the stack.

When an exception occurs, the exception mode's version of *R14* is set to the address after the instruction which has just been completed. The *SPSR* is a copy of the *CPSR* just before the exception occurred. The return from an exception is performed in a similar way to a subroutine return, but using slightly different instructions to ensure full restoration of the state of the program that was being executed when the exception occurred. See 3.4 on page 29 for more details.

3.2.3 The program counter, PC or R15

Register *R15* holds the *Program Counter* known as the PC. It is used to identify which instruction is to be performed *next*. As the PC holds the address of the next instruction it is often referred to as an *instruction pointer*. The name “program counter” dates back to the times when program instructions were read in off of punched cards, it refers to the card position within a stack of cards. In spite of its name it does not actually count anything!

Reading the program counter

When an instruction reads the PC the value returned is the address of the current instruction plus 8 bytes. This is the address of the instruction *after* the *next* instruction to be executed².

This way of reading the PC is primarily used for quick, position-independent addressing of nearby instructions and data, including position-independent branching within a program.

An exception to this rule occurs when an **STR** (Store Register) or **STM** (Store Multiple Registers) instruction stores **R15**. The value stored is UNKNOWN and it is best to avoid the use of these instructions that store **R15**.

Writing the program counter

When an instruction writes to **R15** the normal result is that the value written is treated as an instruction address and the system starts to execute the instruction at that address³.

3.2.4 Current Processor Status Registers: CPSR

Rather surprisingly the *current processor status register* (CPSR) contains the current status of the processor. This includes various condition code flags, interrupt status, processor mode and other status and control information.

The exception modes also have a *saved processor status register* (SPSR), that is used to preserve the value of the CPSR when the associated exception occurs. Because the User and System modes are not exception modes, there is no **SPSR** available.

Figure 3.4 shows the format of the CPSR and the SPSR registers.

31	30	29	28	27	...	8	7	6	5	4	...	0
N	Z	C	V	SBZ			I	F	SBZ	Mode		

Figure 3.4: Structure of the Processor Status Registers

The processors' status is split into two distinct parts: the User flags and the Systems Control flags. The upper halfword is accessible in User mode and contains a set of flags which can be used to effect the operation of a program, see section 3.3. The lower halfword contains the System Control information.

Any bit not currently used is reserved for future use and should be zero, and are marked SBZ in the figure. The I and F bits indicate if Interrupts (I) or Fast Interrupts (F) are allowed. The Mode bits indicate which operating mode the processor is in (see 3.1 on page 23).

The system flags can only be altered when the processor is in protected mode. User mode programs can not alter the status register except for the condition code flags.

3.3 Flags

The upper four bits of the status register contains a set of four flags, collectively known as the *condition code*. The condition code flags are:

² This is caused by the processor having already fetched the next instruction from memory while it was deciding what the current instruction was. Thus the PC is still the next instruction to be executed, but that is not the instruction immediately after the current one.

³ As the processor has already fetched the instruction after the current instruction it is required to flush the instruction cache and start again. This will cause a short, but not significant, delay.

Negative	(N)
Zero	(Z)
Carry	(C)
Overflow	(V)

The condition code can be used to control the flow of the program execution. The is often abbreviated to just $\langle cc \rangle$.

- N** The Negative (sign) flag takes on the value of the most significant bit of a result. Thus when an operation produces a negative result the negative flag is set and a positive result results in a the negative flag being reset. This assumes the values are in standard two's complement form. If the values are unsigned the negative flag can be ignored or used to identify the value of the most significant bit of the result.
- Z** The Zero flag is set when an operation produces a zero result. It is reset when an operation produces a non-zero result.
- C** The Carry flag holds the carry from the most significant bit produced by arithmetic operations or shifts. As with most processors, the carry flag is inverted after a subtraction so that the flag acts as a borrow flag after a subtraction.
- V** The Overflow flag is set when an arithmetic result is greater than can be represented in a register.

Many instructions can modify the flags, these include comparison, arithmetic, logical and move instructions. Most of the instructions have an **S** qualifier which instructs the processor to set the condition code flags or not.

3.4 Exceptions

Exceptions are generated by internal and external sources to cause the processor to handle an event, such as an externally generated interrupt or an attempt to execute an undefined instruction. The ARM supports seven types of exception, and a provides a privileged processing mode for each type. Table 3.2 lists the type of exception and the processor mode associated with it.

When an exception occurs, some of the standard registers are replaced with registers specific to the exception mode. All exception modes have their own Stack Pointer (**SP**) and Link (**LR**) registers. The fast interrupt mode has more registers (**R8_fiq** – **R12_fiq**) for fast interrupt processing.

Exception Type	Processor Mode	
Reset	Supervisor	svc
Software Interrupt	Supervisor	svc
Undefined Instruction	Undefined	und
Prefetch Abort	Abort	abt
Data Abort	Abort	abt
Interrupt	IRQ	irq
Fast Interrupt	FIQ	fiq

Table 3.2: Exception processing modes

The seven exceptions are:

Reset when the Reset pin is held low, this is normally when the system is first turned on or when the reset button is pressed.

Software Interrupt is generally used to allow user mode programs to call the operating system. The user program executes a software interrupt (SWI, A.19 on page 144) instruction with a argument which identifies the function the user wishes to preform.

Undefined Instruction is when an attempt is made to preform an undefined instruction. This normally happens when there is a logical error in the program and the processor starts to execute data rather than program code.

Prefetch Abort occurs when the processor attempts to access memory that does not exist or the processor has executed the breakpoint (BKPT) instruction, ?? on page ??.

Data Abort occurs when attempting to access a word on a non-word aligned boundary. The lower two bits of a memory must be zero when accessing a word.

Interrupt occurs when an external device asserts the IRQ (interrupt) pin on the processor. This can be used by external devices to request attention from the processor. An interrupt can not be interrupted with the exception of a fast interrupt.

Fast Interrupt occurs when an external device asserts the FIQ (fast interrupt) pin. This is designed to support data transfer and has sufficient private registers to remove the need for register saving in such applications. A fast interrupt can not be interrupted.

When an exception occurs, the processor halts execution after the current instruction. The state of the processor is preserved in the *Saved Processor Status Register* (SPSR) so that the original program can be resumed when the exception routine has completed. The address of the instruction the processor was just about to execute is placed into the Link Register of the appropriate processor mode. The processor is now ready to begin execution of the exception handler.

The exception handler are located a pre-defined locations known as *exception vectors*. It is the responsibility of an operating system to provide suitable exception handling.

3.5 Instruction Set

Why are a microprocessor's instructions referred to as an instruction set? Because the microprocessor designer selects the instruction complement with great care; it must be easy to execute complex operations as a sequence of simple events, each of which is represented by one instruction from a well-designed instruction set.

Assembler often frighten users who are new to programming. Yet taken in isolation, the operations involved in the execution of a single instruction are usually easy to follow. Furthermore, you need not attempt to understand all the instructions at once. As you study each of the programs in these notes you will learn about the specific instructions involved.

Table 3.3 lists the instruction mnemonics. This provides a survey of the processors capabilities, and will also be useful when you need a certain kind of operation but are either unsure of the specific mnemonics or not yet familiar with what instructions are available.

See Chapter 4 and Appendix A for a detailed description of the individual instructions and chapters 6 through to 12 for a discussion on how to use them.

The ARM instruction set can be divided into six broad classes of instruction.

- Data Movement
- Arithmetic
- Memory Access
- Logical and Bit Manipulation
- Flow Control
- System Control / Privileged

Operation Mnemonic	Meaning	Operation Mnemonic	Meaning
ADC	Add with Carry	ORR	Logical OR
ADD	Add	RSB	Reverse Subtract
AND	Logical AND	RSC	Reverse Subtract with Carry
B	Unconditional Branch	SBC	Subtract with Carry
Bcc	Branch on Condition	SMLAL	Mult Accum Signed Long
BIC	Bit Clear	SMULL	Multiply Signed Long
BL	Branch and Link	STM	Store Multiple
CMP	Compare	STR	Store Register (Word)
EOR	Exclusive OR	STRB	Store Register (Byte)
LDM	Load Multiple	SUB	Subtract
LDR	Load Register (Word)	SWI	Software Interrupt
LDRB	Load Register (Byte)	SWP	Swap Word Value
MLA	Multiply Accumulate	SWPB	Swap Byte Value
MOV	Move	TEQ	Test Equivalence
MRS	Load SPSR or CPSR	TST	Test
MSR	Store to SPSR or CPSR	UMLAL	Mult Accum Unsigned Long
MUL	Multiply	UMULL	Multiply Unsigned Long
MVN	Logical NOT		

Table 3.3: Instruction Mnemonics

Mnemonic	Condition	Mnemonic	Condition
CS	<i>Carry Set</i>	CC	<i>Carry Clear</i>
EQ	<i>Equal (Zero Set)</i>	NE	<i>Not Equal (Zero Clear)</i>
VS	<i>Overflow Set</i>	VC	<i>Overflow Clear</i>
GT	<i>Greater Than</i>	LT	<i>Less Than</i>
GE	<i>Greater Than or Equal</i>	LE	<i>Less Than or Equal</i>
PL	<i>Plus (Positive)</i>	MI	<i>Minus (Negative)</i>
HI	<i>Higher Than</i>	LO	<i>Lower Than (aka CC)</i>
HS	<i>Higher or Same (aka CS)</i>	LS	<i>Lower or Same</i>

Table 3.4: $\langle cc \rangle$ (Condition code) Mnemonics

Before we look at each of these groups in a little more detail there are a few ideas which belong to all groups worthy of investigation.

3.5.1 Conditional Execution: $\langle cc \rangle$

Almost all ARM instructions contain a *condition* field which allows it to be executed conditionally dependent on the condition code flags (3.3 on page 28). If the flags indicate that the corresponding condition is true when the instruction starts executing, it executes normally. Otherwise, the instruction does nothing.

Table 3.4 shows a list of the condition codes and their mnemonics. To indicate that an instruction is conditional we simply place the mnemonic for the condition code after the mnemonic for the instruction. If no condition code mnemonic is used the instruction will always be executed.

For example the following instruction will move the value of the register *R1* into the *R0* register only when the Carry flag has been set, *R0* will remain unaffected if the C flag was clear.

```
MOVCS    R0, R1
```

Note that the *Greater* and the *Less* conditions are for use with signed numbers while the *Higher* and *Lower* conditions are for use with unsigned numbers. These condition codes only really make sense after a comparison (CMP) instruction, see A.6 on page 138.

Most data-processing instructions can also update the condition codes according to their result. Placing an “S” after the mnemonic will cause the flags to be updated. For example there are two versions of the MOV instruction:

```
MOV    R0, #0    Will move the value 0 into the register R0 without setting the flags.
MOVS   R0, #0    Will do the same, move the value 0 into the register R0, but it will also set
                  the condition code flags accordingly, the Zero flag will be set, the Negative flag
                  will be reset and the Carry and oVerflow flags will not be effected.
```

If an instruction has this ability we denote it using $\langle S \rangle$ in our description of the instruction. The $\langle S \rangle$ always comes after the $\langle cc \rangle$ (conditional execution) modification if it is given. Thus the full description of the move instruction would be:

$\text{MOV}\langle cc \rangle\langle S \rangle \quad Rd, \langle op1 \rangle$

With all this in mind what does the following code fragment do?

```
MOVS    R0, R1
MOVEQS  R0, R2
MOVEQ   R0, R3
```

The first instruction will move *R1* into *R0* unconditionally, but it will also set the N and Z flags accordingly. Thus the second instruction is only executed if the Z flag is set, i.e., the value of *R1* was zero. If the value of *R1* was not zero the instruction is skipped. If the second instruction is executed it will copy the value of *R2* into *R0* and it will also set the N and Z flags according to the value of *R2*. Thus the third instruction is only executed if both *R1* and *R2* are both zero.

3.5.2 Data Processing Operands: $\langle op1 \rangle$

The majority of the instructions relate to data processing of some form. One of the operands to these instructions is routed through the Barrel Shifter. This means that the operand can be modified before it is used. This can be very useful when dealing with lists, tables and other complex data structures. We denote instructions of this type as taking one of its arguments from $\langle op1 \rangle$.

An $\langle op1 \rangle$ argument may come from one of two sources, a constant value or a register, and be modified in five different ways. See Chapter 5 for more detailed information.

Unmodified Value

You can use a value or a register unmodified by simply giving the value or the register name. For example the following instructions will demonstrate the two methods:

```
MOV    R0, #1234    Will move the immediate constant value 123410 into the register R0
MOV    R0, R1        Will move the value in the register R1 into the register R0
```

Logical Shift Left

This will take the value of a register and shift the value up, towards the most significant bit, by n bits. The number of bits to shift is specified by either a constant value or another register. The lower bits of the value are replaced with a zero. This is a simple way of performing a multiply by a power of 2 ($\times 2^n$).

MOV R0, R1, LSL #2 R0 will become the value of R1 shifted left by 2 bits. The value of R1 is not changed.

MOV R0, R1, LSL R2 R0 will become the value of R1 shifted left by the number of bits specified in the R2 register. R0 is the only register to change, both R1 and R2 are not effected by this operation.

If the instruction is to set the status register, the carry flag (C) is the last bit that was shifted out of the value.

Logical Shift Right

Logical Shift Right is very similar to Logical Shift Left except it will shift the value to the right, towards the least significant bit, by n bits. It will replace the upper bits with zeros, thus providing an efficient unsigned divide by 2^n function ($| \div 2^n |$). The number of bits to shift may be specified by either a constant value or another register.

MOV R0, R1, LSR #2 R0 will take on the value of R1 shifted to the right by 2 bits. The value of R1 is not changed.

MOV R0, R1, LSR R2 As before R0 will become the value of R1 shifted to the right by the number of bits specified in the R2 register. R1 and R2 are not altered by this operation.

If the instruction is to set the status register, the carry flag (C) is the last bit to be shifted out of the value.

Arithmetic Shift Right

The Arithmetic Shift Right is rather similar to the Logical Shift Right, but rather than replacing the upper bits with a zero, it maintains the value of the most significant bit. As the most significant bit is used to hold the sign, this means the sign of the value is maintained, thus providing a signed divide by 2^n operation ($\div 2^n$).

MOV R0, R1, ASR #2 Register R0 will become the value of register R1 shifted to the right by 2 bits, with the sign maintained.

MOV R0, R1, ASR R2 Register R0 will become the value of the register R1 shifted to the right by the number of bits specified by the R2 register. R1 and R2 are not altered by this operation.

Given the distinction between the Logical and Arithmetic Shift Right, why is there no Arithmetic Shift Left operation?

As a signed number is stored in two's complement the upper most bits hold the sign of the number. These bits can be considered insignificant unless the number is of a sufficient size to require their use. Thus an Arithmetic Shift Left is not required as the sign is automatically preserved by the Logical Shift.

Rotate Right

In the Rotate Right operation, the least significant bit is copied into the carry (C) flag, while the value of the C flag is copied into the most significant bit of the value. In this way none of the bits in the value are lost, but are simply moved from the lower bits to the upper bits of the value.

MOV R0, R1, ROR #2 This will rotate the value of *R1* by two bits. The most significant bit of the resulting value will be the same as the least significant bit of the original value. The second most significant bit will be the same as the Carry flag. In the *S* version the Carry flag will be set to the second least significant bit of the original value. The value of *R1* is not changed by this operation.

MOV R0, R1, ROR R2 Register *R0* will become the value of the register *R1* rotated to the right by the number of bits specified by the *R2* register. *R1* and *R2* are not altered by this operation.

Why is there no corresponding Rotate Left operation?

An Add With Carry (ADC, A.2 on page 136) to a zero value provides this service for a single bit. The designers of the instruction set believe that a Rotate Left by more than one bit would never be required, thus they have not provided a ROL function.

Rotate Right Extended

This is similar to a Rotate Right by one bit. The *extended* section of the fact that this function moves the value of the Carry (C) flag into the most significant bit of the value, and the least significant bit of the value into the Carry (C) flag. Thus it allows the Carry flag to be propagated through multi-word values, thereby allowing values larger than 32-bits to be used in calculations.

MOV R0, R1, RRX The register *R0* become the same as the value of the register *R1* rotated through the carry flag by one bit. The most significant bit of the value becomes the same as the current Carry flag, while the Carry flag will be the same as the least significant bit of *R1*. The value of *R1* will not be changed.

3.5.3 Memory Access Operands: $\langle op2 \rangle$

The memory address used in the memory access instructions may also be modified by the barrel shifter. This provides for more advanced access to memory which is particularly useful when dealing with more advanced data structures. It allows pre- and post-increment instructions that update memory pointers as a side effect of the instruction. This makes loops which pass through memory more efficient. We denote instructions of this type as taking one of its arguments from $\langle op2 \rangle$. For a full discussion of the $\langle op2 \rangle$ addressing mode we refer the reader to Chapter 5 on page 59.

There are three main methods of specifying a memory address ($\langle op2 \rangle$), all of which include an offset value of some form. This offset can be specified in one of three ways:

Constant Value

An immediate constant value can be provided. If no offset is specified an immediate constant value of zero is assumed.

Register

The offset can be specified by another register. The value of the register is added to the address held in another register to form the final address.

Scaled

The offset is specified by another register which can be scaled by one of the shift operators used for $\langle op1 \rangle$. More specifically by the Logical Shift Left (LSL), Logical Shift Right (LSR), Arithmetic Shift Right (ASR), ROTate Right (ROR) or Rotate Right Extended (RRX) shift operators, where the number of bits to shift is specified as a constant value.

Offset Addressing

In *offset addressing* the memory address is formed by adding (or subtracting) an offset to or from the value held in a base register.

LDR R0, [R1]	Will load the register R0 with the 32-bit word at the memory address held in the register R1. In this instruction there is no offset specified, so an offset of zero is assumed. The value of R1 is not changed in this instruction.
LDR R0, [R1, #4]	Will load the register R0 with the word at the memory address calculated by adding the constant value 4 to the memory address contained in the R1 register. The register R1 is not changed by this instruction.
LDR R0, [R1, R2]	Loads the register R0 with the value at the memory address calculated by adding the value in the register R1 to the value held in the register R2. Both R1 and R2 are not altered by this operation.
LDR R0, [R1, R2, LSL #2]	Will load the register R0 with the 32-bit value at the memory address calculated by adding the value in the R1 register to the value obtained by shifting the value in R2 left by 2 bits. Both registers, R1 and R2 are not effected by this operation.

This is particularly useful for indexing into a complex data structure. The start of the data structure is held in a *base* register, R1 in this case, and the offset to access a particular field within the structure is then added to the base address. Placing the offset in a register allows it to be calculated at run time rather than fixed. This allows for looping through a table.

A scaled value can also be used to access a particular item of a table, where the size of the item is a power of two. For example, to locate item 7 in a table of 32-bit values we need only shift the index value 6 left by 2 bits (6×2^2) to calculate the value we need to add as an offset to the start of the table held in a register, R1 in our example. Remember that the computer count from zero, thus we use an index value of 6 rather than 7. A 32-bit number requires 4 bytes of storage which is 2^2 , thus we only need a 2-bit left shift.

Pre-Index Addressing

In *pre-index addressing* the memory address is formed in the same way as for offset addressing. The address is not only used to access memory, but the base register is also modified to hold the new value. In the ARM system this is known as a *write-back* and is denoted by placing an exclamation mark after the end of the $\langle op2 \rangle$ code.

Pre-Index address can be particularly useful in a loop as it can be used to automatically increment or decrement a counter or memory pointer.

LDR R0, [R1, #4] !	Will load the register <i>R0</i> with the word at the memory address calculated by adding the constant value 4 to the memory address contained in the <i>R1</i> register. The new memory address is placed back into the base register, register <i>R1</i> .
LDR R0, [R1, R2] !	Loads the register <i>R0</i> with the value at the memory address calculated by adding the value in the register <i>R1</i> to the value held in the register <i>R2</i> . The offset register, <i>R2</i> , is not altered by this operation, the register holding the base address, <i>R1</i> , is modified to hold the new address.
LDR R0, [R1, R2, LSL #2] !	First calculates the new address by adding the value in the base address register, <i>R1</i> , to the value obtained by shifting the value in the offset register, <i>R2</i> , left by 2 bits. It will then load the 32-bit at this address into the destination register, <i>R0</i> . The new address is also written back into the base register, <i>R1</i> . The offset register, <i>R2</i> , will not be effected by this operation.

Post-Index Addressing

In *post-index address* the memory address is the base register value. As a side-effect, an offset is added to or subtracted from the base register value and the result is written back to the base register.

Post-index addressing uses the value of the base register without modification. It then applies the modification to the address and writes the new address back into the base register. This can be used to automatically increment or decrement a memory pointer after it has been used, so it is pointing to the next location to be used.

As the instruction must preform a write-back we do not need to include an exclamation mark. Rather we move the closing bracket to include only the base register, as that is the register holding the memory address we are going to access.

LDR R0, [R1], #4	Will load the register <i>R0</i> with the word at the memory address contained in the base register, <i>R1</i> . It will then calculate the new value of <i>R1</i> by adding the constant value 4 to the current value of <i>R1</i> .
LDR R0, [R1], R2	Loads the register <i>R0</i> with the value at the memory address held in the base register, <i>R1</i> . It will then calculate the new value for the base register by adding the value in the offset register, <i>R2</i> , to the current value of the base register. The offset register, <i>R2</i> , is not altered by this operation.
LDR R0, [R1], R2, LSL #2	First loads the 32-bit value at the memory address contained in the base register, <i>R1</i> , into the destination register, <i>R0</i> . It will then calculate the new value for the base register by adding the current value to the value obtained by shifting the value in the offset register, <i>R2</i> , left by 2 bits. The offset register, <i>R2</i> , will not be effected by this operation.

4 *The ARM Instruction Set*

This chapter describes the ARM instruction set and contains the following sections:

- Instruction set encoding
- The condition field
- Branch instructions
- Data-processing instructions
- Multiply instructions
- Miscellaneous arithmetic instructions
- Status register access instructions
- Load and store instructions
- Load and Store Multiple instructions
- Semaphore instructions
- Exception-generating instructions
- Coprocessor instructions
- Extending the instruction set

4.1 Instruction set encoding

Figure 3-1 shows the ARM instruction set encoding.

All other bit patterns are UNPREDICTABLE or UNDEFINED. See *Extending the instruction set* for a description of the cases where instructions are UNDEFINED.

An entry in square brackets, for example [1], indicates that more information is given after the figure.

Data processing immediate shift	$\langle cond \rangle [1] 000 \langle opcode \rangle S R \langle n \rangle R \langle d \rangle \langle shift\ amount \rangle \langle shift \rangle 0 R \langle m \rangle$
Miscellaneous instructions	$\langle cond \rangle [1] 00010xx0xxxxxx0xxxxxx$
Data processing register shift [2]	$\langle cond \rangle [1] 000 \langle opcode \rangle S R \langle n \rangle R \langle d \rangle R \langle s \rangle 0 \langle shift \rangle 1 R \langle m \rangle$
Miscellaneous Instructions	$\langle cond \rangle [1] 00010xx0xxxxxx0xxxxxx$
Multiplies, extra load/stores	$\langle cond \rangle [1] 000xxxxxx0xxxxxx1xxxx$
Data processing immediate [2]	$\langle cond \rangle [1] 001 \langle opcode \rangle S R \langle n \rangle R \langle d \rangle \langle rotate \rangle \langle immediate \rangle$
Undefined instruction [3]	$\langle cond \rangle [1] 00110x00xxxxxx0xxxxxx$
Move immediate to status register	$\langle cond \rangle [1] 00110R10 \langle Mask \rangle \langle SBO \rangle \langle rotate \rangle \langle immediate \rangle$
Load/store immediate offset	$\langle cond \rangle [1] 010PUBWL R \langle n \rangle R \langle d \rangle \langle immediate \rangle$
Load/store register offset	$\langle cond \rangle [1] 011PUBWL R \langle n \rangle R \langle d \rangle \langle shift\ amount \rangle \langle shift \rangle 0 R \langle m \rangle$
Undefined instruction	$\langle cond \rangle [1] 011xxxxxx0xxxxxx1xxxx$
Undefined instruction [4,7]	$11110xxxxxx0xxxxxx$
Load/store multiple	$\langle cond \rangle [1] 100PU SWL R \langle n \rangle \langle register\ list \rangle$
Undefined instruction [4]	$11111000xxxxxx0xxxxxx$
Branch and branch with link	$\langle cond \rangle [1] 101L \langle 24\text{-bit}\ offset \rangle$
Branch and change to Thumb [4]	$1111101H \langle 24\text{-bit}\ offset \rangle$
Copro load/store [6]	$\langle cond \rangle [5] 110PU NWL R \langle n \rangle CR \langle d \rangle \langle cp_num \rangle \langle 8\text{-bit}\ offset \rangle$
Copro data processing	$\langle cond \rangle [5] 1110 \langle opcode1 \rangle CR \langle n \rangle CR \langle d \rangle \langle cp_num \rangle \langle opcode2 \rangle 0 CR \langle m \rangle$
Copro register transfer	$\langle cond \rangle [5] 1110 \langle opcode1 \rangle L CR \langle n \rangle R \langle d \rangle \langle cp_num \rangle \langle opcode2 \rangle 1 CR \langle m \rangle$
Software interrupt	$\langle cond \rangle [1] 1111 \langle swi\ number \rangle$
Undefined [4]	$11111111xxxxxx0xxxxxx$

Figure 3-1 The ARM Instruction Set Summary

- 1 The $\langle cond \rangle$ field is not allowed to be 1111 in this line. Other lines deal with the cases where bits[31:28] of the instruction are 1111.
- 2 If the $\langle opcode \rangle$ field is of the form 10xx and the S field is 0, one of the following lines applies instead.
- 3 UNPREDICTABLE prior to ARM architecture version 4.
- 4 UNPREDICTABLE prior to ARM architecture version 5.
- 5 If the $\langle cond \rangle$ field is 1111, this instruction is UNPREDICTABLE prior to ARM architecture version 5.
- 6 The coprocessor double register transfer instructions are described in Chapter A10 *Enhanced DSP Extension*.
- 7 In E variants of architecture version 5 and above, the cache preload instruction PLD uses a small number of these instruction encodings.

4.1.1 Multiplies and extra load/store instructions

Figure 3-2 shows extra multiply and load/store instructions. An entry in square brackets, for example [1], indicates that more information is given below the figure.

Multiply (accumulate)	$\langle cond \rangle 000000AS R \langle d \rangle R \langle n \rangle R \langle s \rangle 1001 R \langle m \rangle$
Multiply (accumulate) long	$\langle cond \rangle 00001UAS R \langle dHi \rangle R \langle dLo \rangle R \langle s \rangle 1001 R \langle m \rangle$
Swap/swap byte	$\langle cond \rangle 00010B00 R \langle n \rangle R \langle d \rangle \langle SBZ \rangle 1001 R \langle m \rangle$
Load/store halfword register offset [1]	$\langle cond \rangle 000PU0WL R \langle n \rangle R \langle d \rangle \langle SBZ \rangle 1001 R \langle m \rangle$
Load/store halfword immediate offset [1]	$\langle cond \rangle 000PU1WL R \langle n \rangle R \langle d \rangle \langle HiOffset \rangle 1011 \langle LoOffset \rangle$
Load/store two words register offset [2]	$\langle cond \rangle 000PU0W0 R \langle n \rangle R \langle d \rangle \langle SBZ \rangle 11S1 R \langle m \rangle$
Load signed halfword/byte register offset [1]	$\langle cond \rangle 000PU0W1 R \langle n \rangle R \langle d \rangle \langle SBZ \rangle 11H1 R \langle m \rangle$
Load/store two words immediate offset [2]	$\langle cond \rangle 000PU1W0 R \langle n \rangle R \langle d \rangle \langle HiOffset \rangle 11S1 \langle LoOffset \rangle$
Load signed halfword/byte immediate offset [1]	$\langle cond \rangle 000PU1W1 R \langle n \rangle R \langle d \rangle \langle HiOffset \rangle 11H1 \langle LoOffset \rangle$

Figure 3-2 Multiplies and extra load/store instructions

- 1 UNPREDICTABLE prior to ARM architecture version 4.
- 2 These instructions are described in Chapter A10 *Enhanced DSP Extension*.

Note

Any instruction with bits[27:25] = 000, bit[7] = 1, bit[4] = 1, and $\langle cond \rangle$ not equal to 1111, and which is not specified in Figure 3-2 or its notes, is an undefined instruction (or UNPREDICTABLE prior to ARM architecture version 4).

End Note**4.1.2 Miscellaneous instructions**

Figure 3-3 shows the remaining ARM instruction encodings. An entry in square brackets, for example [1], indicates that more information is given below the figure.

Move status register to register	$\langle cond \rangle$ 0 0 0 1 0 R 0 0 $\langle SBO \rangle$ R $\langle d \rangle$ $\langle SBZ \rangle$ 0 0 0 0 $\langle SBZ \rangle$
Move register to status register	$\langle cond \rangle$ 0 0 0 1 0 R 1 0 $\langle mask \rangle$ $\langle SBO \rangle$ $\langle SBZ \rangle$ 0 0 0 0 R $\langle m \rangle$
Branch/exchange instruction set [1]	$\langle cond \rangle$ 0 0 0 1 0 0 1 0 $\langle SBO \rangle$ $\langle SBO \rangle$ $\langle SBO \rangle$ 0 0 0 1 R $\langle m \rangle$
Count leading zeros [2]	$\langle cond \rangle$ 0 0 0 1 0 1 1 0 $\langle SBO \rangle$ R $\langle d \rangle$ $\langle SBO \rangle$ 0 0 0 1 R $\langle m \rangle$
Branch and link/exchange [2]	$\langle cond \rangle$ 0 0 0 1 0 0 1 0 $\langle SBO \rangle$ $\langle SBO \rangle$ $\langle SBO \rangle$ 0 0 1 1 R $\langle m \rangle$
Enhanced DSP add/subtracts [4]	$\langle cond \rangle$ 0 0 0 1 0 $\langle op \rangle$ 0 R $\langle n \rangle$ R $\langle d \rangle$ $\langle SBZ \rangle$ 0 1 0 1 R $\langle m \rangle$
Software breakpoint [2,3]	$\langle cond \rangle$ 0 0 0 1 0 0 1 0 $\langle immed \rangle$ 0 1 1 1 $\langle immed \rangle$
Enhanced DSP multiplies[4]	$\langle cond \rangle$ 0 0 0 1 0 $\langle op \rangle$ 0 R $\langle d \rangle$ R $\langle n \rangle$ R $\langle s \rangle$ 1 y x 0 R $\langle m \rangle$

Figure 3-3 Miscellaneous instructions

- 1 Defined in ARM architecture version 5 and above, and in T variants of ARM architecture version 4.
- 2 This is an undefined instruction in ARM architecture version 4, and is UNPREDICTABLE prior to ARM architecture version 4.
- 3 If the cond field of this instruction is not 1110, it is UNPREDICTABLE.
- 4 The enhanced DSP instructions are described in Chapter A10 *Enhanced DSP Extension*.

Note

Any instruction with bits[27:23] = 00010, bit[20] = 0, bit[7] and bit[4] not both 1, and cond is not equal to 1111, and which is not specified in Figure 3-3 or its notes, is an undefined instruction (or UNPREDICTABLE prior to architecture version 4).

End Note**4.2 The condition field**

Almost all ARM instructions can be *conditionally executed*, which means that they only have their normal effect on the programmer's model state, memory and coprocessors if the N, Z, C and V flags in the CPSR satisfy a condition specified in the instruction. If the flags do not satisfy this condition, the instruction acts as a NOP: that is, execution advances to the next instruction as normal, including any relevant checks for interrupts and prefetch aborts, but has no other effect.

Prior to ARM architecture version 5, all ARM instructions could be conditionally executed. A few instructions have been introduced subsequently which can only be executed unconditionally.

Every instruction contains a 4-bit condition code field in bits 31 to 28.

This field contains one of the 16 values described in Table 3-1. Most instruction mnemonics can be extended with the letters defined in the mnemonic extension field.

If the *always* (AL) condition is specified, the instruction is executed irrespective of the value of the condition code flags. The absence of a condition code on an instruction mnemonic implies the AL condition code.

4.2.1 Condition code 0b1111

As indicated in Table 3-1, if the condition field is 0b1111, the behavior depends on the architecture version:

- Prior to ARM architecture version 3, a condition field of 0b1111 meant that the instruction was never executed. The mnemonic extension for this condition was NV.

Note

Use of this condition is now obsolete and unsupported.

End Note

- In ARM architecture version 3 and version 4, any instruction with a condition field of 0b1111 is UNPREDICTABLE.
- In ARM architecture version 5 and above, a condition field of 0b1111 is used to encode various additional instructions which can only be executed unconditionally. All instruction encoding diagrams which show bits[31:28] as $\langle cond \rangle$ only match instructions in which these bits are not equal to 0b1111, unless otherwise stated in the individual instruction description.

Table 3-1 Condition codes

Opcode [31:28]	Mnemonic extension	Meaning	Condition Flag state
0000	EQ	Equal	Z set
0001	NE	Not Equal	Z clear
0010	CS / HS	Carry Set / Unsigned Higer or Same	C set
0011	CC / LO	Carry Clear / Unsigned Lower	C clear
0100	MI	Minus / Negative	N set
0101	PL	Plus / Positive	N clear
0110	VS	Overflow Set	V set
0111	VC	Overflow Clear	V clear
1000	HI	Unsigned Higher	C set and Z clear
1001	LS	Unsigned Lower or Same	C clear or Z set
1010	GE	Signed Greater than or Equal	N set and V set, or N clear and V clear (C==V)
1011	LT	Signed Less Than	N set and V clear, or N clear and V set (C != V)
1100	GT	Signed Greater Than	Z clear, and either N set and V set, or N clear and V clear (Z==0,N==V)
1101	LE	Signed Less than or Equal	Z set, or N set and V clear, or N clear and V set (Z==1 or N!=V)
1110	AL	Always (unconditional)	—
1111	(NV)	Never	—

4.3 Branch instructions

All ARM processors support a branch instruction that allows a conditional branch forwards or backwards up to 32MB. As the PC is one of the general-purpose registers (R15), a branch or jump can also be generated by writing a value to R15.

A subroutine call can be performed by a variant of the standard branch instruction. As well as allowing a branch forward or backward up to 32MB, the Branch with Link (BL) instruction preserves the address of the instruction after the branch (the return address) in the LR (R14).

In T variants of ARM architecture version 4, and in ARM architecture version 5 and above, the Branch and Exchange (BX) instruction copies the contents of a general-purpose register Rm to the PC (like a `MOV PC,R⟨m⟩` instruction), with the additional functionality that if bit[0] of the transferred value is 1, the processor shifts to Thumb state. Together with the corresponding Thumb instructions, this allows *interworking* branches between ARM and Thumb code.

Interworking subroutine calls can be generated by combining BX with an instruction to write a suitable return address to the LR, such as an immediately preceding `MOV LR,PC` instruction.

In ARM architecture version 5 and above, there are also two types of Branch with Link and Exchange (BLX) instruction:

- One type takes a register operand Rm, like a BX instruction. This instruction behaves like a BX instruction, and additionally writes the address of the next instruction into the LR. This provides a more efficient interworking subroutine call than a sequence of `MOV LR,PC` followed by `BX R⟨m⟩`.
- The other type behaves like a BL instruction, branching backwards or forwards by up to 32MB and writing a return link to the LR, but shifts to Thumb state rather than staying in ARM state as BL does. This provides a more efficient alternative to loading the subroutine address into Rm followed by a `BLX R⟨m⟩` instruction when it is known that a Thumb subroutine is being called and that the subroutine lies within the 32MB range.

A load instruction provides a way to branch anywhere in the 4GB address space (known as a *long branch*). A 32-bit value is loaded directly from memory into the PC, causing a branch. A long branch can be preceded by `MOV LR,PC` or another instruction that writes the LR to generate a long subroutine call. In ARM architecture version 5 and above, bit[0] of the value loaded by a long branch controls whether the subroutine is executed in ARM state or Thumb state, just like bit[0] of the value moved to the PC by a BX instruction. Prior to ARM architecture version 5, bits[1:0] of the value loaded into the PC are ignored, and a load into the PC can only be used to call a subroutine in ARM state.

In non-T variants of ARM architecture version 5, the instructions described above can cause an entry into Thumb state despite the fact that the Thumb instruction set is not present. This causes the instruction at the branch target to enter the undefined instruction trap. See *The control bits* for more details.

4.3.1 Examples

```

B      label           ; branch unconditionally to label
BCC    label           ; branch to label if carry flag is clear
BEQ    label           ; branch to label if zero flag is set
MOV    PC, #0          ; R15 = 0, branch to location zero
BL     func            ; subroutine call to function
func
...
...
MOV    PC, LR          ; R15=R14, return to instruction after the BL
MOV    LR, PC          ; store the address of the instruction
                        ; after the next one into R14 ready to return
LDR    PC, =func       ; load a 32-bit value into the program counter
```

4.3.2 List of branch instructions

- B, BL Branch, and Branch with Link. See *B*, *BL*.
 BLX Branch with Link and Exchange. See *BLX*(1) and *BLX*(2).
 BX Branch and Exchange Instruction Set. See *BX*.

4.4 Data-processing instructions

ARM has 16 data-processing instructions, shown in Table 3-2.

Table 3-2 Data-processing instructions

Opcode	Mnemonic	Operation	Action
0000	AND	Logical AND	Rd := Rn AND shifter_operand
0001	EOR	Logical Exclusive OR	Rd := Rn EOR shifter_operand
0010	SUB	Subtract	Rd := Rn - shifter_operand
0011	RSB	Reverse Subtract	Rd := shifter_operand - Rn
0100	ADD	Add	Rd := Rn + shifter_operand
0101	ADC	Add with Carry	Rd := Rn + shifter_operand + Carry Flag
0110	SBC	Subtract with Carry	Rd := Rn - shifter_operand - NOT(Carry Flag)
0111	RSC	Reverse Subtract with Carry	Rd := shifter_operand - Rn - NOT(Carry Flag)
1000	TST	Test	Update flags after Rn AND shifter_operand
1001	TEQ	Test Equivalence	Update flags after Rn EOR shifter_operand
1010	CMP	Compare	Update flags after Rn - shifter_operand
1011	CMN	Compare Negated	Update flags after Rn + shifter_operand
1100	ORR	Logical (inclusive) OR	Rd := Rn OR shifter_operand
1101	MOV	Move	Rd := shifter_operand (no first operand)
1110	BIC	Bit Clear	Rd := Rn AND NOT(shifter_operand)
1111	MNV	Move Not	Rd := NOT shifter_operand (no first operand)

Most data-processing instructions take two source operands, though Move and Move Not take only one. The compare and test instructions only update the condition flags. Other data-processing instructions store a result to a register and optionally update the condition flags as well.

Of the two source operands, one is always a register. The other is called a *shifter operand* and is either an immediate value or a register. If the second operand is a register value, it can have a shift applied to it. **CMP**, **CMN**, **TST** and **TEQ** always update the condition code flags. The assembler automatically sets the S bit in the instruction for them, and the corresponding instruction with the S bit clear is not a data-processing instruction, but instead lies in one of the instruction extension spaces (see *Extending the instruction set*). The remaining instructions update the flags if an S is appended to the instruction mnemonic (which sets the S bit in the instruction). See *The condition code flags* for more details.

4.4.1 Instruction encoding

$\langle opcode1 \rangle \{ \langle cond \rangle \} \{ S \} \langle Rd \rangle, \langle shifter operand \rangle$
 $\langle opcode1 \rangle := \text{MOV} \mid \text{MVN}$

$\langle opcode2 \rangle \{ \langle cond \rangle \} \langle Rn \rangle, \langle shifter operand \rangle$
 $\langle opcode2 \rangle := \text{CMP} \mid \text{CMN} \mid \text{TST} \mid \text{TEQ}$

$\langle opcode3 \rangle \{ \langle cond \rangle \} \{ S \} \langle Rd \rangle, \langle Rn \rangle, \langle shifter operand \rangle$
 $\langle opcode3 \rangle := \text{ADD} \mid \text{SUB} \mid \text{RSB} \mid \text{ADC} \mid \text{SBC} \mid \text{RSC} \mid \text{AND} \mid \text{BIC} \mid \text{EOR} \mid \text{ORR}$

I bit [25] Distinguishes between the immediate and register forms of *shifter operand*.

S bit [20] Signifies that the instruction updates the condition codes.

Rn [19:16] Specifies the first source operand register.

Rd [15:12] Specifies the destination register.

shifter operand [11:0] Specifies the second source operand. See *Addressing Mode 1 - Data-processing operands* for details of the shifter operands.

4.4.2 List of data-processing instructions

ADC	Add with Carry.
ADD	Add.
AND	Logical AND.
BIC	Logical Bit Clear.
CMN	Compare Negative.
CMP	Compare.
EOR	Logical EOR.
MOV	Move.
MVN	Move Negative.
ORR	Logical OR.
RSB	Reverse Subtract.
RSC	Reverse Subtract with Carry.
SBC	Subtract with Carry.
SUB	Subtract.
TEQ	Test Equivalence.
TST	Test.

4.5 Multiply instructions

ARM has two classes of Multiply instruction:

- normal, 32-bit result
- long, 64-bit result

All Multiply instructions take two register operands as the input to the multiplier. The ARM processor does not directly support a multiply-by-constant instruction due to the efficiency of shift and add, or shift and reverse subtract instructions.

4.5.1 Normal multiply

There are two Multiply instructions that produce 32-bit results:

MUL Multiplies the values of two registers together, truncates the result to 32 bits, and stores the result in a third register.

MLA Multiplies the values of two registers together, adds the value of a third register, truncates the result to 32 bits, and stores the result in a fourth register. This can be used to perform multiply-accumulate operations.

Both Multiply instructions can optionally set the N (Negative) and Z (Zero) condition code flags. No distinction is made between signed and unsigned variants. Only the least significant 32 bits of the result are stored in the destination register, and the sign of the operands does not affect this value.

4.5.2 Long multiply

There are four Multiply instructions that produce 64-bit results (long multiply).

Two of the variants multiply the values of two registers together and store the 64-bit result in third and fourth registers. There are signed **SMULL** and unsigned **UMULL** variants. The signed variants produce a different result in the most significant 32 bits if either or both of the source operands is negative.

The remaining two variants multiply the values of two registers together, add the 64-bit value from the third and fourth registers and store the 64-bit result back into those registers (third and fourth). There are signed **SMLAL** and unsigned **UMLAL** variants. These instructions perform a long multiply and accumulate.

All four long multiply instructions can optionally set the N (Negative) and Z (Zero) condition code flags.

4.5.3 Examples

MUL	R4, R2, R1	; Set R4 to value of R2 multiplied by R1
MULS	R4, R2, R1	; R4 = R2 x R1, set N and Z flags
MLA	R7, R8, R9, R3	; R7 = R8 x R9 + R3
SMULL	R4, R8, R2, R3	; R4 = bits 0 to 31 of R2 x R3
		; R8 = bits 32 to 63 of R2 x R3
UMULL	R6, R8, R0, R1	; R8, R6 = R0 x R1
UMLAL	R5, R8, R0, R1	; R8, R5 = R1 + R8, R5

4.5.4 List of multiply instructions

MLA	Multiply Accumulate.
MUL	Multiply.
SMLAL	Signed Multiply Accumulate Long.
SMULL	Signed Multiply Long.
UMLAL	Unsigned Multiply Accumulate Long.
UMULL	Unsigned Multiply Long.

4.6 Miscellaneous arithmetic instructions

In addition to the normal data-processing and multiply instructions, versions 5 and above of the ARM architecture include a Count Leading Zeros **CLZ** instruction. This instruction returns the number of 0 bits at the most significant end of its operand before the first 1 bit is encountered (or 32 if its operand is zero). Two typical applications for this are:

- To determine how many bits the operand should be shifted left in order to *normalize* it, so that its most significant bit is 1. (This can be used in integer division routines.)
- To locate the highest priority bit in a bit mask.

4.6.1 Instruction encoding

CLZ{<cond>} <Rd>, <Rm>

Rd Specifies the destination register.

Rm Specifies the operand register.

4.6.2 List of miscellaneous arithmetic instructions

CLZ Count Leading Zeros.

4.7 Status register access instructions

There are two instructions for moving the contents of a program status register to or from a general-purpose register. Both the CPSR and SPSR can be accessed.

Each status register is split into four 8-bit fields that can be individually written:

Bits[31:24]	The flags field.
Bits[23:16]	The status field.
Bits[15:8]	The extension field.
Bits[7:0]	The control field.

To date, the ARM architecture does not use the status and extension fields, and three bits are unused in the flags field. The four condition code flags occupy bits[31:28]. In E variants of architecture versions 5 and above, the Q flag occupies bit[27]. See *The Q flag* for more information on the Q flag. The control field contains two interrupt disable bits, five processor mode bits, and the Thumb bit on ARM architecture version 5 and above and on T variants of ARM architecture version 4 (see *The T bit*).

The unused bits of the status registers might be used in future ARM architectures, and must not be modified by software. Therefore, a read-modify-write strategy must be used to update the value of a status register to ensure future compatibility.

The status registers are readable to allow the read part of the read-modify-write operation, and to allow all processor state to be preserved (for instance, during process context switches).

The status registers are writable to allow the write part of the read-modify-write operation, and allow all processor state to be restored.

4.7.1 CPSR value

Altering the value of the CPSR has three uses:

- sets the value of the condition code flags (and of the Q flag when it exists) to a known value
- enables or disable interrupts
- changes processor mode (for instance, to initialize stack pointers).

Note

The T bit must not be changed directly by writing to the CPSR, but only via the BX instruction, and in the implicit SPSR to CPSR moves in instructions designed for exception return. Attempts to enter or leave Thumb state by directly altering the T bit can have UNPREDICTABLE consequences.

End Note

4.7.2 Examples

These examples assume that the ARM processor is already in a privileged mode. If the ARM processor starts in User mode, only the flag update has any effect.

```

MRS    R0, CPSR                ; Read the CPSR
BIC     R0, R0, #0xF0000000     ; Clear the N, Z, C and V bits
MSR     CPSR_f, R0              ; Update the flag bits in the CPSR
                                           ; N, Z, C and V flags now all clear

MRS     R0, CPSR                ; Read the CPSR
ORR     R0, R0, #0x80           ; Set the interrupt disable bit
MSR     CPSR_c, R0              ; Update the control bits in the CPSR
                                           ; interrupts (IRQ) now disabled

MRS     R0, CPSR                ; Read the CPSR
BIC     R0, R0, #0x1F           ; Clear the mode bits
ORR     R0, R0, #0x11           ; Set the mode bits to FIQ mode
MSR     CPSR_c, R0              ; Update the control bits in the CPSR
                                           ; now in FIQ mode

```

4.7.3 List of status register access instructions

MRS Move PSR to General-purpose Register.
 MSR Move General-purpose Register to PSR.

4.8 Load and store instructions

The ARM architecture supports two broad types of instruction which load or store the value of a single register from or to memory:

- The first type can load or store a 32-bit word or an 8-bit unsigned byte.
- The second type can load or store a 16-bit unsigned halfword, and can load and sign extend a 16-bit halfword or an 8-bit byte. This type of instruction is only available in ARM architecture version 4 and above.

4.8.1 Addressing modes

In both types of instruction, the addressing mode is formed from two parts:

- the base register
- the offset.

The base register can be any one of the general-purpose registers (including the PC, which allows PC-relative addressing for position-independent code).

The offset takes one of three formats:

Immediate The offset is an unsigned number that can be added to or subtracted from the base register. Immediate offset addressing is useful for accessing data elements that are a fixed distance from the start of the data object, such as structure fields, stack offsets and input/output registers.

For the word and unsigned byte instructions, the immediate offset is a 12-bit number. For the halfword and signed byte instructions, it is an 8-bit number.

Register The offset is a general-purpose register (not the PC), that can be added to or subtracted from the base register. Register offsets are useful for accessing arrays or blocks of data.

Scaled register The offset is a general-purpose register (not the PC) shifted by an immediate value, then added to or subtracted from the base register. The same shift operations used for data-processing instructions can be used (Logical Shift Left, Logical Shift Right, Arithmetic Shift Right and Rotate Right), but Logical Shift Left is the most useful as it allows an array indexed to be scaled by the size of each array element.

Scaled register offsets are only available for the word and unsigned byte instructions.

As well as the three types of offset, the offset and base register are used in three different ways to form the memory address. The addressing modes are described as follows:

Offset The base register and offset are added or subtracted to form the memory address.

Pre-indexed The base register and offset are added or subtracted to form the memory address. The base register is then updated with this new address, to allow automatic indexing through an array or memory block.

Post-indexed The value of the base register alone is used as the memory address. The base register and offset are added or subtracted and this value is stored back in the base register, to allow automatic indexing through an array or memory block.

4.8.2 Load and Store word or unsigned byte instructions

Load instructions load a single value from memory and write it to a general-purpose register.

Store instructions read a value from a general-purpose register and store it to memory.

Load and Store instructions have a single instruction format:

LDR|STR {<cond>} {B} {T} Rd, <addressing mode>

I, P, U, W [25] [24] [23] [21]

Are bits that distinguish between different types of <addressing mode>.

L bit [20]

Distinguishes between a Load (L==1) and a Store instruction (L==0).

B bit [22]

Distinguishes between an unsigned byte (B==1) and a word (B==0) access.

Rn [19:16]

Specifies the base register used by <addressing mode>.

Rd [15:12]

Specifies the register whose contents are to be loaded or stored.

4.8.3 Load and Store Halfword and Load Signed Byte

Load instructions load a single value from memory and write it to a general-purpose register.

Store instructions read a value from a general-purpose register and store it to memory.

Load and Store Halfword and Load Signed Byte instructions have a single instruction format:

`LDR|STR{<cond>}H|SH|SB Rd, <addressing mode>`

addr mode [11:8] and [3:0]

Are addressing-mode-specific bits.

I, P, U, W [22] [24] [23] [21]

Are bits that specify the type of addressing mode (see *Addressing Mode 3 - Miscellaneous Loads and Stores*).

L bit [20]

Distinguishes between a Load (L==1) and a Store instruction (L==0).

S bit [6]

Distinguishes between a signed (S==1) and an unsigned (S==0) halfword access. If the L bit is zero and S bit is one, the instruction is UNPREDICTABLE.

H bit [5]

Distinguishes between a halfword (H==1) and a signed byte (H==0) access. If the S bit and H bit are both zero, this instruction encodes a SWP or Multiply instruction.

Rn [19:16]

Specifies the base register used by the addressing mode.

Rd [15:12]

Specifies the register whose contents are to be loaded or stored.

4.8.4 Examples

LDR	R1, [R0]	; Load R1 from the address in R0
LDR	R8, [R3, #4]	; Load R8 from the address in R3 + 4
LDR	R12, [R13, #-4]	; Load R12 from R13 - 4
STR	R2, [R1, #0x100]	; Store R2 to the address in R1 + 0x100
LDRB	R5, [R9]	; Load byte into R5 from R9
		; (zero top 3 bytes)
LDRB	R3, [R8, #3]	; Load byte to R3 from R8 + 3
		; (zero top 3 bytes)
STRB	R4, [R10, #0x200]	; Store byte from R4 to R10 + 0x200
LDR	R11, [R1, R2]	; Load R11 from the address in R1 + R2
STRB	R10, [R7, -R4]	; Store byte from R10 to addr in R7 - R4
LDR	R11, [R3, R5, LSL #2]	; Load R11 from R3 + (R5 x 4)
LDR	R1, [R0, #4]!	; Load R1 from R0 + 4, then R0 = R0 + 4
STRB	R7, [R6, #-1]!	; Store byte from R7 to R6 - 1,
		; then R6 = R6 - 1
LDR	R3, [R9], #4	; Load R3 from R9, then R9 = R9 + 4

STR	R2, [R5], #8	; Store R2 to R5, then R5 = R5 + 8
LDR	R0, [PC, #40]	; Load R0 from PC + 0x40 (= address of ; the LDR instruction + 8 + 0x40)
LDR	R0, [R1], R2	; Load R0 from R1, then R1 = R1 + R2
LDRH	R1, [R0]	; Load halfword to R1 from R0 ; (zero top 2 bytes)
LDRH	R8, [R3, #2]	; Load halfword into R8 from R3 + 2
LDRH	R12, [R13, #-6]	; Load halfword into R12 from R13 - 6
STRH	R2, [R1, #0x80]	; Store halfword from R2 to R1 + 0x80
LDRSH	R5, [R9]	; Load signed halfword to R5 from R9
LDRSB	R3, [R8, #3]	; Load signed byte to R3 from R8 + 3
LDRSB	R4, [R10, #0xC1]	; Load signed byte to R4 from R10 + 0xC1
LDRH	R11, [R1, R2]	; Load halfword into R11 from address ; in R1 + R2
STRH	R10, [R7, -R4]	; Store halfword from R10 to R7 - R4
LDRSH	R1, [R0, #2]!	; Load signed halfword R1 from R0 + 2, ; then R0 = R0 + 2
LDRSB	R7, [R6, #-1]!	; Load signed byte to R7 from R6 - 1, ; then R6 = R6 - 1
LDRH	R3, [R9], #2	; Load halfword to R3 from R9, ; then R9 = R9 + 2
STRH	R2, [R5], #8	; Store halfword from R2 to R5, ; then R5 = R5 + 8

4.8.5 List of load and store instructions

LDR	Load Word.
LDRB	Load Byte.
LDRBT	Load Byte with User Mode Privilege.
LDRH	Load Unsigned Halfword.
LDRSB	Load Signed Byte.
LDRSH	Load Signed Halfword.
LDRT	Load Word with User Mode Privilege.
STR	Store Word.
STRB	Store Byte.
STRBT	Store Byte with User Mode Privilege.
STRH	Store Halfword.
STRT	Store Word with User Mode Privilege.

4.9 Load and Store Multiple instructions

Load Multiple instructions load a subset, or possibly all, of the general-purpose registers from memory.

Store Multiple instructions store a subset, or possibly all, of the general-purpose registers to memory.

Load and Store Multiple instructions have a single instruction format:

LDM{<cond>}<addressing mode> Rn{!}, <registers>{^}
 STM{<cond>}<addressing mode> Rn{!}, <registers>{^}

where:

<addressing mode> = IA | IB | DA | DB | FD | FA | ED | EA

registers [15:0]

The list of <registers> has one bit for each general-purpose register. Bit 0 is for R0, and bit 15 is for R15 (the PC).

The register syntax list is an opening bracket, followed by a comma-separated list of registers, followed by a closing bracket. A sequence of consecutive registers can be specified by separating the first and last registers in the range with a minus sign.

P, U, W [24] [23] [21]

These distinguish between the different types of addressing mode (see *Addressing Mode 4 - Load and Store Multiple*).

S bit [22]

For LDMs that load the PC, the S bit indicates that the CPSR is loaded from the SPSR after all the registers have been loaded. For all STMs, and LDMs that do not load the PC, it indicates that when the processor is in a privileged mode, the User mode banked registers are transferred and not the registers of the current mode.

L bit [20]

This distinguishes between a Load (L==1) and a Store (L==0) instruction.

Rn [19:16]

This specifies the base register used by the addressing mode.

4.9.1 Examples

```
STMFD    R13!, {R0 - R12, LR}
LDMFD    R13!, {R0 - R12, PC}
LDMIA    R0, {R5 - R8}
STMDA    R1!, {R2, R5, R7 - R9, R11}
```

4.9.2 List of Load and Store Multiple instructions

LDM Load Multiple.
 LDM User Registers Load Multiple.
 LDM Load Multiple with Restore CPSR.
 STM Store Multiple.
 STM User Registers Store Multiple.

4.10 Semaphore instructions

The ARM instruction set has two semaphore instructions:

- Swap (SWP)

- Swap Byte (SWPB)

These instructions are provided for process synchronization. Both instructions generate an atomic load and store operation, allowing a memory semaphore to be loaded and altered without interruption.

SWP and SWPB have a single addressing mode, whose address is the contents of a register. Separate registers are used to specify the value to store and the destination of the load. If the same register is specified for both of these, SWP exchanges the value in the register and the value in memory.

The semaphore instructions do not provide a compare and conditional write facility. If wanted, this must be done explicitly.

4.10.1 Examples

```

SWP      R12, R10, [R9]          ; load R12 from address R9 and
                                ; store R10 to address R9

SWPB     R3, R4, [R8]           ; load byte to R3 from address R8 and
                                ; store byte from R4 to address R8

SWP      R1, R1, [R2]           ; Exchange value in R1 and address in R2

```

4.10.2 List of semaphore instructions

```

SWP      Swap.
SWPB     Swap Byte.

```

4.11 Exception-generating instructions

The ARM instruction set provides two types of instruction whose main purpose is to cause a processor exception to occur:

- The Software Interrupt (SWI) instruction is used to cause a SWI exception to occur (see *Software Interrupt exception*). This is the main mechanism in the ARM instruction set by which User mode code can make calls to privileged Operating System code.
- The Breakpoint (BKPT) instruction is used for software breakpoints in ARM architecture versions 5 and above. Its default behavior is to cause a Prefetch Abort exception to occur (see *Prefetch Abort (instruction fetch memory abort)*). A debug monitor program which has previously been installed on the Prefetch Abort vector can handle this exception.

If debug hardware is present in the system, it is allowed to override this default behavior. Details of whether and how this happens are IMPLEMENTATION DEFINED.

4.11.1 Instruction encodings

```

SWI{<cond>} <immed 24>
BKPT <immediate>

```

In both SWI and BKPT, the immediate fields of the instruction are ignored by the ARM processor. The SWI or Prefetch Abort handler can optionally be written to load the instruction that caused the exception and extract these fields. This allows them to be used to communicate extra information about the Operating System call or breakpoint to the handler.

BKPT	Breakpoint.
SWI	Software Interrupt.

The ARM instruction set provides three types of instruction for communicating with coprocessors. These allow:

- The instruction set distinguishes up to 16 coprocessors with a 4-bit field in each coprocessor instruction, so each coprocessor is assigned a particular number.

One coprocessor can use more than one of the 16 numbers if a large coprocessor instruction set is required.

Coprocessors execute the same instruction stream as ARM, ignoring ARM instructions and coprocessor instructions for other coprocessors. Coprocessor instructions that cannot be executed by coprocessor hardware cause an undefined instruction trap, allowing software emulation of coprocessor hardware.

Not all fields in coprocessor instructions are used by the ARM processor. Coprocessor register specifiers and opcodes are defined by individual coprocessors. Therefore, only generic instruction mnemonics are provided for coprocessor instructions. Assembler macros can be used to transform custom coprocessor mnemonics into these generic mnemonics, or to regenerate the opcodes manually.

CDP	p5, 2, c12, c10, c3, 4	; Coproc 5 data operation ; opcode 1 = 2, opcode 2 = 4 ; destination register is 12 ; source registers are 10 and 3
MRC	p15, 5, R4, c0, c2, 3	; Coproc 15 transfer to ARM register ; opcode 1 = 5, opcode 2 = 3 ; ARM destination register = R4 ; coproc source registers are 0 and 2

MCR	p14, 1, R7, c7, c12, 6	; ARM register transfer to Coproc 14 ; opcode 1 = 1, opcode 2 = 6 ; ARM source register = R7 ; coproc dest registers are 7 and 12
LDC	p6, CR1, [R4]	; Load from memory to coprocessor 6 ; ARM register 4 contains the address ; Load to CP reg 1
LDC	p6, CR4, [R2, #4]	; Load from memory to coprocessor 6 ; ARM register R2 + 4 is the address ; Load to CP reg 4
STC	p8, CR8, [R2, #4]!	; Store from coprocessor 8 to memory ; ARM register R2 + 4 is the address ; after the transfer R2 = R2 + 4 ; Store from CP reg 8
STC	p8, CR9, [R2], #-16	; Store from coprocessor 8 to memory ; ARM register R2 holds the address ; after the transfer R2 = R2 - 16 ; Store from CP reg 9

4.12.2 List of coprocessor instructions

CDP Coprocessor Data Operations.
 LDC Load Coprocessor Register.
 MCR Move to Coprocessor from ARM Register.
 MRC Move to ARM Register from Coprocessor.
 STC Store Coprocessor Register.

Note

Coprocessor instructions are not implemented in ARM architecture version 1.

End Note

4.13 Extending the instruction set

Successive versions of the ARM architecture have extended the instruction set in a number of areas. This section describes the six areas where extensions have occurred, and where further extensions might occur in the future:

- Undefined instruction space.
- Arithmetic instruction extension space.
- Control instruction extension space.
- Load/store instruction extension space.
- Coprocessor instruction extension space.
- Unconditional instruction extension space.

Instructions in these areas which have not yet been allocated a meaning are either UNDEFINED or UNPREDICTABLE. To determine which, use the following rules:

1. The *decode bits* of an instruction are defined to be bits[27:20] and bits[7:4].

In ARM architecture version 5 and above, the result of ANDing bits[31:28] together is also a decode bit. This bit determines whether the condition field is 0b1111, which is used in ARM architecture version 5 and above to encode various instructions which can only be executed unconditionally. See *Condition code 0b1111* and *Unconditional instruction extension space* for more information.

2. If the decode bits of an instruction are equal to those of a defined instruction, but the whole instruction is not a defined instruction, then the instruction is UNPREDICTABLE.

For example, suppose an instruction has:

- bits[31:28] not equal to 0b1111
- bits[27:20] equal to 0b00010000
- bits[7:4] equal to 0b0000

but where:

- bit[11] of the instruction is 1.

Here, the instruction is in the control instruction extension space and has the same decode bits as an MRS instruction, but is not a valid MRS instruction because bit[11] of an MRS instruction should be zero. Using the above rule, this instruction is UNPREDICTABLE.

3. In ARM architecture version 4 and above, if the decode bits of an instruction are not equal to those of any defined instruction, then the instruction is UNDEFINED.
4. In ARM architecture version 3 and below, if the decode bits of an instruction are not equal to those of any defined instruction, then the instruction is:
 - UNDEFINED if it is in the undefined instruction space
 - UNPREDICTABLE if it is in any of the other five areas.

Each of rules 2 to 4 above applies separately to each ARM architecture version. As a result, the status of an instruction might differ between architecture versions. Usually, this happens because an instruction which was UNPREDICTABLE or UNDEFINED in an earlier architecture version becomes a defined instruction in a later version.

4.13.1 Undefined instruction space

Instructions with the following opcodes are undefined instruction space:

```
opcode[31:24] = <cond>
opcode[27:25] = 0b011
opcode[4] = 1
```

The meaning of instructions in the undefined instruction space is UNDEFINED on all versions of the ARM architecture.

In general, undefined instructions might be used to extend the ARM instruction set in the future. However, it is intended that instructions with the following encoding will not be used for this:

```
opcode[31:24] = <cond>
opcode[27:20] = 0b01111111
opcode[7:4] = 0b1111
```

If a programmer wants to use an undefined instruction for software purposes, with minimal risk that future hardware will treat it as a defined instruction, one of the instructions with this encoding must be used.

4.13.2 Arithmetic instruction extension space

Instructions with the following opcodes are the arithmetic instruction extension space:

```
opcode[27:24] == 0b0000
opcode[7:4] == 0b1001
opcode[31:28] != 0b1111 /* Only required for version 5 and above */
```

The field names given are guidelines suggested to simplify implementation.

```
opcode[31:24] = <cond>
opcode[27:24] = 0b0000
opcode[23:20] = <op1>
opcode[19:16] = Rn
opcode[15:12] = Rd
opcode[11:8] = Rs
opcode[7:4] = 0b1001
opcode[3:0] = Rm
```

Table 3-3 summarizes the instructions that have already been allocated in this area.

Table 3-3 Arithmetic instruction space

Instructions	op1	Architecture versions
MUL, MULS	000S	Version 2 and above
MLA, MLAS	001S	Version 2 and above
UMULL, UMULLS	100S	All M variants
UMLAL, UMLALS	101S	All M variants
SMULL, SMULLS	110S	All M variants
SMLAL, SMLALS	111S	All M variants

4.13.3 Control instruction extension space

Instructions with the following opcodes are the control instruction space.

```
opcode[27:26] == 0b00
opcode[24:23] == 0b10
opcode[20] == 0
opcode[31:28] != 0b1111 /* Only required for version 5 and above */
```

and not:

```

opcode[25] == 0
opcode[7] == 1
opcode[4] == 1

```

The field names given are guidelines suggested to simplify implementation.

opcode[31:28]	$\langle cond \rangle$	$\langle cond \rangle$	$\langle cond \rangle$
opcode[27:26]	0 0	0 0	0 0
opcode[25]	0	0	1
opcode[24:23]	1 0	1 0	1 0
opcode[22:21]	$\langle op1 \rangle$	$\langle op1 \rangle$	$\langle op1 \rangle$
opcode[20]	0	0	0
opcode[19:16]	Rn	Rn	Rn
opcode[15:12]	Rd	Rd	Rd
opcode[11:8]	Rs	Rs	$\langle rotate\ imm \rangle$
opcode[7:5]	$\langle op2 \rangle$	0 $\langle op2 \rangle$	$\langle imm8 \rangle$
opcode[4]	0	1	
opcode[3:0]	Rm	Rm	

Table 3-4 summarizes the instructions that have already been allocated in this area.

Table 3-4 Control extension space instructions

Instruction	Bit[25]	Bits[7:4]	op1	Architecture versions
MRS	0	0000	x0	Version 3 and above
MSR (register form)	0	0000	x1	Version 3 and above
BX	0	0001	01	Version 5 and above
				plus T variants of version 4
CLZ	0	0001	11	Version 5 and above
BLX (register form)	0	0011	01	Version 5 and above
QADD	0	0101	00	E variants of version 5 and above
QSUB	0	0101	01	E variants of version 5 and above
QDADD	0	0101	10	E variants of version 5 and above
QDSUB	0	0101	11	E variants of version 5 and above
BKPT	0	0111	01	Version 5 and above
SMLA $\langle x \rangle \langle y \rangle$	0	1yx0	00	E variants of version 5 and above
SMLAW $\langle y \rangle$	0	1y00	01	E variants of version 5 and above
SMULW $\langle y \rangle$	0	1y10	01	E variants of version 5 and above
SMLAL $\langle x \rangle \langle y \rangle$	0	1yx0	10	E variants of version 5 and above
SMUL $\langle x \rangle \langle y \rangle$	0	1yx0	11	E variants of version 5 and above
MSR (immediate form)	1	xxxx	x1	Version 3 and above

4.13.4 Load/store instruction extension space

Instructions with the following opcodes are the load/store instruction extension space:

```

opcode[27:25] == 0b000
opcode[7] == 1
opcode[4] == 1
opcode[31:28] != 0b1111 /* Only required for version 5 and above */

```

and not:

```
opcode[24] == 0
opcode[6:5] == 0
```

The field names given are guidelines suggested to simplify implementation.

```
opcode[31:28] = <cond>
opcode[27:25] = 0 0 0
opcode[24] = P
opcode[23] = U
opcode[22] = B
opcode[21] = W
opcode[20] = L
opcode[19:16] = Rn
opcode[15:12] = Rd
opcode[11:8] = Rs
opcode[7] = 1
opcode[6:5] = op1
opcode[4] = 1
opcode[3:0] = Rm
```

Table 3-5 summarizes the instructions that have already been allocated in this area.

Table 3-5 Load/store instructions			
Instruction	Bits[24:20]	op1	Architecture versions
SWP/SWPB	1 0 B 0 0	0 0	Version 3 and above, plus ARMv2a
STRH	P U I W 0	0 1	Version 4 and above
LDRD	P U I W 0	1 0	E variants of version 5 and above, except v5TEvP
STRD	P U I W 0	1 1	E variants of version 5 and above, except v5TEvP
LDRH	P U I W 1	0 1	Version 4 and above
LDRSB	P U I W 1	1 0	Version 4 and above
LDRSH	P U I W 1	1 1	Version 4 and above

4.13.5 Coprocessor instruction extension space

Instructions with the following opcodes are the coprocessor instruction extension space:

```
opcode[27:23] == 0b11000
opcode[21] == 0
```

The field names given are guidelines suggested to simplify implementation.

```
opcode[31:28] = <cond>
opcode[27:23] = 1 1 0 0 0
opcode[22] = x
opcode[21] = 0
opcode[20] = x
opcode[19:16] = Rn
opcode[15:12] = CRd
opcode[11:8] = <cp num>
opcode[7:0] = <offset>
```

In ARM architecture version 3 and below, all instructions in the coprocessor instruction extension space are UNPREDICTABLE.

In all variants of architecture version 4, and in non-E variants of architecture 5, all instructions in the coprocessor instruction extension space are UNDEFINED. It is IMPLEMENTATION DEFINED how an ARM processor achieves this. The options are:

- The ARM processor might take the undefined instruction trap directly.
- The ARM processor might require attached coprocessors not to respond to such instructions. This causes the undefined instruction trap to be taken (see *Undefined Instruction exception*).

In E variants of architecture version 5, instructions in the coprocessor instruction extension space are treated as follows:

- Instructions with bit[22] == 0 are UNDEFINED and are handled in precisely the same way as described above for non-E variants.
- Instructions with bit[22] == 1 are the MCRR and MRRC instructions described in Chapter A10 *Enhanced DSP Extension*.

4.13.6 Unconditional instruction extension space

In ARM architecture version 5 and above, instructions with the following opcode are the unconditional instruction space:

```
opcode[31:28] = 1 1 1 1
opcode[27:20] = <opcode1>
opcode[7:4]   = <opcode2>
```

Table 3-6 summarizes the instructions that have already been allocated in this area.

Table 3-6 Unconditional instruction extension space

Instruction	opcode1	opcode2	Architecture versions
PLD	0 1 1 1 U 1 0 1	x x x x	E variants of version 5 and above, except v5TEvP
BLX (address form)	1 0 1 x x x x x	x x x x	Version 5 and above
STC2	1 1 0 x x x x 0	x x x x	Version 5 and above
LDC2	1 1 0 x x x x 1	x x x x	Version 5 and above
CDP2	1 1 1 0 x x x x	x x x 0	Version 5 and above
MCR2	1 1 1 0 x x x 0	x x x 1	Version 5 and above
MRC2	1 1 1 0 x x x 1	x x x 1	Version 5 and above

5 *ARM Addressing Modes*

5.1 Data-processing operands

There are 11 addressing modes used to calculate the $\langle op1 \rangle$ in an ARM data-processing instruction. The general instruction syntax is:

opcode $\langle cc \rangle \langle S \rangle$ *Rd*, *Rn*, $\langle op1 \rangle$

where $\langle op1 \rangle$ is one of the options shown in table 5.1:

	Syntax	Mode
1	# $\langle value \rangle$	Immediate
2	<i>Rm</i>	Register
3	<i>Rm</i> , LSL # $\langle value \rangle$	Logical shift left by immediate
4	<i>Rm</i> , LSL <i>Rs</i>	Logical shift left by register
5	<i>Rm</i> , LSR # $\langle value \rangle$	Logical shift right by immediate
6	<i>Rm</i> , LSR <i>Rs</i>	Logical shift right by register
7	<i>Rm</i> , ASR # $\langle value \rangle$	Arithmetic shift right by immediate
8	<i>Rm</i> , ASR <i>Rs</i>	Arithmetic shift right by register
9	<i>Rm</i> , ROR # $\langle value \rangle$	Rotate right by immediate
10	<i>Rm</i> , ROR <i>Rs</i>	Rotate right by register
11	<i>Rm</i> , RRX	Rotate right with extend

Table 5.1: $\langle op1 \rangle$ addressing modes

5.1.1 The shifter operand

As well as producing the shifter operand, the shifter produces a carry-out which some instructions write into the Carry Flag. The default register operand (register *Rm* specified with no shift) uses the form register shift left by immediate, with the immediate set to zero.

The shifter operand takes one of the following three basic formats.

Immediate operand value

An immediate operand value is formed by rotating an 8-bit constant (in a 32-bit word) by an even number of bits (0, 2, 4, 8, ..., 26, 28, 30). Therefore, each instruction contains an 8-bit constant and a 4-bit rotate to be applied to that constant.

Some valid constants are:

0xFF, 0x104, 0xFF0, 0xFF00, 0xFF000, 0xFF00000, 0xF000000F

Some invalid constants are:

0x101, 0x102, 0xFF1, 0xFF04, 0xFF003, 0xFFFFFFFF, 0xF000001F

For example:

```
MOV    R0, #0           ; Move zero to R0
ADD    R3, R3, #1       ; Add one to the value of register 3
CMP    R7, #1000        ; Compare value of R7 with 1000
BIC    R9, R8, #0xFF00  ; Clear bits 8-15 of R8 and store in R9
```

Register operand value

A register operand value is simply the value of a register. The value of the register is used directly as the operand to the data-processing instruction. For example:

```
MOV    R2, R0           ; Move the value of R0 to R2
ADD    R4, R3, R2       ; Add R2 to R3, store result in R4
CMP    R7, R8           ; Compare the value of R7 and R8
```

Shifted register operand value

A shifted register operand value is the value of a register, shifted (or rotated) before it is used as the data-processing operand. There are five types of shift:

ASR	Arithmetic Shift Right
LSL	Logical Shift Left
LSR	Logical Shift Right
ROR	ROtate Right
RRX	Rotate Right with eXtend

The number of bits to shift by is specified either as an immediate or as the value of a register. For example:

```
MOV    R2, R0, LSL #2   ; Shift R0 left by 2, write to R2, (R2=R0x4)
ADD    R9, R5, R5, LSL #3 ; R9 = R5 + R5 x 8 or R9 = R5 x 9
RSB    R9, R5, R5, LSL #3 ; R9 = R5 x 8 - R5 or R9 = R5 x 7
SUB    R10, R9, R8, LSR #4 ; R10 = R9 - R8 / 16
MOV    R12, R4, ROR R3   ; R12 = R4 rotated right by value of R3
```

5.1.2 Immediate

Syntax $\# \langle value \rangle$

Operation $\langle op1 \rangle \leftarrow \text{IR}(\text{immed_8}) \text{ Rotate_Right } (\text{IR}(\text{immed_4}) * 2)$
 if rotate_4 $\neq 0$ then
 $\text{ALU}(C) \leftarrow \langle op1 \rangle(31)$

Description This data-processing operand provides a constant (defined in the instruction) operand to a data-processing instruction.

$\langle value \rangle$ specifies the immediate constant wanted. It is an 8-bit immediate (IR(immed_8)) and a 4-bit immediate rotate (IR(immed_4)), so that $\langle value \rangle$ is equal to the result of rotating the 8-bit value right by twice the 4-bit value.

The value $\langle op1 \rangle$ is formed by rotating (to the right) an 8-bit immediate value to any even bit position in a 32-bit word. If the rotate immediate is zero, the carry-out from the shifter is the value of the C flag, otherwise, it is set to bit[31] of the value of $\langle op1 \rangle$.

Notes Not all 32-bit immediates are legitimate. Only those that can be formed by rotating an 8-bit immediate right by an even amount are valid 32-bit immediates for this format.

For more precise syntax exists, where the both `immed_8` and `immed_4` can be specified directly:

$$\# \langle immed_8 \rangle, \langle rotate_amount \rangle$$

where $\langle rotate_amount \rangle$ is the number of bits to rotate the 8-bit $\langle immed_8 \rangle$ value. Thus:

$$\#0xFF, 8 \quad \text{is the same as} \quad \#0xFF00$$

Note that $\langle rotate_amount \rangle$ must be even.

5.1.3 Register

Syntax Rm

Operation $\langle op1 \rangle \leftarrow Rm$

Description This operand provides the value of the register Rm directly.

Notes If the PC is specified as register Rm , the value used is the address of the next instruction, that is to say the address of the current instruction plus 8.

5.1.4 Logical Shift Left by Immediate

Syntax $Rm, \text{LSL } \# \langle value \rangle$

Operation $\langle op1 \rangle \leftarrow Rm \text{ Logical_Shift_Left } IR(\langle value \rangle)$
 if $IR(\langle value \rangle) \neq 0$
 $ALU(C) \leftarrow Rm(32 - \langle value \rangle)$

Description This operand is used to provide value of a register shifted left (multiplied by a constant power of two).

This instruction operand is the value of register Rm , logically shifted left by an immediate $\langle value \rangle$ in the range 0 to 31. Zeros are inserted into the vacated bit positions. The carry flag is the last bit shifted out, or unaffected if no shift is specified.

Notes If the value of the shift ($\langle value \rangle$) is zero, the operand can be written as just Rm (see 5.1.3).

If the PC is specified as register Rm , the value used is the address of the next instruction, that is to say the address of the current instruction plus 8.

5.1.5 Logical Shift Left by Register

Syntax	$Rm, \text{LSL } Rs$
Operation	$\langle op1 \rangle \leftarrow Rm \text{ Logical_Shift_Left } Rs(7:0)$ $\text{if } Rs(7:0) \neq 0$ $ALU(C) \leftarrow Rs(32 - Rs(7:0))$
Description	This operand is used to provide the value of a register multiplied by a variable power of two. This instruction operand is the value of register Rm, logically shifted left by the value in the least significant byte of register Rs. Zeros are inserted into the vacated bit positions. The carry flag is the last bit shifted out, which is zero if the shift amount is more than 32, or unaffected if the shift amount is zero.
Notes	Specifying the PC as register Rm , or register Rs has UNPREDICTABLE results.

5.1.6 Logical Shift Right by Immediate

Syntax	$Rm, \text{LSR } \# \langle value \rangle$
Operation	$\langle op1 \rangle \leftarrow Rm \text{ Logical_Shift_Right } IR(\text{value})$ $\text{if } IR(\text{value}) = 0$ $ALU(C) \leftarrow Rm(31)$ else $ALU(C) \leftarrow Rm(IR(\text{value}) - 1)$
Description	This operand is used to provide the unsigned value of a register shifted right (divided by a constant power of two). This instruction operand is the value of register Rm, logically shifted right by an immediate $\langle value \rangle$ in the range 1 to 32. Zeros are inserted into the vacated bit positions. The carry flag is the last bit shifted out.
Notes	If the PC is specified as register Rm , the value used is the address of the next instruction, that is to say the address of the current instruction plus 8.

5.1.7 Logical Shift Right by Register

Syntax	$Rm, \text{LSR } Rs$
Operation	$\langle op1 \rangle \leftarrow Rm \text{ Logical_Shift_Right } Rs(7:0)$ $\text{if } Rs(7:0) \neq 0$ $ALU(C) \leftarrow Rm(Rs(7:0) - 1)$
Description	This operand is used to provide the unsigned value of a register shifted right (divided by a variable power of two). It is produced by the value of register Rm, logically shifted right by the value in the least significant byte of register Rs. Zeros are inserted into the vacated bit positions. The carry flag is the last bit shifted out, which is zero if the shift amount is more than 32, or unchanged if the shift amount is zero.
Notes	Specifying the PC as register Rm , or register Rs has UNPREDICTABLE results.

5.1.8 Arithmetic Shift Right by Immediate

Syntax	$Rm, ASR \# \langle value \rangle$
Operation	$\langle op1 \rangle \leftarrow Rm \text{ Arithmetic_Shift_Right } IR(value)$ if $IR(value) = 0$ $ALU(C) \leftarrow Rm(31)$ else $ALU(C) \leftarrow Rm(IR(value) - 1)$
Description	This operand is used to provide the signed value of a register arithmetically shifted right (divided by a constant power of two). This instruction operand is the value of register Rm, arithmetically shifted right by an immediate $\langle value \rangle$ in the range 1 to 32. The sign bit of Rm (bit 31) is inserted into the vacated bit positions, thus maintaining the sign of the value. The carry flag is the last bit shifted out.
Notes	If the PC is specified as register Rm , the value used is the address of the next instruction, that is to say the address of the current instruction plus 8.

5.1.9 Arithmetic Shift Right by Register

Syntax	$Rm, ASR Rs$
Operation	$\langle op1 \rangle \leftarrow Rm \text{ Arithmetic_Shift_Right } Rs(7:0)$ if $Rs(7:0) \neq 0$ $ALU(C) \leftarrow Rm(Rs(7:0) - 1)$
Description	This operand is used to provide the signed value of a register arithmetically shifted right (divided by a variable power of two). This instruction operand is the value of register Rm arithmetically shifted right by the value in the least significant byte of register Rs. The sign bit of Rm (bit 31) is inserted into the vacated bit positions. The carry flag is the last bit shifted out, which is the sign bit of Rm if the shift amount is more than 32, or unaffected if the shift amount is zero.
Notes	Specifying the PC as register Rm , or register Rs has UNPREDICTABLE results.

5.1.10 Rotate Right by Immediate

Syntax	$Rm, ROR \# \langle value \rangle$
Operation	$\langle op1 \rangle \leftarrow Rm \text{ Rotate_Right } IR(value)$ $ALU(C) \leftarrow Rm(IR(value) - 1)$
Description	This operand is used to provide the value of a register rotated by a constant value. This instruction operand is the value of register Rm rotated right by an immediate $\langle value \rangle$ in the range 1 to 31. As bits are rotated off the right end, they are inserted into the vacated bit positions on the left. The carry flag is the last bit rotated off the right end.
Notes	If the PC is specified as register Rm , the value used is the address of the next instruction, that is to say the address of the current instruction plus 8.

5.1.11 Rotate Right by Register

Syntax	$Rm, ROR\ Rs$
Operation	$\langle op1 \rangle \leftarrow Rm\ Rotate_Right\ Rs(4:0)$ if $Rs(4:0) \neq 0$ $ALU(C) \leftarrow Rm(Rs(4:0) - 1)$
Description	This operand is used to provide the value of a register rotated by a variable value. This instruction operand is produced by the value of register Rm rotated right by the value in the least significant byte of register Rs. As bits are rotated off the right end, they are inserted into the vacated bit positions on the left. The carry flag is the last bit rotated off the right end, or unaffected if the shift amount is zero.
Notes	Specifying the PC as register Rm , or register Rs has UNPREDICTABLE results.

5.1.12 Rotate Right with Extend

Syntax	RRm, RRX
Operation	$\langle op1 \rangle \leftarrow (CPSR(C)\ Logical_Shift_Left\ 31)\ OR\ (Rm\ Logical_Shift_Right\ 1)$ $ALU(C) \leftarrow Rm(0)$
Description	This operand can be used to perform a 33-bit rotate right using the Carry Flag as the 33rd bit. This instruction operand is the value of register Rm shifted right by one bit, with the Carry Flag replacing the vacated bit position. The carry flag is the bit shifted off the right end.
Notes	A rotate left with extend can be performed with an ADC instruction (A.2 on page 136). If the PC is specified as register Rm, the value used is the address of the next instruction, that is to say the address of the current instruction plus 8.

5.2 Memory Access

There are nine addressing modes used to calculate the address for a Load and Store Word or Unsigned Byte instruction. The general instruction syntax is:

$$opcode\langle cc \rangle\langle B \rangle\langle H \rangle\langle T \rangle\ Rd, \langle op2 \rangle$$

where $\langle op2 \rangle$ is one of the nine options listed in table 5.2 on the next page.

5.2.1 Immediate Offset

Syntax	$[Rn, \# \pm \langle value \rangle]$
Operation	$\langle op2 \rangle \leftarrow Rn + IR(value)$
Description	This addressing mode calculates an address by adding or subtracting the value of an immediate offset to or from the value of the base register Rn .

	Syntax	Mode
1	$[Rn, \# \pm \langle value \rangle]$	Immediate offset
2	$[Rn, Rm]$	Register offset
3	$[Rn, Rm, \langle shift \rangle \# \langle value \rangle]$	Scaled register offset
4	$[Rn, \# \pm \langle value \rangle] !$	Immediate pre-indexed
5	$[Rn, Rm] !$	Register pre-indexed
6	$[Rn, Rm, \langle shift \rangle \# \langle value \rangle] !$	Scaled register pre-indexed
7	$[Rn], \# \pm \langle value \rangle$	Immediate post-indexed
8	$[Rn], Rm$	Register post-indexed
9	$[Rn], Rm, \langle shift \rangle \# \langle value \rangle$	Scaled register post-indexed

Table 5.2: Memory Addressing Modes

Usage	This addressing mode is useful for accessing structure (record) fields, and accessing parameters and local variables in a stack frame. With an offset of zero, the address produced is the unaltered value of the base register Rn .
Notes	The syntax $[Rn]$ is treated as an abbreviation for $[Rn, \#0]$. If the PC is specified as register Rm , the value used is the address of the next instruction, that is to say the address of the current instruction plus 8.

5.2.2 Register Offset

Syntax	$[Rn, Rm]$
Operation	$\langle op2 \rangle \leftarrow Rn + Rm$
Description	This addressing mode calculates an address by adding or subtracting the value of the index register Rm to or from the value of the base register Rn .
Usage	This addressing mode is used for pointer plus offset arithmetic, and accessing a single element of an array of bytes.
Notes	If the PC is specified as register Rn , the value used is the address of the next instruction, that is to say the address of the current instruction plus 8. Specifying the PC as register Rm , has UNPREDICTABLE results.

5.2.3 Scaled Register Offset

Syntax	One of: $[\langle Rn \rangle, Rm, LSL \# \langle value \rangle]$ $[\langle Rn \rangle, Rm, LSR \# \langle value \rangle]$ $[\langle Rn \rangle, Rm, ASR \# \langle value \rangle]$ $[\langle Rn \rangle, Rm, ROR \# \langle value \rangle]$ $[\langle Rn \rangle, Rm, RRX]$
Operation	LSL: $index \leftarrow Rm \text{ Logical_Shift_Left } IR(value)$ LSR: $index \leftarrow Rm \text{ Logical_Shift_Right } IR(value)$ ASR: $index \leftarrow Rm \text{ Arithmetic_Shift_Right } IR(value)$ ROR: $index \leftarrow Rm \text{ Rotate_Right } IR(value)$ RRX: $index \leftarrow (CSPR(C) \text{ Logical_Shift_Left } 31) \text{ OR } (Rm \text{ Logical_Shift_Right } 1)$ $\langle op2 \rangle \leftarrow Rn + index$

Description	These five addressing modes calculate an address by adding or subtracting the shifted or rotated value of the index register Rm to or from the value of the base register Rn .
Usage	These addressing modes are used for accessing a single element of an array of values larger than a byte.
Notes	If the PC is specified as register Rn , the value used is the address of the next instruction, that is to say the address of the current instruction plus 8. Specifying the PC as register Rm , has UNPREDICTABLE results.

5.2.4 Immediate Pre-indexed

Syntax	$[Rn, \# \pm \langle value \rangle] !$
Operation	$\langle op2 \rangle \leftarrow Rn + IR(\langle value \rangle)$ $\langle cc \rangle: Rn \leftarrow \langle op2 \rangle$
Description	This addressing mode calculates an address by adding or subtracting the value of an immediate $\langle value \rangle$ to or from the value of the base register Rn. If the condition specified in the instruction ($\langle cc \rangle$) matches the condition code status, the calculated address is written back to the base register Rn.
Usage	This addressing mode is used for pointer access to arrays with automatic update of the pointer value.
Notes	Specifying the PC as register Rm , has UNPREDICTABLE results.

5.2.5 Register Pre-indexed

Syntax	$[Rn, Rm] !$
Operation	$\langle op2 \rangle \leftarrow Rn + Rm$ $\langle cc \rangle: Rn \leftarrow \langle op2 \rangle$
Description	This addressing mode calculates an address by adding or subtracting the value of an index register Rm to or from the value of the base register Rn. If the condition specified in the instruction ($\langle cc \rangle$) matches the condition code status, the calculated address is written back to the base register Rn.
Notes	If the same register is specified for Rn and Rm, the result is UNPREDICTABLE. Specifying the PC as register Rm, has UNPREDICTABLE results.

5.2.6 Scaled Register Pre-indexed

Syntax	One of:
	$[Rn, Rm, LSL \# \langle value \rangle] !$
	$[Rn, Rm, LSR \# \langle value \rangle] !$
	$[Rn, Rm, ASR \# \langle value \rangle] !$
	$[Rn, Rm, ROR \# \langle value \rangle] !$
	$[Rn, Rm, RRX] !$

Operation	LSL: $\text{index} \leftarrow Rm \text{ Logical_Shift_Left } IR(\text{value})$
	LSR: $\text{index} \leftarrow Rm \text{ Logical_Shift_Right } IR(\text{value})$
	ASR: $\text{index} \leftarrow Rm \text{ Arithmetic_Shift_Right } IR(\text{value})$
	ROR: $\text{index} \leftarrow Rm \text{ Rotate_Right } IR(\text{value})$
	RRX: $\text{index} \leftarrow (\text{CSPR}(C) \text{ Logical_Shift_Left } 31) \text{ OR } (Rm \text{ Logical_Shift_Right } 1)$
	$\langle op2 \rangle \leftarrow Rn + \text{index}$ $\langle cc \rangle: Rn \leftarrow \langle op2 \rangle$
Description	These five addressing modes calculate an address by adding or subtracting the shifted or rotated value of the index register Rm to or from the value of the base register Rn .
	If the condition specified in the instruction ($\langle cc \rangle$) matches the condition code status, the calculated address is written back to the base register Rn .
Notes	If the same register is specified for Rn and Rm , the result is UNPREDICTABLE.
	Specifying the PC as register Rm , has UNPREDICTABLE results.

5.2.7 Immediate Post-indexed

Syntax	$[Rn], \# \pm \langle value \rangle$
Operation	$\langle op2 \rangle \leftarrow Rn$
	$\langle cc \rangle: Rn \leftarrow Rn + IR(\text{value})$
Description	This addressing mode uses the value of the base register Rn as the address for the memory access.
	If the condition specified in the instruction ($\langle cc \rangle$) matches the condition code status, the value of the immediate offset is added to or subtracted from the value of the base register Rn and written back to the base register Rn .
Usage	This addressing mode is used for pointer access to arrays with automatic update of the pointer value.
Notes	Specifying the PC as register Rm , has UNPREDICTABLE results.

5.2.8 Register Post-indexed

Syntax	$[Rn], Rm$
Operation	$\langle op2 \rangle \leftarrow Rn$
	$\langle cc \rangle: Rn \leftarrow Rn + Rm$
Description	This addressing mode uses the value of the base register Rn as the address for the memory access.
	If the condition specified in the instruction ($\langle cc \rangle$) matches the condition code status, the value of the index register Rm is added to or subtracted from the value of the base register Rn and written back to the base register Rn .
Notes	If the same register is specified for Rn and Rm , the result is UNPREDICTABLE.
	Specifying the PC as register Rm , has UNPREDICTABLE results.

5.2.9 Scaled Register Post-indexed

Syntax One of:

[*Rn*] , *Rm* , LSL #(*value*)
 [*Rn*] , *Rm* , LSR #(*value*)
 [*Rn*] , *Rm* , ASR #(*value*)
 [*Rn*] , *Rm* , ROR #(*value*)
 [*Rn*] , *Rm* , RRX

Operation $\langle op2 \rangle \leftarrow Rn$
 LSL: index $\leftarrow Rm$ Logical_Shift_Left IR(*value*)
 LSR: index $\leftarrow Rm$ Logical_Shift_Right IR(*value*)
 ASR: index $\leftarrow Rm$ Arithmetic_Shift_Right IR(*value*)
 ROR: index $\leftarrow Rm$ Rotate_Right IR(*value*)
 RRX: index $\leftarrow (CSPR(C) \text{ Logical_Shift_Left } 31) \text{ OR } (Rm \text{ Logical_Shift_Right } 1)$
 $\langle cc \rangle$: *Rn* $\leftarrow Rn + \text{index}$

Description This addressing mode uses the value of the base register *Rn* as the address for the memory access.

If the condition specified in the instruction ($\langle cc \rangle$) matches the condition code status, the shifted or rotated value of index register *Rm* is added to or subtracted from the value of the base register *Rn* and written back to the base register *Rn*.

Notes If the same register is specified for *Rn* and *Rm*, the result is UNPREDICTABLE.
 Specifying the PC as register *Rm*, has UNPREDICTABLE results.

6 *Beginning Programs*

This chapter contains some very elementary programs. They will introduce some fundamental features of the ARM. In addition, these programs demonstrate some primitive tasks that are common to assembly language programs for many different applications.

6.1 Example Programs

The only way to learn assembly language programming is through experience. Throughout these notes we use example programs to demonstrate various aspects of assembly programming.

Each of the program examples contains a number of parts:

A Title that describes the general problem

A statement of purpose that describes the task the program performs and the memory locations used.

A sample problem with data and results.

An algorithm if the program logic is complex.

Source program or assembly language listing.

Explanatory notes that discusses the instructions and methods used in the program.

Each example is written and assembled as a stand-alone program. They can be downloaded from the web site.

6.1.1 Program Listing Format

The examples in the book are the actual source code used to generate the executables. Sometimes you may need to use the listing output of the ARM assembler (the `.lis` file), and in any case you should be aware of the fact that you can generate a listing file. See the section on the ARMulator environment which follows for details of how to generate a `.lis` listing file. As an example, here is the assembler listing for program 6.1:

The listing is effectively split into seven columns as follows:

1. The first column is the line number of the original source (`.s`) program file.
2. The eight-digit number in the second column is the offset of the hexadecimal address of the first byte of the object code shown in the next column.

3. The two to eight-digit hex value in the third column is the object code/binary produced by the assembly instruction on that line. The first byte of this code is stored at the memory address given in the previous column. Subsequent bytes are stored in successive bytes.
Occasionally, more code is produced than this column has space to display, in which case the code is carried over on to the next line.
4. This column is the first of the original source program. It represents the label field of the source program as described in Chapter 2.
5. This is the Operation Code or Mnemonic field described in Chapter 2. This is the machine instruction or assembler directive.
6. The Operand or Address Field from Chapter 2. This is normally the operand(s) to the assembler directive or machine instruction in the previous column. A few instructions do not require operands.
7. This is the final part of the original source program line. This is Chapter 2's comment field.

If you wish to assemble these examples, key in the source statements only; do not enter the addresses or object codes, since the assembler program will generate them. You could also download the source (.s) file from the web site.

6.1.2 Guidelines for Examples

We have used the following guidelines in construction of the examples:

1. Standard ARM assembler notation is used, as summarized in Chapter 2.
2. The forms in which data and addresses appear are selected for clarity rather than for consistency. We use hexadecimal numbers for memory addresses, instruction codes, and BCD data; decimal for numeric constants; binary for logical masks; and ASCII for characters.
3. Frequently used instructions and programming techniques are emphasized.
4. Examples illustrate tasks that microprocessors perform in communication, instrumentation, computers, business equipment, industrial, and military applications.
5. Detailed comments are included.
6. Simple and clear structures are emphasised, but programs are written as efficiently as possible within this guideline. Notes accompanying programs often describe more efficient procedures.
7. Programs use consistent memory allocation. Each program starts in memory location 8000₁₆ and ends with the `SWI &11` (Software Interrupt) instruction. Each program is written as an independent procedure or subroutine although no assumptions are made concerning the state of the microprocessor on procedure entry. You may prefer to modify the way a program ends. This could be done by replacing the `SWI &11` instruction with an endless loop instruction such as:

```
HERE B    HERE
```

8. Programs use standard ARM assembler directives. We introduced assembler directives conceptually in Chapter 2. When first examining programming examples, you can ignore assembler directives if you do not understand them. Assembler directives do not contribute to program logic, which is what you will be trying to understand initially; but they are a necessary part of every assembly language program, so you will have to learn how to use them before you write any executable programs. Including assembler directives in all program examples will help you become familiar with the functions they perform.

6.1.3 Trying the examples

To test one of the example programs, first download the source from the web site. Go to the start menu and call up the “Armulate” program. Next open the source file that you have downloaded by using the normal “File | Open” menu option. This will open your program source in a separate window within the “Armulate” environment.

The next step is to create a new Project within the environment. Select the “Project” menu option, then “New”. Give your project the same name as the source file that you are using (there is no need to use a file extension – it will automatically be saved as a `.apj` file).

Once you have given the file a name, a further dialog will open as shown in the figure 6.1.

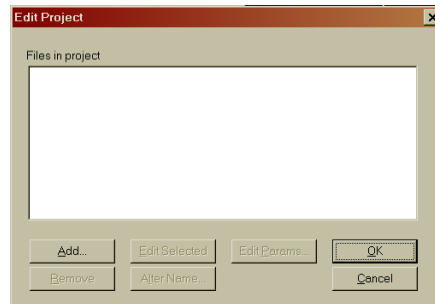


Figure 6.1: New Project Dialog

Click the “Add” button, and you will again be presented with a file dialog, which will display the source files in the current directory. Select the relevant source file and “OK” the dialog. You will be returned to the previous dialog, but you will see now that your source file is included in the project. “OK” the “Edit Project” dialog, and you will be returned to the Armulate environment, now with two windows open within it, one for the source code and one for the project.

We recommend that you always create a `.lis` listing file for each project that you create. Do this by selecting the “Options” menu with the project window in focus, then the “Assembler” item. This will open the dialog shown in figure 6.2.

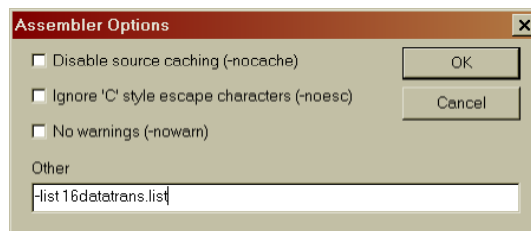


Figure 6.2: Assembler Options Dialog

Enter `-list [yourfilename].lis` into the “Other” text box and “OK” the dialog.

You have now created your project and are ready to assemble and debug your code.

Additional information on the Armulator is available via the help menu item.

6.1.4 Program Initialization

All of the programming examples presented in these notes pay particular attention to the correct initialization of constants and operands. Often this requires additional instructions that may

appear superfluous, in that they do not contribute directly to the solution of the stated problem. Nevertheless, correct initialization is important in order to ensure the proper execution of the program every time.

We want to stress correct initialization; that is why we are going to emphasize this aspect of problems.

6.1.5 Special Conditions

For the same reasons that we pay particular attention to special conditions that can cause a program to fail. Empty lists and zero indexes are two of the most common circumstances overlooked in sample problems. It is critically important when using microprocessors that you learn with your very first program to anticipate unusual circumstances; they frequently cause your program to fail. You must build in the necessary programming steps to account for these potential problems.

6.2 Program Examples

6.2.1 16-Bit Data Transfer

Move the contents of one 16-bit variable `Value` to another 16-bit variable `Result`.

Sample Problems

Input: `Value` = `C123`
Output: `Result` = `C123`

Program 6.1: `16bitdatatrans.s` — *16bit data transfer*

```

1  *          16bit data transfer
2
3          TTL      16bitdatatrans
4          AREA     Program, CODE, READONLY
5          ENTRY
6
7  Main
8          LDRB     R1, Value           ;Load the value to be moved
9          STR      R1, Result         ;Store it back in a different location
10         SWI      &11
11
12  Value    DCW     &C123              ;Value to be moved
13          ALIGN
14          Result  DCW     0            ;Need to do this because working with 16bit value
15          END                      ;Storage space

```

This program solves the problem in two simple steps. The first instruction loads data register `R1` with the 16-bit value in location `Value`. The next instruction saves the 16-bit contents of data register `R1` in location `Result`.

As a reminder of the necessary elements of an assembler program for the ARMulator, notice that this, and all the other example programs have the following elements. Firstly there must be an `ENTRY` directive. This tells the assembler where the first executable instruction is located. Next there must be at least one `AREA` directive, at the start of the program, and there may be other `AREA` directives to define data storage areas. Finally there must be an `END` directive, to show where the code ends. The absence of any of these will cause the assembly to fail with an error.

Another limitation to bear in mind is that ARMulator instructions will only deal with `BYTE` (8 bits) or `WORD` (32 bit) data sizes. It is possible to declare `HALF-WORD` (16 bit) variables by the use

of the `DCW` directive, but it is necessary to ensure consistency of storage of `HALF-WORD` by the use of the `ALIGN` directive. You can see the use of this in the first worked example.

In addition, under the RISC architecture of the ARM, it is not possible to directly manipulate data in storage. Even if no actual manipulation of the data is taking place, as in this first example, it is necessary to use the `LDR` or `LDRB` and `STR` or `STRB` to move data to a different area of memory.

This version of the `LDR` instruction moves the 32-bit word contained in memory location `Value` into a register and then stores it using the `STR` instruction at the memory location specified by `Result`.

Notice that, by default, every program is allocated a literal pool (a storage area) after the last executable line. In the case of this, and most of the other programs, we have formalised this by the use of the `AREA Data1, DATA` directive. Instruction on how to find addresses of variables will be given in the seminars.

6.2.2 One's Complement

From the bitwise complement of the contents of the 16-bit variable `Value`.

Sample Problems

Input: `Value` = C123
Output: `Result` = FFFF3EDC

Program 6.2: `onescomp.s` — *Find the one's compliment of a number*

```

1  *      Find the one's compliment of a number
2
3          TTL      onescomp
4          AREA     Program, CODE, READONLY
5          ENTRY
6
7  Main
8      LDR      R1, Value          ;load the number to be complimented
9      MVN      R1, R1            ;NOT the contents of R1
10     STR      R1, Result        ;store the result
11     SWI      &11
12
13  Value DCD     &C123           ;value to be complemented
14  Result DCD     0              ;storage for result
15     END

```

This program solves the problem in three steps. The first instruction moves the contents of location `Value` into data register `R1`. The next instruction `MVN` takes the logical complement of data register `R1`. Finally, in the third instruction the result of the logical complement is stored in `Value`.

Note that any data register may be referenced in any instruction that uses data registers, but note the use of `R15` for the program counter, `R14` for the link register and `R13` for the stack pointer. Thus, in the `LDR` instruction we've just illustrated, any of the general purpose registers could have been used.

The `LDR` and `STR` instructions in this program, like those in Program 6.1, demonstrate one of the ARM's addressing modes. The data reference to `Value` as a source operand is an example of immediate addressing. In immediate addressing the offset to the address of the data being referenced (less 8 bytes) is contained in the extension word(s) following the operation word of the instruction. As shown in the assembly listing, the offset to the address corresponding to `Value` is found in the extension word for the `LDR` and `STR` instructions.

6.2.3 16-Bit Addition

Add the contents of the 16-bit variable **Value1** to the contents of the 16-bit variable **Value2** and place the result in the 16-bit variable **Result**.

Sample Problems

Input: **Value1** = C123
 Value2 = 02AA
 Output: **Result** = C3CD

Program 6.3a: 16bitadd.s — *Add two 16bit numbers*

```

1  *      Add two 16bit numbers
2
3          TTL      16bitadd
4          AREA     Program, CODE, READONLY
5          ENTRY
6
7  Main
8          LDR      R1, Value1          ;load the first number
9          LDR      R2, Value2          ;load the second number
10         ADD      R1, R1, R2          ;add them together into R1 (x = x + y)
11         STR      R1, Result          ;store the result
12         SWI      &11
13
14  Value1  DCW      &C123              ;first value to be added
15         ALIGN
16  Value2  DCW      &02AA              ;second value to be added
17         ALIGN
18  Result  DCW      0                  ;storage for result
19         END

```

The ADD instruction in this program is an example of a three-operand instruction. Unlike the LDR instruction, this instruction's third operand not only represents the instruction's destination but may also be used to calculate the result. The format:

$$\text{DESTINATION} \leftarrow \text{SOURCE1 } operation \text{ SOURCE2}$$

is common to many of the instructions.

As with any microprocessor, there are many instruction sequences you can execute which will solve the same problem. Program 6.3b, for example, is a modification of Program 6.3a and uses offset addressing instead of immediate addressing.

Program 6.3b: 16bitadd-2.s — *Add two 16bit numbers and store the result*

```

1  *      Add two 16bit numbers and store the result
2
3          TTL      16bitadd-2
4          AREA     Program, CODE, READONLY
5          ENTRY
6
7  Main
8          ADR      R0, Value1          ;load the address of first value
9          LDR      R1, [R0]            ;load what is at that address
10         ADD      R0, R0, #0x4        ;adjust the pointer
11         LDR      R2, [R0]            ;and load what is at the new addr
12         ADD      R1, R1, R2          ;add together
13         ADR      R0, Result          ;load the storage address
14         STR      R1, [R0]            ;store the result
15         SWI      &11                  ;all done

```

```

16
17 Value1 DCD    &C123           ;first value
18 Value2 DCD    &A05B           ;second value
19 Result DCD    0               ;space to store result
20      END

```

The `ADR` pseudo-instruction introduces a new addressing mode — offset addressing, which we have not used previously. Immediate addressing lets you define a data constant and include that constant in the instruction's associated object code. The assembler format identifies immediate addressing with a `#` preceding the data constant. The size of the data constant varies depending on the instruction. Immediate addressing is extremely useful when small data constants must be referenced.

The `ADR` pseudo-instruction could be replaced by the use of the instruction `LDR` together with the use of the `=` to indicate that the address of the data should be loaded rather than the data itself.

The second addressing mode — offset addressing — uses immediate addressing to load a pointer to a memory address into one of the general purpose registers.

Program 6.3b also demonstrates the use of base register plus offset addressing. In this example we have performed this operation manually on line 10 (`ADD R0, R0, #0x4`), which increments the address stored in `R0` by 4 bytes or one `WORD`. There are much simpler and more efficient ways of doing this, such as pre-index or post-index addressing which we will see in later examples.

Another advantage of this addressing mode is its faster execution time as compared to immediate addressing. This improvement occurs because the address extension word(s) does not have to be fetched from memory prior to the actual data reference, after the initial fetch.

A final advantage is the flexibility provided by having `R0` hold an address instead of being fixed as part of the instruction. This flexibility allows the same code to be used for more than one address. Thus if you wanted to add the values contained in consecutive variables `Value3` and `Value4`, you could simply change the contents of `R0`.

6.2.4 Shift Left One Bit

Shift the contents of the 16-bit variable `Value` to the left one bit. Store the result back in `Result`.

Sample Problems

```

Input:  Value   =  4242  (0100 0010 0100 00102)
Output: Result  =  8484  (1000 0100 1000 01002)

```

Program 6.4: `shiftright.s` — *Shift Left one bit*

```

1  *      Shift Left one bit
2
3      TTL      shiftright
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R1, Value           ;Load the value to be shifted
9      MOV      R1, R1, LSL#0x1    ;Shift left one bit
10     STR      R1, Result          ;Store the result
11     SWI      &11
12
13 Value DCD    &4242               ;Value to be shifted
14 Result DCD    0                 ;Space to store result
15     END

```

The **MOV** instruction is used to perform a logical shift left. Using the operand format of the **MOV** instruction shown in Program 6.4, a data register can be shifted from 1 to 25 bits on either a byte, word or longword basis. Another form of the **LSL** operation allows a shift counter to be specified in another data register.

6.2.5 Byte Disassembly

Divide the least significant byte of the 8-bit variable **Value** into two 4-bit nibbles and store one nibble in each byte of the 16-bit variable **Result**. The low-order four bits of the byte will be stored in the low-order four bits of the least significant byte of **Result**. The high-order four bits of the byte will be stored in the low-order four bits of the most significant byte of **Result**.

Sample Problems

Input: **Value** = 5F
Output: **Result** = 050F

Program 6.5: **splitbyte.s** — *Disassemble a byte into its high and low order nibbles*

```

1  *      Disassemble a byte into its high and low order nibbles
2
3      TTL      splitbyte
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R1, Value           ;Load the value to be disassembled
9      LDR      R2, Mask           ;Load the bitmask
10     MOV      R3, R1, LSR#0x4     ;Copy just the high order nibble into R3
11     MOV      R3, R3, LSL#0x8     ;now left shift it one byte
12     AND      R1, R1, R2         ;AND the original number with the bitmask
13     ADD      R1, R1, R3         ;Add the result of that to
14                                     ;what we moved into R3
15     STR      R1, Result         ;Store the result
16     SWI      &11
17
18  Value  DCB      &FB             ;Value to be shifted
19         ALIGN
20         ;keep the memory boundaries
21  Mask   DCW      &000F          ;bitmask = %0000000000001111
22         ALIGN
23  Result DCD      0              ;Space to store result
24     END

```

This is an example of byte manipulation. The ARM allows most instructions which operate on words also to operate on bytes. Thus, by using the **B** suffix, all the **LDR** instructions in Program 6.5 become **LDRB** instructions, therefore performing byte operations. The **STR** instruction must remain, since we are storing a *halfword* value. If we were only dealing with a one byte result, we could use the **STRB** byte version of the store instruction.

Remember that the **MOV** instruction performs register-to-register transfers. This use of the **MOV** instruction is quite frequent.

Generally, it is more efficient in terms of program memory usage and execution time to minimise references to memory.

6.2.6 Find Larger of Two Numbers

Find the larger of two 32-bit variables **Value1** and **Value2**. Place the result in the variable **Result**. Assume the values are unsigned.

Sample Problems

			a	b
Input:	Value1	=	12345678	12345678
	Value2	=	87654321	0ABCDEF1
Output:	Result	=	87654321	12345678

Program 6.6: `comparenum.s` — *Find the larger of two numbers*

```

1  *      Find the larger of two numbers
2
3      TTL      comparenum
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R1, Value1          ;Load the first value to be compared
9      LDR      R2, Value2          ;Load the second value to be compared
10     CMP      R1, R2              ;Compare them
11     BHI      Done                ;if R1 contains the highest
12     MOV      R1, R2              ;otherwise overwrite R1
13  Done
14     STR      R1, Result           ;Store the result
15     SWI      &11
16
17  Value1 DCD    &FEDCA987          ;Value to be compared
18  Value2 DCD    &12345678          ;Value to be compared
19  Result DCD    0                  ;Space to store result
20     END

```

The Compare instruction, `CMP`, sets the status register flags as if the destination, `R1`, were subtracted from the source `R2`. The order of the operands is the same as the operands in the subtract instruction, `SUB`.

The conditional transfer instruction `BHI` transfers control to the statement labeled `Done` if the unsigned contents of `R2` are greater than or equal to the contents of `R1`. Otherwise, the next instruction (on line 12) is executed. At `Done`, register `R2` will always contain the larger of the two values.

The `BHI` instruction is one of several conditional branch instructions. To change the program to operate on signed numbers, simply change the `BHI` to `BGE` (Branch if Greater than or Equal to):

```

...
CMP    R1,R2
BGE    Done
...

```

You can use the following table 6.1 to use when performing signed and unsigned comparisons.

Note that the same instructions are used for signal and unsigned addition, subtraction, or comparison; however, the comparison operations are different.

The conditional branch instructions are an example of program counter relative addressing. In other words, if the branch condition is satisfied, control will be transferred to an address relative to the current value of the program counter. Dealing with compares and branches is an important part of programming. Don't confuse the sense of the `CMP` instruction. After a compare, the relation tested is:

DESTINATION *condition* SOURCE

For example, if the condition is “less than,” then you test for destination less than source. Become familiar with all of the conditions and their meanings. Unsigned compares are very useful when comparing two addresses.

Compare Condition	Signed	Unsigned
greater than or equal	BGE	BCC
greater than	BGT	BHI
equal	BEQ	BEQ
not equal	BNE	BNE
less than or equal	BLS	BLS
less than	BLT	BCS

Table 6.1: Signed/Unsigned Comparisons

6.2.7 64-Bit Addition

Add the contents of two 64-bit variables `Value1` and `Value2`. Store the result in `Result`.

Sample Problems

```
Input:  Value1 = 12A2E640, F2100123
        Value2 = 001019BF, 40023F51
Output: Result  = 12B30000, 32124074
```

Program 6.7: 64bitadd.s — 64 bit addition

```
1  *      64 bit addition
2
3      TTL      64bitadd
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      ADR      R0, Value1          ;Pointer to first value
9      LDMIA    R0, {R1,R2}        ;Load the value to be added
10     ADR      R0, Value2          ;Pointer to second value
11     LDMIA    R0, {R3,R4}        ;Load the value to be added
12     ADDS     R6, R2, R4          ;Add lower 4 bytes and set carry flag
13     ADC      R5, R1, R3          ;Add upper 4 bytes including carry
14     ADR      R0, Result          ;Pointer to Result
15     STMIA    R0, {R5, R6}        ;Store the result
16     SWI      &5
17     SWI      &11
18
19 Value1 DCD    &12A2E640,&F2100123 ;Value to be added
20 Value2 DCD    &001019BF,&40023F51 ;Value to be added
21 Result DCD    0                  ;Space to store result
22     END
```

Here we introduce several important and powerful instructions from the ARM instruction set. The first of these is `LDMIA`, which may appear to be a confusing acronym, but which simply stands for “Load many increment after”. As before, at line 8 we use the `ADR` instruction which causes register `R0` to hold the starting address of `Value1`. At line 9 the instruction `LDMIA R0, {R1,R2}` fetches the next two 32-bit values, starting at the location pointed to by `R0` and places them in registers `R1` and `R2`. This process is repeated for the next two 32-bit values.

Next, the two low order `WORDS`, held in `R2` and `R4` are added, and the result stored in `R6`.

This is all straightforward, but note now the use of the `S` suffix to the `ADD` instruction. This forces the update of the flags as a result of the `ADD` operation. In other words, if the result of the addition results in a carry, the carry flag bit will be set.

Now the `ADC` (add with carry) instruction is used to add the two high order `WORDS`, held in `R1` and `R3`, but taking into account any carry resulting from the previous addition.

Finally, the `STMIA` instruction is used to store the result now held in `R5` and `R6`.

6.2.8 Table of Factorials

Calculate the factorial of the 8-bit variable **Value** from a table of factorials **DataTable**. Store the result in the 16-bit variable **Result**. Assume **Value** has a value between 0 and 7.

Sample Problems

Input:	FTABLE	=	0001	(0!	=	1 ₁₀)
		=	0001	(1!	=	1 ₁₀)
		=	0002	(2!	=	2 ₁₀)
		=	0006	(3!	=	6 ₁₀)
		=	0018	(4!	=	24 ₁₀)
		=	0078	(5!	=	120 ₁₀)
		=	02D0	(6!	=	720 ₁₀)
		=	13B0	(7!	=	5040 ₁₀)
	Value	=	05			
Output:	Result	=	0078	(5!	=	120 ₁₀)

Program 6.8: **factorial.s** — *Lookup the factorial from a table by using the address of the memory location*

```

1  *      Lookup the factorial from a table by
2  *      using the address of the memory location
3
4      TTL      factorial
5      AREA     Program, CODE, READONLY
6      ENTRY
7
8  Main
9      LDR      R0, =DataTable      ;load the address of the lookup table
10     LDR      R1, Value           ;offset of value to be looked up
11     MOV      R1, R1, LSL#0x2     ;data is declared as 32bit - need
12                                     ;to quadruple the offset to point at the
13                                     ;correct memory location
14     ADD      R0, R0, R1          ;R0 now contains memory address to store
15     LDR      R2, [R0]
16     ADR      R3, Result          ;the address where we want to store the answer
17     STR      R2, [R3]           ;store the answer
18
19     SWI      &11
20
21     AREA     DataTable, DATA
22
23     DCD      1                  ;0! = 1          ;the data table containing the factorials
24     DCD      1                  ;1! = 1
25     DCD      2                  ;2! = 2
26     DCD      6                  ;3! = 6
27     DCD      24                 ;4! = 24
28     DCD      120                ;5! = 120
29     DCD      720                ;6! = 720
30     DCD      5040               ;7! = 5040
31  Value  DCB      5
32          ALIGN
33  Result DCW      0
34
35     END

```

The approach to this table lookup problem, as implemented in this program, demonstrates the use of offset addressing. The first two LDR instructions, load register **R0** with the start address of the lookup table¹, and register **R1** contents of **Value**.

¹ Note that we are using a LDR instruction as the data table is sufficiently far away from the instruction that an ADR instruction is not valid.

The actual calculation of the entry in the table is determined by the first operand of the `R1, R1, LSL #0x2` instruction. The long word contents of address register `R1` are added to the long word contents of data register `R0` to form the effective address used to index the table entry. When `R0` is used in this manner, it is referred to as an index register.

6.3 Problems

You should try the programming problems at the end of each chapter to ensure that you understand the ideas presented in the chapter. You should use the programming examples as guidelines for solving the problems. Don't forget to run your solutions on the ARMulator to ensure that they are correct.

The following guidelines will help in solving the problems:

1. Comment each program so that others can understand it. The comments can be brief and ungrammatical. They should explain the purpose of a section or instruction in the program, but should not describe the operation of instructions, that description is available in manuals. You do not have to comment each statement or explain the obvious. You may follow the format of the examples but provide less detail.
2. Emphasise clarity, simplicity, and good structure in programs. While programs should be reasonably efficient, do not worry about saving a single byte of program memory or a few microseconds.
3. Make programs reasonably general. Do not confuse parameters (such as the number of elements in any array) with fixed constants (such as the code for the letter "C").
4. Never assume fixed initial values for parameters.
5. Use assembler notation as shown in the examples and defined in Chapter 2.
6. Use symbolic notation for address and data references. Symbolic notation should also be used even for constants (such as `DATA_SELECT` instead of `2_00000100`). Also use the clearest possible form for data (such as `'C'` instead of `0x43`).
7. Use meaningful names for labels and variables, e.g., `SUM` or `CHECK` rather than `X` or `Z`.
8. Execute each program with the emulator. There is no other way of ensuring that your program is correct. We have provided sample data with each problem. Be sure that the program works for special cases.

6.3.1 64-Bit Data Transfer

Move the contents of the 64-bit variable `VALUE` to the 64-bit variable `RESULT`.

Sample Problems

```
Input:  VALUE    3E2A42A1
        21F260A0

Output: RESULT   3E2A42A1
        21F260A0
```

6.3.2 32-Bit Subtraction

Subtract the contents of the 32-bit variable **VALUE1** from the contents of the 32-bit variable **VALUE2** and store the result back in **VALUE1**.

Sample Problems

```
Input:  VALUE1  12343977
        VALUE2  56782182
Output: VALUE1  68AC5AF9
```

6.3.3 Shift Right Three Bits

Shift the contents of the 32-bit variable **VALUE** right three bits. Clear the three most significant bit position.

Sample Problems

		Test A	Test B
Input:	VALUE	415D7834	9284C15D
Output:	VALUE	082BAF06	1250982B

6.3.4 Halfword Assembly

Combine the low four bits of each of the four consecutive bytes beginning at **LIST** into one 16-bit halfword. The value at **LIST** goes into the most significant nibble of the result. Store the result in the 32-bit variable **RESULT**.

Sample Problems

```
Input:  LIST    0C
          02
          06
          09
Output: RESULT  0000C269
```

6.3.5 Find Smallest of Three Numbers

The three 32-bit variables **VALUE1**, **VALUE2** and **VALUE3**, each contain an unsigned number. Store the smallest of these numbers in the 32-bit variable **RESULT**.

Sample Problems

```
Input:  VALUE1  91258465
        VALUE2  102C2056
        VALUE3  70409254
Output: RESULT  102C2056
```

6.3.6 Sum of Squares

Calculate the squares of the contents of word **VALUE1** and word **VALUE2** then add them together. Please the result into the word **RESULT**.

Sample Problems

Input: **VALUE1** 00000007
 VALUE2 00000032
 Output: **RESULT** 000009F5

That is $7^2 + 50^2 = 49 + 2500 = 2549$ (decimal)
 or $7^2 + 32^2 = 31 + 9C4 = 9F5$ (hexadecimal)

6.3.7 Shift Left n bits

Shift the contents of the word **VALUE** left. The number of bits to shift is contained in the word **COUNT**. Assume that the shift count is less than 32. The low-order bits should be cleared.

Sample Problems

		Test A	Test B
Input:	VALUE	182B	182B
	COUNT	0003	0020
Output:	VALUE	C158	0000

In the first case the value is to be shifted left by three bits, while in the second case the same value is to be shifted by thirty two bits.

7 Program Loops

The program loop is the basic structure that forces the CPU to repeat a sequence of instructions. Loops have four sections:

1. The initialisation section, which establishes the starting values of counters, pointers, and other variables.
2. The processing section, where the actual data manipulation occurs. This is the section that does the work.
3. The loop control section, which updates counters and pointers for the next iteration.
4. The concluding section, that may be needed to analyse and store the results.

The computer performs Sections 1 and 4 only once, while it may perform Sections 2 and 3 many times. Therefore, the execution time of the loop depends mainly on the execution time of Sections 2 and 3. Those sections should execute as quickly as possible, while the execution times of Sections 1 and 4 have less effect on overall program speed.

There are typically two methods of programming a loop, these are the “repeat ... until” loop (Algorithm 7.1a) and the “while” loop (Algorithm 7.1b). The repeat-until loop results in the computer always executing the processing section of the loop at least once. On the other hand, the computer may not execute the processing section of the while loop at all. The repeat-until loop is more natural, but the while loop is often more efficient and eliminates the problem of going through the processing sequence once even where there is no data for it to handle.

The computer can use the loop structure to process large sets of data (usually called “arrays”). The simplest way to use one sequence of instructions to handle an array of data is to have the program increment a register (usually an index register or stack pointer) after each iteration. Then the register will contain the address of the next element in the array when the computer repeats the sequence of instructions. The computer can then handle arrays of any length with a single program.

Register indirect addressing is the key to the processing arrays since it allows you to vary the actual address of the data (the “*effective address*”) by changing the contents of a register. The autoincrementing mode is particularly convenient for processing arrays since it automatically updates the register for the next iteration. No additional instruction is necessary. You can even have an automatic increment by 2 or 4 if the array contains 16-bit or 32-bit data or addresses.

Although our examples show the processing of arrays with autoincrementing (adding 1, 2, or 4 after each iteration), the procedure is equally valid with autodecrementing (subtracting 1, 2, or 4 before

Algorithm 7.1a

Initialisation Section
Repeat
Processing Section
Loop Control Section
Until task completed
Concluding Section

Algorithm 7.1b

Initialisation Section
While task incomplete
Processing Section
Repeat

each iteration). Many programmers find moving backward through an array somewhat awkward and difficult to follow, but it is more efficient in many situations. The computer obviously does not know backward from forward. The programmer, however, must remember that the processor increments an address register after using it but decrements an address register before using it. This difference affects initialisation as follows:

1. When moving forward through an array (autoincrementing), start the register pointing to the lowest address occupied by the array.
2. When moving backward through an array (autodecrementing), start the register pointing one step (1, 2, or 4) beyond the highest address occupied by the array.

7.1 Program Examples

Program 7.1a: Ch5Ex1.s — *Add a series of 16 bit numbers by using a table address*

```

1  *      Add a series of 16 bit numbers by using a table address look-up
2
3      TTL      Ch5Ex1
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R0, =Data1          ;load the address of the lookup table
9      EOR      R1, R1, R1          ;clear R1 to store sum
10     LDR      R2, Length           ;init element count
11  Loop
12     LDR      R3, [R0]             ;get the data
13     ADD      R1, R1, R3           ;add it to r1
14     ADD      R0, R0, #+4          ;increment pointer
15     SUBS     R2, R2, #0x1         ;decrement count with zero set
16     BNE     Loop                 ;if zero flag is not set, loop
17     STR      R1, Result           ;otherwise done - store result
18     SWI      &11
19
20     AREA     Data1, DATA
21
22  Table DCW    &2040                ;table of values to be added
23         ALIGN 32 bit aligned
24         DCW    &1C22
25         ALIGN
26         DCW    &0242
27         ALIGN
28  TablEnd DCD  0
29
30     AREA     Data2, DATA
31  Length DCW    (TablEnd - Table) / 4 ;because we're having to align
32         ALIGN
33  Result DCW    0                   ;gives the loop count
34                                     ;storage for result
35     END

```

Program 7.1b: Ch5Ex2.s — *Add a series of 16 bit numbers by using a table address look-up*

```

1  *      Add a series of 16 bit numbers by using a table address look-up
2  *      This example has nothing in the lookup table, and the program handles this
3
4      TTL      Ch5Ex2
5      AREA     Program, CODE, READONLY

```

```

6          ENTRY
7
8  Main
9          LDR    R0, =Data1          ;load the address of the lookup table
10         EOR    R1, R1, R1          ;clear R1 to store sum
11         LDR    R2, Length          ;init element count
12         CMP    R2, #0
13         BEQ    Done
14  Loop
15         LDR    R3, [R0]            ;get the data that R0 points to
16         ADD    R1, R1, R3          ;add it to r1
17         ADD    R0, R0, #+4         ;increment pointer
18         SUBS    R2, R2, #0x1        ;decrement count with zero set
19         BNE    Loop              ;if zero flag is not set, loop
20  Done
21         STR    R1, Result          ;otherwise done - store result
22         SWI    &11
23
24         AREA   Data1, DATA
25
26  Table
27  TablEnd DCD    0                  ;Table is empty
28
29         AREA   Data2, DATA
30  Length  DCW    (TablEnd - Table) / 4 ;because we're having to align
31         ALIGN
32         Result DCW    0              ;storage for result
33
34         END

```

Program 7.1c: Ch5Ex3.s — *Scan a series of 32 bit numbers to find how many are negative*

```

1  *      Scan a series of 32 bit numbers to find how many are negative
2
3          TTL    Ch5Ex3
4          AREA   Program, CODE, READONLY
5          ENTRY
6
7  Main
8          LDR    R0, =Data1          ;load the address of the lookup table
9          EOR    R1, R1, R1          ;clear R1 to store count
10         LDR    R2, Length          ;init element count
11         CMP    R2, #0
12         BEQ    Done                ;if table is empty
13  Loop
14         LDR    R3, [R0]            ;get the data
15         CMP    R3, #0
16         BPL    Looptest            ;skip next line if +ve or zero
17         ADD    R1, R1, #1          ;increment -ve number count
18  Looptest
19         ADD    R0, R0, #+4          ;increment pointer
20         SUBS    R2, R2, #0x1        ;decrement count with zero set
21         BNE    Loop              ;if zero flag is not set, loop
22  Done
23         STR    R1, Result          ;otherwise done - store result
24         SWI    &11
25
26         AREA   Data1, DATA
27
28  Table  DCD    &F1522040            ;table of values to be added
29         DCD    &7F611C22
30         DCD    &80000242
31  TablEnd DCD    0
32
33         AREA   Data2, DATA
34  Length  DCW    (TablEnd - Table) / 4 ;because we're having to align

```

```

35      ALIGN                      ;gives the loop count
36 Result DCW      0              ;storage for result
37
38      END

```

Program 7.1d: Ch5Ex4.s — *Scan a series of 16 bit numbers to find how many are negative*

```

1  *      Scan a series of 16 bit numbers to find how many are negative
2
3      TTL      Ch5Ex4
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R0, =Data1          ;load the address of the lookup table
9      EOR      R1, R1, R1          ;clear R1 to store count
10     LDR      R2, Length          ;init element count
11     CMP      R2, #0
12     BEQ      Done               ;if table is empty
13 Loop
14     LDR      R3, [R0]            ;get the data
15     AND      R3, R3, #0x8000     ;bit wise AND to see if the 16th
16     CMP      R3, #0x8000         ;bit is 1
17     BMI      Looptest           ;skip next line if zero
18     ADD      R1, R1, #1          ;increment -ve number count
19 Looptest
20     ADD      R0, R0, #+4         ;increment pointer
21     SUBS     R2, R2, #0x1        ;decrement count with zero set
22     BNE      Loop              ;if zero flag is not set, loop
23 Done
24     STR      R1, Result          ;otherwise done - store result
25     SWI      &11
26
27     AREA     Data1, DATA
28
29 Table DCW      &F152             ;table of values to be tested
30     ALIGN
31     DCW      &7F61
32     ALIGN
33     DCW      &8000
34     ALIGN
35 TablEnd DCD    0
36
37     AREA     Data2, DATA
38 Length DCW     (TablEnd - Table) / 4 ;because we're having to align
39     ALIGN                      ;gives the loop count
40 Result DCW      0              ;storage for result
41
42     END

```

Program 7.1e: Ch5Ex5.s — *Scan a series of 16 bit numbers to find the largest*

```

1  *      Scan a series of 16 bit numbers to find the largest
2
3      TTL      Ch5Ex5
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R0, =Data1          ;load the address of the lookup table
9      EOR      R1, R1, R1          ;clear R1 to store largest
10     LDR      R2, Length          ;init element count
11     CMP      R2, #0

```

```

12      BEQ      Done          ;if table is empty
13  Loop      LDR      R3, [R0]      ;get the data
14            CMP      R3, R1        ;bit is 1
15            BCC      Looptest     ;skip next line if zero
16            MOV      R1, R3        ;increment -ve number count
17  Looptest   ADD      R0, R0, #+4   ;increment pointer
18            SUBS      R2, R2, #0x1  ;decrement count with zero set
19            BNE      Loop          ;if zero flag is not set, loop
20  Done      STR      R1, Result     ;otherwise done - store result
21            SWI      &11
22
23      AREA     Data1, DATA
24
25  Table     DCW      &A152          ;table of values to be tested
26            ALIGN
27            DCW      &7F61
28            ALIGN
29            DCW      &F123
30            ALIGN
31            DCW      &8000
32            ALIGN
33  TablEnd   DCD      0
34
35      AREA     Data2, DATA
36
37  Length    DCW      (TablEnd - Table) / 4 ;because we're having to align
38            ALIGN    ;gives the loop count
39  Result     DCW      0              ;storage for result
40
41      END

```

Program 7.1f: Ch5Ex6.s — Normalize a binary number

```

1  *      normalize a binary number
2
3      TTL      Ch5Ex6
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R0, =Data1          ;load the address of the lookup table
9      EOR      R1, R1, R1          ;clear R1 to store shifts
10     LDR      R3, [R0]            ;get the data
11     CMP      R3, R1              ;bit is 1
12     BEQ      Done                ;if table is empty
13  Loop
14     ADD      R1, R1, #1           ;increment pointer
15     MOVS      R3, R3, LSL#0x1    ;decrement count with zero set
16     BPL      Loop                ;if negative flag is not set, loop
17  Done
18     STR      R1, Shifted          ;otherwise done - store result
19     STR      R3, Normal
20     SWI      &11
21
22     AREA     Data1, DATA
23
24  Table
25  *      DCD      &30001000        ;table of values to be tested
26  *      DCD      &00000001
27  *      DCD      &00000000
28      DCD      &C1234567
29
30     AREA     Result, DATA

```

```

31
32  Number DCD      Table
33  Shifted DCB      0                ;storage for shift
34          ALIGN
35  Normal  DCD      0                ;storage for result
36
37          END

```

7.2 Problems

7.2.1 Checksum of data

Calculate the checksum of a series of 8-bit numbers. The length of the series is defined by the variable `LENGTH`. The label `START` indicates the start of the table. Store the checksum in the variable `CHECKSUM`. The checksum is formed by adding all the numbers in the list, ignoring the carry over (or overflow).

Note: Checksums are often used to ensure that data has been correctly read. A checksum calculated when reading the data is compared to a checksum that is stored with the data. If the two checksums do not agree, the system will usually indicate an error, or automatically read the data again.

Sample Problem:

Input:	<code>LENGTH</code>	00000003	(Number of items)
	<code>START</code>	28	(Start of data table)
		55	
		26	
Output:	<code>CHECKSUM</code>	28 + 55 + 26	(Data Checksum)
		= 00101000 (28)	
		+ 01010101 (55)	
		= 01111101 (7D)	
		+ 00100110 (26)	
		= 10100011 (A3)	

7.2.2 Number of Zero, Positive, and Negative numbers

Determine the number of zero, positive (most significant bit zero, but entire number not zero), and negative (most significant bit set) elements in a series of signed 32-bit numbers. The length of the series is defined by the variable `LENGTH` and the starting series of numbers start with the `START` label. Place the number of negative elements in variable `NUMNEG`, the number of zero elements in variable `NUMZERO` and the number of positive elements in variable `NUMPOS`.

Sample Problem:

Input:	<code>LENGTH</code>	6	(Number of items)
	<code>START</code>	76028326	(Start of data table — Positive)
		8D489867	(Negative)
		21202549	(Positive)
		00000000	(Zero)
		E605546C	(Negative)
		00000004	(Positive)
Output:	<code>NUMNEG</code>	2	(2 negative numbers: 8D489867 and E605546C)
	<code>NUMZERO</code>	1	(1 zero value)
	<code>NUMPOS</code>	3	(3 positive numbers: 76028326, 21202549 and 00000004)

7.2.3 Find Minimum

Find the smallest element in a series of unsigned bytes. The length of the series is defined by the variable **LENGTH** with the series starting at the **START** label. Store the minimum byte value in the **NUMMIN** variable.

Sample Problem:

Input:	LENGTH	5	(<i>Number of items</i>)
	START	65	(<i>Start of data table</i>)
		79	
		15	
		E3	
		72	
Output:	NUMMIN	15	(<i>Smallest of the five</i>)

7.2.4 Count 1 Bits

Determine the number of bits which are set in the 32-bit variable **NUM**, storing the result in the **NUMBITS** variable.

Sample Problem:

Input:	NUM	2866B794 = 0011 1000 0110 0110 1011 0111 1001 0100
Output:	NUMBITS	0F = 15

7.2.5 Find element with most 1 bits

Determine which element in a series of 32-bit numbers has the largest number of bits set. The length of the series is defined by the **LENGTH** variable and the series starts with the **START** label. Store the value with the most bits set in the **NUM** variable.

Sample Problem:

Input:	LENGTH	5	(<i>Number of items</i>)
	START	205A15E3	(0010 0000 0101 1010 0001 0101 1101 0011 — 13)
		256C8700	(0010 0101 0110 1100 1000 0111 0000 0000 — 11)
		295468F2	(0010 1001 0101 0100 0110 1000 1111 0010 — 14)
		29856779	(0010 1001 1000 0101 0110 0111 0111 1001 — 16)
		9147592A	(1001 0001 0100 0111 0101 1001 0010 1010 — 14)
Output:	NUM	29856779	(<i>Number with most 1-bits</i>)

8 *Character-Coded Data*

Microprocessors often handle data which represents printed characters rather than numeric quantities. Not only do keyboards, printers, communications devices, displays, and computer terminals expect or provide character-coded data, but many instruments, test systems, and controllers also require data in this form. ASCII (American Standard Code for Information Interchange) is the most commonly used code, but others exist.

We use the standard seven-bit ASCII character codes, as shown in Table 8.1; the character code occupies the low-order seven bits of the byte, and the most significant bit of the byte holds a 0 or a parity bit.

8.1 Handling data in ASCII

Here are some principles to remember in handling ASCII-coded data:

- The codes for the numbers and letters form ordered sequences. Since the ASCII codes for the characters “0” through “9” are 30_{16} through 39_{16} you can convert a decimal digit to the equivalent ASCII characters (and ASCII to decimal) by simple adding the ASCII offset: $30_{16} = \text{ASCII “0”}$. Since the codes for letters (41_{16} through $5A_{16}$ and 61_{16} through $7A_{16}$) are in order, you can alphabetise strings by sorting them according to their numerical values.
- The computer does not distinguish between printing and non-printing characters. Only the I/O devices make that distinction.
- An ASCII I/O device handles data only in ASCII. For example, if you want an ASCII printer to print the digit “7”, you must send it 37_{16} as the data; 07_{16} will ring the bell. Similarly, if a user presses the “9” key on an ASCII keyboard, the input data will be 39_{16} ; 09_{16} is the tab key.
- Many ASCII devices do not use the entire character set. For example, devices may ignore many control characters and may not print lower-case letters.
- Despite the definition of the control characters many devices interpret them differently. For example they typically uses control characters in a special way to provide features such as cursor control on a display, and to allow software control of characteristics such as rate of data transmission, print width, and line length.
- Some widely used ASCII control characters are:

$0A_{16}$	LF	line feed
$0D_{16}$	CR	carriage return
08_{16}	BS	backspace
$7F_{16}$	DEL	rub out or delete character

LSB	MSB								Control Characters			
	0	1	2	3	4	5	6	7				
0	NUL	DLE	SP	0	@	P	'	p	NUL	Null	DLE	Data link escape
1	SOH	DC1	!	1	A	Q	a	q	SOH	Start of heading	DC1	Device control 1
2	STX	DC2	"	2	B	R	b	r	STX	Start of text	DC2	Device control 2
3	ETX	DC3	#	3	C	S	c	s	ETX	End of text	DC3	Device control 3
4	EOT	DC4	\$	4	D	T	d	t	EOT	End of tx	DC4	Device control 4
5	ENQ	NAK	%	5	E	U	e	u	ENQ	Enquiry	NAK	Negative ack
6	ACK	SYN	&	6	F	V	f	v	ACK	Acknowledge	SYN	Synchronous idle
7	BEL	ETB	'	7	G	W	g	w	BEL	Bell, or alarm	ETB	End of tx block
8	BS	CAN	(	8	H	X	h	x	BS	Backspace	CAN	Cancel
9	HT	EM	)	9	I	Y	i	y	HT	Horizontal tab	EM	End of medium
A	LF	SUB	*	:	J	Z	j	z	LF	Line feed	SUB	Substitute
B	VT	ESC	+	;	K	[	k	{	VT	Vertical tab	ESC	Escape
C	FF	FS	,	<	L	\	l		FF	Form feed	FS	File separator
D	CR	GS	-	=	M	]	m	}	CR	Carriage return	GS	Group separator
E	SO	RS	.	>	N	^	n	~	SO	Shift out	RS	Record separator
F	SI	US	/	?	O	_	o	DEL	SI	Shift in	US	Unit separator
									SP	Space	DEL	Delete

Table 8.1: Hexadecimal ASCII Character Codes

- Each ASCII character occupies eight bits. This allows a large character set but is wasteful when only a few characters are actually being used. If, for example, the data consists entirely of decimal numbers, the ASCII format (allowing one digit per byte) requires twice as much storage, communications capacity, and processing time as the BCD format (allowing two digits per byte).

The assembler includes a feature to make character-coded data easy to handle, single quotation marks around a character indicate the character's ASCII value. For example,

```
MOV    R3, #'A'
```

is the same as

```
MOV    R3, #0x41
```

The first form is preferable for several reasons. It increases the readability of the instruction, it also avoids errors that may result from looking up a value in a table. The program does not depend on ASCII as the character set, since the assembler handles the conversion using whatever code has been designed for.

8.2 A string of characters

Individual characters on their own are not really all that helpful. As humans we need a string of characters in order to form meaningful text. In assembly programming it is normal to have to process one character at a time. However, the assembler does at least allow us to store a string of bytes (characters) in a friendly manner with the DCB directive. For example, line 26 of program 8.1a is:

```
DCB    "Hello, World", CR
```

which will produce the following binary data:

Binary:	48	65	6C	6C	6F	2C	20	57	6F	72	6C	64	0D
Text:	H	e	l	l	o	,	SP	W	o	r	l	d	CR

Use table 8.1 to check that this is correct. In order to make the program just that little bit more readable, line 5 defines the label `CR` to have the value for a Carriage Return (`0D16`).

There are three main methods for handling strings: Fixed Length, Terminated, and Counted. It is normal for a high level language to support just one method. `C/C++` and `Java` all support the use of Zero-Terminated strings, while `Pascal` and `Ada` use counted strings. Although it is possible to provide your own support for the alternative string type it is seldom done. A good programmer will use a mix of methods depending of the nature of the strings concerned.

8.2.1 Fixed Length Strings

A fixed length string is where the string is of a predefined and fixed size. For example, in a system where it is known that all strings are going to be ten characters in length, we can simply reserve 10 bytes for the string.

This has an immediate advantages in that the management of the strings is simple when compared to the alternative methods. For example we only need one label for an array of strings, and we can calculate the starting position of the n^{th} string by a simple multiplication.

This advantage is however also a major disadvantage. For example a persons name can be anything from two characters to any number of characters. Although it would be possible to reserve sufficient space for the longest of names this amount of memory would be required for all names, including the two letter ones. This is a significant waist of memory.

It would be possible to reserve just ten characters for each name. When a two letter name appears it would have to be padded out with spaces in order to make the name ten characters in length. When a name longer than ten characters appears it would have to be truncated down to just ten characters thus chopping off part of the name. This requires extra processing and is not entirely friendly to users who happen to have a long name.

When there is little memory and all the strings are known in advance it may be a good idea to use fixed length strings. For example, command driven systems tend to use a fixed length strings for the list of commands.

8.2.2 Terminated Strings

A terminated string is one that can be of any length and uses a special character to mark the end of the string, this character is known at the *sentinel*. For example program 8.1a uses the carriage return as it's sentinel.

Over the years several different sentinels have been used, these include `$` (`2616`), `EOT` (End of Text – `0416`), `CR` (Carriage Return – `0D16`), `LF` (Line Feed – `0A16`) and `NUL` (No character – `0016`). Today the most commonly used sentinel is the `NUL` character, primarily because it is used by `C/C++`. The `NUL` character also has a good feeling about it, as it is represented by the value 0, has no other meaning and it is easier to detected than any other character. This is frequently referred to as a Null- or Zero-Terminated string or simply as an ASCIIZ string.

The terminated string has the advantage that it can be of any length. Processing the string is fairly simply, you enter into a loop processing each character at a time until you reach the sentinel. The disadvantage is that the sentinel character can not appear in the string. This is another reason why the `NUL` character is such a good choice for the sentinel.

8.2.3 Counted Strings

A counted string is one in which the first one or two byte holds the length of the string in characters. Thus a counted string can be of any number of characters up to the largest unsigned number that can be stored in the first byte/word.

A counted string may appear rather clumsy at first. Having the length of the string as a binary value has a distinct advantage over the terminated string. It allow the use of the counting instructions that have been included in many instruction sets. This means we can ignore the testing for a sentinel character and simply decrement our counter, this is a far faster method of working.

To scan through an array of strings we simply point to the first string, and add the length count to our pointer to obtain the start of the next string. For a terminated string we would have to scan for the sentinel for each string.

There are two disadvantages with the counted string. The string does have a maximum length, 255 characters or 64K depending on the size of the count value (8- or 16-bit). Although it is normally felt that 64K should be sufficient for most strings. The second disadvantage is their perceived complexity. Many people feel that the complexity of the counted string outweighs the speed advantage.

8.3 International Characters

As computing expands outside of the English speaking world we have to provide support for languages other than standard American. Many European languages use letters that are not available in standard ASCII, for example: œ, Œ, ø, Ø, æ, Æ, ł, Ł, ß, į, and ı. This is particularly important when dealing with names: Ångström, Karlstraße or Łukasiewicz.

The ASCII character set is not even capable of handling English correctly. When we borrow a word from another language we also use it's *diacritic marks* (or *accents*). For example I would rather see pâté on a menu rather than pate. ASCII does not provide support for such accents.

To overcome this limitation the international community has produced a new character encoding, known as *Unicode*. In Unicode the character code is two bytes long, the first byte indicates which *character set* the character comes from, while the second byte indicates the character position within the character set. The traditional ASCII character set is incorporated into Unicode as character set zero. In the revised C standard a new data type of `wchar` was defined to cater for this new "wide character".

While Unicode is sufficient to represent the characters from most modern languages, it is not sufficient to represent all the written languages of the world, ancient and modern. Hence an extended version, known as Unicode-32 is being developed where the character set is a 23-bit value (three bytes). Unicode is a subset of Unicode-32, while ASCII is a subset of Unicode.

Although we do not consider Unicode you should be aware of the problem of international character sets and the solution Unicode provides.

8.4 Program Examples

Program 8.1a: Ch6Ex1.s — *Find the length of a string*

```

1  *      find the length of a string
2
3      TTL      Ch6Ex1
4
```

```

5  CR      EQU      0x0D
6
7          AREA     Program, CODE, READONLY
8          ENTRY
9
10 Main
11      ADR      R0, Data1          ;load the address of the lookup table
12      EOR      R1, R1, R1        ;clear R1 to store count
13  Loop
14      LDRB     R2, [R0], #1      ;load the first byte into R2
15      CMP      R2, #CR          ;is it the terminator
16      BEQ      Done            ;if not, Loop
17      ADD      R1, R1, #1        ;increment count
18      B        Loop
19  Done
20      STR      R1, CharCount      ;otherwise done - store result
21      SWI      &11
22
23      AREA     Data1, DATA
24
25  Table
26      DCB      "Hello, World", CR
27      ALIGN
28
29      AREA     Result, DATA
30  CharCount
31      DCB      0                  ;storage for count
32      ALIGN
33
34      END

```

Program 8.1b: Ch6Ex2.s — *Find the length of a null terminated string*

```

1  *      find the length of a null terminated string
2
3      TTL      Ch6Ex2
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      ADR      R0, Data1          ;load the address of the lookup table
9      MOV      R1, #-1           ;start count at -1
10  Loop
11      ADD      R1, R1, #1        ;increment count
12      LDRB     R2, [R0], #1      ;load the first byte into R2
13      CMP      R2, #0           ;is it the terminator
14      BNE      Loop
15
16      STR      R1, CharCount      ;otherwise done - store result
17      SWI      &11
18
19      AREA     Data1, DATA
20
21  Table
22      DCB      "Hello, World", 0
23      ALIGN
24
25      AREA     Result, DATA
26  CharCount
27      DCB      0                  ;storage for count
28      ALIGN
29
30      END

```

 Program 8.1c: Ch6Ex3.s — *Find the length of a string*

```

1  *      find the length of a string
2
3          TTL      Ch6Ex3
4
5  Blank  EQU      " "
6          AREA     Program, CODE, READONLY
7          ENTRY
8
9  Main
10         ADR      R0, Data1          ;load the address of the lookup table
11         MOV      R1, #Blank        ;store the blank char in R1
12  Loop
13         LDRB     R2, [R0], #1      ;load the first byte into R2
14         CMP      R2, R1            ;is it a blank
15         BEQ      Loop              ;if so loop
16
17         SUB      R0, R0, #1        ;otherwise done - adjust pointer
18         STR      R0, Pointer        ;and store it
19         SWI      &11
20
21         AREA     Data1, DATA
22
23  Table
24         DCB      "      7      "
25         ALIGN
26
27         AREA     Result, DATA
28  Pointer DCD      0                ;storage for count
29         ALIGN
30
31         END

```

 Program 8.1d: Ch6Ex4.s — *Supress leading zeros in a string*

```

1  *      supress leading zeros in a string
2
3          TTL      Ch6Ex4
4
5  Blank  EQU      " "
6  Zero   EQU      "0"
7          AREA     Program, CODE, READONLY
8          ENTRY
9
10 Main
11         ADR      R0, Data1          ;load the address of the lookup table
12         MOV      R1, #Zero          ;store the zero char in R1
13         MOV      R3, #Blank        ;and the blank char in R3
14  Loop
15         LDRB     R2, [R0], #1      ;load the first byte into R2
16         CMP      R2, R1            ;is it a zero
17         BNE      Done              ;if not, done
18
19         SUB      R0, R0, #1        ;otherwise adjust the pointer
20         STRB     R3, [R0]          ;and store it blank char there
21         ADD      R0, R0, #1        ;otherwise adjust the pointer
22         B        Loop              ;and loop
23
24  Done
25         SWI      &11                ;all done
26
27         AREA     Data1, DATA
28
29  Table

```

```

30      DCB      "000007000"
31      ALIGN
32
33      AREA     Result, DATA
34  Pointer DCD      0          ;storage for count
35      ALIGN
36
37      END

```

Program 8.1e: Ch6Ex5.s — *Set the parity bit on a series of characters store the amended string in Result*

```

1  *      set the parity bit on a series of characters
2  *      store the amended string in Result
3
4      TTL      Ch6Ex5
5
6      AREA     Program, CODE, READONLY
7      ENTRY
8
9  Main
10     ADR      R0, Data1          ;load the address of the lookup table
11     ADR      R5, Pointer
12     LDRB     R1, [R0], #1      ;store the string length in R1
13     CMP      R1, #0
14     BEQ      Done              ;nothing to do if zero length
15  MainLoop
16     LDRB     R2, [R0], #1      ;load the first byte into R2
17     MOV      R6, R2            ;keep a copy of the original char
18     MOV      R2, R2, LSL #24   ;shift so that we are dealing with msb
19     MOV      R3, #0            ;zero the bit counter
20     MOV      R4, #7            ;init the shift counter
21
22  ParLoop
23     MOVS     R2, R2, LSL #1     ;left shift
24     BPL      DontAdd           ;if msb is not a one bit, branch
25     ADD      R3, R3, #1        ;otherwise add to bit count
26  DontAdd
27     SUBS     R4, R4, #1        ;update shift count
28     BNE      ParLoop          ;loop if still bits to check
29     TST      R3, #1            ;is the parity even
30     BEQ      Even              ;if so branch
31     ORR      R6, R6, #0x80     ;otherwise set the parity bit
32     STRB     R6, [R5], #1      ;and store the amended char
33     B        Check
34  Even       STRB R6, [R5], #1   ;store the unamended char if even pty
35  Check      SUBS R1, R1, #1     ;decrement the character count
36     BNE      MainLoop
37
38  Done      SWI      &11
39
40     AREA     Data1, DATA
41
42  Table     DCB      6          ;data table starts with byte length of string
43            DCB      0x31      ;the string
44            DCB      0x32
45            DCB      0x33
46            DCB      0x34
47            DCB      0x35
48            DCB      0x36
49
50     AREA     Result, DATA
51     ALIGN
52  Pointer   DCD      0          ;storage for parity characters
53
54     END

```

 Program 8.1f: Ch6Ex6.s — *Compare two counted strings for equality*

```

1  *      compare two counted strings for equality
2
3      TTL      Ch6Ex6
4
5      AREA     Program, CODE, READONLY
6      ENTRY
7
8  Main
9      LDR      R0, =Data1          ;load the address of the lookup table
10     LDR      R1, =Data2
11     LDR      R2, Match           ;assume strings not equal - set to -1
12     LDR      R3, [R0], #4        ;store the first string length in R3
13     LDR      R4, [R1], #4        ;store the second string length in R4
14     CMP      R3, R4
15     BNE      Done               ;if they are different lengths,
16                                     ;they can't be equal
17     CMP      R3, #0             ;test for zero length if both are
18     BEQ      Same               ;zero length, nothing else to do
19
20  *      if we got this far, we now need to check the string char by char
21  Loop
22     LDRB      R5, [R0], #1        ;character of first string
23     LDRB      R6, [R1], #1        ;character of second string
24     CMP      R5, R6              ;are they the same
25     BNE      Done               ;if not the strings are different
26     SUBS      R3, R3, #1          ;use the string length as a counter
27     BEQ      Same               ;if we got to the end of the count
28                                     ;the strings are the same
29     B         Loop              ;not done, loop
30
31  Same   MOV      R2, #0           ;clear the -1 from match (0 = match)
32  Done   STR      R2, Match        ;store the result
33         SWI      &11
34
35     AREA     Data1, DATA
36  Table1 DCD      3                ;data table starts with byte length of string
37         DCB      "CAT"           ;the string
38
39     AREA     Data2, DATA
40  Table2 DCD      3                ;data table starts with byte length of string
41         DCB      "CAT"           ;the string
42
43     AREA     Result, DATA
44     ALIGN
45  Match  DCD      &FFFF           ;storage for parity characters
46
47     END

```

 Program 8.1g: Ch6Ex7.s — *Compare null terminated strings for equality assume that we have no knowledge of the data structure so we must assess the individual strings*

```

1  *      compare null terminated strings for equality
2  *      assume that we have no knowledge of the data structure
3  *      so we must assess the individual strings
4
5      TTL      Ch6Ex7
6
7      AREA     Program, CODE, READONLY
8      ENTRY
9
10 Main

```

```

11      LDR    R0, =Data1          ;load the address of the lookup table
12      LDR    R1, =Data2
13      LDR    R2, Match          ;assume strings not equal, set to -1
14      MOV    R3, #0             ;init register
15      MOV    R4, #0
16  Count1
17      LDRB   R5, [R0], #1        ;load the first byte into R5
18      CMP    R5, #0             ;is it the terminator
19      BEQ    Count2             ;if not, Loop
20      ADD    R3, R3, #1         ;increment count
21      B      Count1
22  Count2
23      LDRB   R5, [R1], #1        ;load the first byte into R5
24      CMP    R5, #0             ;is it the terminator
25      BEQ    Next              ;if not, Loop
26      ADD    R4, R4, #1         ;increment count
27      B      Count2
28
29  Next   CMP    R3, R4
30         BNE    Done            ;if they are different lengths,
31                                     ;they can't be equal
32         CMP    R3, #0          ;test for zero length if both are
33         BEQ    Same            ;zero length, nothing else to do
34         LDR    R0, =Data1      ;need to reset the lookup table
35         LDR    R1, =Data2
36
37  *      if we got this far, we now need to check the string char by char
38  Loop
39         LDRB   R5, [R0], #1     ;character of first string
40         LDRB   R6, [R1], #1     ;character of second string
41         CMP    R5, R6          ;are they the same
42         BNE    Done            ;if not the strings are different
43         SUBS   R3, R3, #1       ;use the string length as a counter
44         BEQ    Same            ;if we got to the end of the count
45                                     ;the strings are the same
46         B      Loop            ;not done, loop
47
48  Same
49         MOV    R2, #0          ;clear the -1 from match (0 = match)
50  Done
51         STR    R2, Match        ;store the result
52         SWI    &11
53
54         AREA   Data1, DATA
55  Table1 DCB    "Hello, World", 0 ;the string
56         ALIGN
57
58         AREA   Data2, DATA
59  Table2 DCB    "Hello, worl", 0  ;the string
60
61         AREA   Result, DATA
62         ALIGN
63  Match  DCD    &FFFF           ;flag for match
64
65      END

```

8.5 Problems

8.5.1 Length of a Teletypewriter Message

Determine the length of an ASCII message. All characters are 7-bit ASCII with MSB = 0. The string of characters in which the message is embedded has a starting address which is contained

in the **START** variable. The message itself starts with an ASCII *STX* (Start of Text) character (02_{16}) and ends with *ETX* (End of Text) character (03_{16}). Save the length of the message, the number of characters between the *STX* and the *ETX* markers (but not including the markers) in the **LENGTH** variable.

Sample Problem:

Input:	START	String	(<i>Location of string</i>)
		<i>String</i>	02 (STX — <i>Start Text</i>)
			47 (“G”)
			4F (“O”)
			03 (ETX — <i>End Text</i>)
Output:	LENGTH	02	(“GO”)

8.5.2 Find Last Non-Blank Character

Search a string of ASCII characters for the last non-blank character. Starting address of the string is contained in the **START** variable and the string ends with a carriage return character ($0D_{16}$). Place the address of the last non-blank character in the **POINTER** variable.

Sample Problems:

		Test A	Test B
Input:	START	String	String
	<i>String</i>	37 (“7”)	41 (“A”)
		0D (CR)	20 (Space)
			48 (“H”)
			41 (“A”)
			54 (“T”)
			20 (Space)
			20 (Space)
			0D (CR)
Output:	POINTER	<i>First Char</i>	<i>Fourth Char</i>

8.5.3 Truncate Decimal String to Integer Form

Edit a string of ASCII decimal characters by replacing all digits to the right of the decimal point with ASCII blanks (20_{16}). The starting address of the string is contained in the **START** variable and the string is assumed to consist entirely of ASCII-coded decimal digits and a possible decimal point ($2E_{16}$). The length of the string is stored in the **LENGTH** variable. If no decimal point appears in the string, assume that the decimal point is at the far right.

Sample Problems:

		Test A	Test B
Input:	START	String	String
	LENGTH	4	3
	<i>String</i>	37 (“7”)	36 (“6”)
		2E (“.”)	37 (“7”)
		38 (“8”)	31 (“1”)
		31 (“1”)	
Output:		37 (“7”)	36 (“6”)
		2E (“.”)	37 (“7”)
		20 (Space)	31 (“1”)
		20 (Space)	

Note that in the second case (Test B) the output is unchanged, as the number is assumed to be “671.”.

8.5.4 Check Even Parity and ASCII Characters

Check for even parity in a string of ASCII characters. A string's starting address is contained in the **START** variable. The first word of the string is its length which is followed by the string itself. If the parity of all the characters in the string are correct, clear the **PARITY** variable; otherwise place all ones (FFFFFFFF₁₆) into the variable.

Sample Problems:

		Test A	Test B
Input:	START	String	String
	<i>String</i>	3	03
		B1 (1011 0001)	B1 (1011 0001)
		B2 (1011 0010)	B6 (1011 0110)
		33 (0011 0011)	33 (0011 0011)
Output:	PARITY	00000000 (True)	FFFFFFFF (False)

8.5.5 String Comparison

Compare two strings of ASCII characters to see which is larger (that is, which follows the other in alphabetical ordering). Both strings have the same length as defined by the **LENGTH** variable. The strings' starting addresses are defined by the **START1** and **START2** variables. If the string defined by **START1** is greater than or equal to the other string, clear the **GREATER** variable; otherwise set the variable to all ones (FFFFFFFF₁₆).

Sample Problems:

		Test A	Test B	Test C
Input:	LENGTH	3	3	3
	START1	String1	String1	String1
	START2	String2	String2	String2
	<i>String1</i>	43 ("C")	43 ("C")	43 ("C")
		41 ("A")	41 ("A")	41 ("A")
		54 ("T")	54 ("T")	54 ("T")
	<i>String2</i>	42 ("B")	52 ("C")	52 ("C")
		41 ("A")	41 ("A")	55 ("U")
		54 ("T")	54 ("T")	54 ("T")
Output:	GREATER	00000000 (CAT > BAT)	00000000 (CAT = CAT)	FFFFFFFF (CAT < CUT)

9 Code Conversion

Code conversion is a continual problem in microcomputer applications. Peripherals provide data in ASCII, BCD or various special codes. The microcomputer must convert the data into some standard form for processing. Output devices may require data in ASCII, BCD, seven-segment or other codes. Therefore, the microcomputer must convert the results to the proper form after it completes the processing.

There are several ways to approach code conversion:

1. Some conversions can easily be handled by algorithms involving arithmetic or logical functions. The program may, however, have to handle special cases separately.
2. More complex conversions can be handled with lookup tables. The lookup table method requires little programming and is easy to apply. However the table may occupy a large amount of memory if the range of input values is large.
3. Hardware is readily available for some conversion tasks. Typical examples are decoders for BCD to seven-segment conversion and Universal Asynchronous Receiver/Transmitters (UARTs) for conversion between parallel and serial formats.

In most applications, the program should do as much as possible of the code conversion work. Most code conversions are easy to program and require little execution time.

9.1 Program Examples

Program 9.1a: Ch7Ex1.s — *Convert a single hex digit to its ASCII equivalent*

```
1  *      convert a single hex digit to its ASCII equivalent
2
3      TTL      Ch7Ex1
4
5      AREA     Program, CODE, READONLY
6      ENTRY
7
8  Main
9      LDR      R0, Digit          ;load the digit
10     LDR      R1, =Result        ;load the address for the result
11     CMP      R0, #0xA          ;is the number < 10 decimal
12     BLT      Add_0             ;then branch
13
14     ADD      R0, R0, #"A"-0xA   ;add offset for 'A' to 'F'
15  Add_0
16     ADD      R0, R0, #0"        ;convert to ASCII
17     STR      R0, [R1]          ;store the result
18     SWI      &11
19
20     AREA     Data1, DATA
```

```

21 Digit
22 DCD      &0C                ;the hex digit
23
24 AREA     Data2, DATA
25 Result DCD      0                ;storage for result
26
27 END

```

Program 9.1b: Ch7Ex2.s — *Convert a 32 bit hexadecimal number to an ASCII string and output to the terminal*

```

1  *      now something a little more adventurous - convert a 32 bit
2  *      hexadecimal number to an ASCII string and output to the terminal
3
4      TTL      Ch7Ex2
5
6      AREA     Program, CODE, READONLY
7      ENTRY
8  Mask EQU     0x0000000F
9
10 start
11      LDR      R1, Digit                ;load the digit
12      MOV      R4, #8                  ;init counter
13      MOV      R5, #28                 ;control right shift
14 MainLoop
15      MOV      R3, R1                  ;copy original word
16      MOV      R3, R3, LSR R5          ;right shift the correct number of bits
17      SUB      R5, R5, #4              ;reduce the bit shift
18      AND      R3, R3, #Mask           ;mask out all but the 1s nibble
19      CMP      R3, #0xA                ;is the number < 10 decimal
20      BLT      Add_0                  ;then branch
21
22      ADD      R3, R3, #"A"-0xA        ;add offset for 'A' to 'F'
23
24 Add_0 ADD      R3, R3, #"0"           ;convert to ASCII
25      MOV      R0, R3                  ;prepare to output
26      SWI      &0                      ;output to console
27      SUBS     R4, R4, #1              ;decrement counter
28      BNE      MainLoop
29
30      MOV      R0, #&OD                ;add a CR character
31      SWI      &0                      ;output it
32      SWI      &11                     ;all done
33
34      AREA     Data1, DATA
35 Digit DCD      &DEADBEEF            ;the hex word
36
37      END

```

Program 9.1c: Ch7Ex3.s — *Convert a decimal number to seven segment binary*

```

1  *      convert a decimal number to seven segment binary
2
3      TTL      Ch7Ex3
4
5      AREA     Program, CODE, READONLY
6      ENTRY
7
8  Main
9      LDR      R0, =Data1              ;load the start address of the table
10     EOR      R1, R1, R1              ;clear register for the code
11     LDRB     R2, Digit              ;get the digit to encode
12     CMP      R2, #9                 ;is it a valid digit?

```

```

13      BHI      Done          ;clear the result
14
15      ADD      R0, R0, R2    ;advance the pointer
16      LDRB     R1, [R0]     ;and get the next byte
17  Done
18      STR      R1, Result    ;store the result
19      SWI      &11          ;all done
20
21      AREA     Data1, DATA
22  Table DCB      &3F          ;the binary conversions table
23      DCB      &06
24      DCB      &5B
25      DCB      &4F
26      DCB      &66
27      DCB      &6D
28      DCB      &7D
29      DCB      &07
30      DCB      &7F
31      DCB      &6F
32      ALIGN
33
34      AREA     Data2, DATA
35  Digit DCB      &05          ;the number to convert
36      ALIGN
37
38      AREA     Data3, DATA
39  Result DCD      0           ;storage for result
40
41      END

```

Program 9.1d: Ch7Ex4.s — *Convert an ASCII numeric character to decimal*

```

1  *      convert an ASCII numeric character to decimal
2
3      TTL      Ch7Ex4
4
5      AREA     Program, CODE, READONLY
6      ENTRY
7
8  Main
9      MOV      R1, #-1        ;set -1 as error flag
10     LDRB     R0, Char        ;get the character
11     SUBS     R0, R0, #"0"    ;convert and check if character is < 0
12     BCC      Done           ;if so do nothing
13     CMP      R0, #9         ;check if character is > 9
14     BHI      Done           ;if so do nothing
15     MOV      R1, R0         ;otherwise....
16  Done
17     STR      R1, Result      ;.....store the decimal no
18     SWI      &11            ;all done
19
20     AREA     Data1, DATA
21  Char DCB      &37           ;ASCII representation of 7
22     ALIGN
23
24     AREA     Data2, DATA
25  Result DCD      0           ;storage for result
26
27     END

```

Program 9.1e: Ch7Ex5.s — *Convert an unpacked BCD number to binary*

```

1  *      convert an unpacked BCD number to binary

```

```

2
3     TTL      Ch7Ex5
4
5     AREA     Program, CODE, READONLY
6     ENTRY
7
8 Main
9     ADR      R0, BCDNum      ;load address of BCD number
10    MOV      R5, #4          ;init counter
11    MOV      R1, #0          ;clear result register
12    MOV      R2, #0          ;and final register
13
14 Loop
15    ADD      R1, R1, R1      ;multiply by 2
16    MOV      R3, R1
17    MOV      R3, R3, LSL #2  ;mult by 8 (2 x 4)
18    ADD      R1, R1, R3      ;= mult by 10
19
20 NoMult
21    LDRB     R4, [R0], #1     ;load digit and incr address
22    ADD      R1, R1, R4      ;add the next digit
23    SUBS     R5, R5, #1      ;decr counter
24    BNE      Loop           ;if counter != 0, loop
25
26    STR      R1, Result      ;store the result
27    SWI      &11            ;all done
28
29 BCDNum    AREA     Data1, DATA
30           DCB      &02,&09,&07,&01 ;an unpacked BCD number
31           ALIGN
32
33 Result    AREA     Data2, DATA
34           DCD      0        ;storage for result
35           END

```

Program 9.1f: Ch7Ex6.s — *Convert an unpacked BCD number to binary using MUL*

```

1 *      convert an unpacked BCD number to binary using MUL
2
3     TTL      Ch7Ex6
4
5     AREA     Program, CODE, READONLY
6     ENTRY
7
8 Main
9     ADR      R0, BCDNum      ;load address of BCD number
10    MOV      R5, #4          ;init counter
11    MOV      R1, #0          ;clear result register
12    MOV      R2, #0          ;and final register
13    MOV      R7, #10         ;multiplication constant
14
15 Loop
16    MOV      R6, R1
17    MUL      R1, R6, R7      ;mult by 10
18    LDRB     R4, [R0], #1     ;load digit and incr address
19    ADD      R1, R1, R4      ;add the next digit
20    SUBS     R5, R5, #1      ;decr counter
21    BNE      Loop           ;if count != 0, loop
22
23    STR      R1, Result      ;store the result
24    SWI      &11            ;all done
25
26 BCDNum    AREA     Data1, DATA
27           DCB      &02,&09,&07,&01 ;an unpacked BCD number
28           ALIGN
29

```

```

30      AREA      Data2, DATA
31 Result DCD      0                      ;storage for result
32
33      END

```

Program 9.1g: Ch7Ex7.s — Store a 16bit binary number as an ASCII string of '0's and '1's

```

1  *      store a 16bit binary number as an ASCII string of '0's and '1's
2
3      TTL      Ch7Ex7
4      AREA      Program, CODE, READONLY
5      ENTRY
6
7  Main
8      ADR      R0, String                ;load starttr address of string
9      ADD      R0, R0, #16                ;adjust for length of string
10     LDRB     R6, String                ;init counter
11     MOV      R2, #"1"                  ;load character '1' to register
12     MOV      R3, #"0"
13     LDR      R1, Number                ;load the number to process
14
15  Loop
16     MOVS     R1, R1, ROR #1             ;rotate right with carry
17     BCS      Loopend                   ;if carry set branch (LSB was a '1' bit)
18     STRB     R3, [R0], #-1             ;otherwise store "0"
19     B        Decr                      ;and branch to counter code
20  Loopend
21     STRB     R2, [R0], #-1             ;store a "1"
22  Decr
23     SUBS     R6, R6, #1                 ;decrement counter
24     BNE      Loop                     ;loop while not 0
25
26     SWI      &11
27
28     AREA      Data1, DATA
29  Number DCD      &31D2                  ;a 16 bit binary number number
30     ALIGN
31
32     AREA      Data2, DATA
33  String DCB      16                      ;storage for result
34     ALIGN
35
36     END

```

9.2 Problems

9.2.1 ASCII to Hexadecimal

Convert the contents of the A_DIGIT variable from an ASCII character to a hexadecimal digit and store the result in the H_DIGIT variable. Assume that A_DIGIT contains the ASCII representation of a hexadecimal digit (7 bits with MSB=0).

Sample Problems:

		Test A	Test B
Input:	A_DIGIT	43 ("C")	36 ("6")
Output:	H_DIGIT	0C	06

		7	6	5	4	3	2	1	0
		0	g	f	e	d	c	b	a
0	3F	0	0	1	1	1	1	1	1
1	06	0	0	0	0	0	1	1	0
2	5B	0	1	0	1	1	0	1	1
3	4F	0	1	0	0	1	1	1	1
4	66	0	1	1	0	0	1	1	0
5	6D	0	1	1	0	1	1	0	1
6	7D	0	1	1	1	1	1	0	1
7	07	0	0	0	0	0	1	1	1
8	7F	0	1	1	1	1	1	1	1
9	6F	0	1	1	0	1	1	1	1
A	77	0	1	1	1	0	1	1	1
B	7C	0	1	1	1	1	1	0	0
C	3A	0	0	1	1	1	0	0	1
D	5E	0	1	0	1	1	1	1	0
E	7A	0	1	1	1	1	0	0	1
F	71	0	1	1	1	0	0	0	1

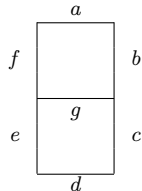


Figure 9.1: Seven-Segment Display

9.2.2 Seven-Segment to Decimal

Convert the contents of the CODE variable from a seven-segment code to a decimal number and store the result in the NUMBER variable. If CODE does not contain a valid seven-segment code, set NUMBER to FF₁₆. Use the seven-segment table given in Figure 9.1 and try to match codes.

Sample Problems:

		Test A	Test B
Input:	CODE	4F	28
Output:	NUMBER	03	FF

9.2.3 Decimal to ASCII

Convert the contents of the variable DIGIT from decimal digit to an ASCII character and store the result in the variable CHAR. If the number in DIGIT is not a decimal digit, set the contents of CHAR to an ASCII space (20₁₆).

Sample Problems:

		Test A	Test B
Input:	DIGIT	07	55
Output:	CHAR	37 (“7”)	20 (Space)

9.2.4 Binary to Binary-Coded-Decimal

Convert the contents of the variable NUMBER to four BCD digits in the STRING variable. The 32-bit number in NUMBER is unsigned and less than 10,000.

Sample Problem:

Input:	NUMBER	1C52	(7250 ₁₀)
Output:	STRING	07	(“7”)
		02	(“2”)
		05	(“5”)
		00	(“0”)

9.2.5 Packed Binary-Coded-Decimal to Binary String

Convert the eight digit packed binary-coded-decimal number in the BCDNUM variable into a 32-bit number in a NUMBER variable.

Sample Problem:

Input: BCDNUM 92529679
Output: NUMBER 0583E409₁₆ (92529679₁₀)

9.2.6 ASCII string to Binary number

Convert the eight ASCII characters in the variable **STRING** to an 8-bit binary number in the variable **NUMBER**. Clear the byte variable **ERROR** if all the ASCII characters are either ASCII “1” or ASCII “0”; otherwise set **ERROR** to all ones (FF₁₆).

Sample Problems:

		Test A		Test B	
Input:	STRING	31	(“1”)	31	(“1”)
		31	(“1”)	31	(“1”)
		30	(“0”)	30	(“0”)
		31	(“1”)	31	(“1”)
		30	(“0”)	30	(“0”)
		30	(“0”)	37	(“7”)
		31	(“1”)	31	(“1”)
		30	(“0”)	30	(“0”)
Output:	NUMBER	D2	(1101 0010)	00	(Valid)
	ERROR	0	(No Error)	FF	(Error)

10 *Arithmetic*

Much of the arithmetic in some microprocessor applications consists of multiple-word binary or decimal manipulations. The processor provides for decimal addition and subtraction, but does not provide for decimal multiplication or division, you must implement these operations with sequences of instruction.

Most processors provide for both signed and unsigned binary arithmetic. Signed numbers are represented in two's complement form. This means that the operations of addition and subtraction are the same whether the numbers are signed or unsigned.

Multiple-precision binary arithmetic requires simple repetitions of the basic instructions. The Carry flag transfers information between words. It is set when an addition results in a carry or a subtraction results in a borrow. Add with Carry and Subtract with Carry use this information from the previous arithmetic operation.

Decimal arithmetic is a common enough task for microprocessors that most have special instructions for this purpose. These instructions may either perform decimal operations directly or correct the results of binary operations to the proper decimal form. Decimal arithmetic is essential in such applications as point-of-sale terminals, check processors, order entry systems, and banking terminals.

You can implement decimal multiplication and division as series of additions and subtractions, respectively. Extra storage must be reserved for results, since a multiplication produces a result twice as long as the operands. A division contracts the length of the result. Multiplications and divisions are time-consuming when done in software because of the repeated operations that are necessary.

10.1 Program Examples

Program 10.1a: Ch8Ex1.s — *16 bit binary multiplication*

```
1  *      16 bit binary multiplication
2
3          TTL      Ch8Ex1
4          AREA     Program, CODE, READONLY
5          ENTRY
6
7  Main
8          LDR      R0, Number1          ;load first number
9          LDR      R1, Number2          ;and second
10         MUL      R0, R1, R0           ;x:= y * x
11  *      MUL      R0, R0, R1           ;won't work - not allowed
12         STR      R0, Result
13
14         SWI      &11                  ;all done
15
16         AREA     Data1, DATA
17  Number1 DCD     &706F                ;a 16 bit binary number
```

```

18  Number2 DCD    &0161                ;another
19          ALIGN
20
21          AREA    Data2, DATA
22  Result  DCD    0                    ;storage for result
23          ALIGN
24
25          END

```

Program 10.1b: Ch8Ex2.s — *Divide a 32 bit binary no by a 16 bit binary no store the quotient and remainder there is no 'DIV' instruction in ARM!*

```

1  *      divide a 32 bit binary no by a 16 bit binary no
2  *      store the quotient and remainder
3  *      there is no 'DIV' instruction in ARM!
4
5          TTL      Ch8Ex2
6          AREA     Program, CODE, READONLY
7          ENTRY
8
9  Main
10         LDR      R0, Number1          ;load first number
11         LDR      R1, Number2          ;and second
12         MOV      R3, #0                ;clear register for quotient
13  Loop
14         CMP      R1, #0                ;test for divide by 0
15         BEQ      Err
16         CMP      R0, R1                ;is the divisor less than the dividend?
17         BLT      Done                  ;if so, finished
18         ADD      R3, R3, #1            ;add one to quotient
19         SUB      R0, R0, R1            ;take away the number you first thought of
20         B        Loop                  ;and loop
21  Err
22         MOV      R3, #0xFFFFFFFF      ;error flag (-1)
23  Done
24         STR      R0, Remain            ;store the remainder
25         STR      R3, Quotient          ;and the quotient
26         SWI      &11                  ;all done
27
28         AREA     Data1, DATA
29  Number1 DCD      &0075CBB1           ;a 16 bit binary number
30  Number2 DCD      &0141                ;another
31         ALIGN
32
33         AREA     Data2, DATA
34  QuotientDCD      0                    ;storage for result
35  Remain  DCD      0                    ;storage for remainder
36         ALIGN
37
38         END

```

Program 10.1c: Ch8Ex3.s — *Add two packed BCD numbers to give a packed BCD result*

```

1  *      add two packed BCD numbers to give a packed BCD result
2
3          TTL      Ch8Ex3
4          AREA     Program, CODE, READONLY
5          ENTRY
6
7  Mask    EQU      0x0000000F
8
9  Main
10         LDR      R0, =Result          ;address for storage

```

```

11      LDR      R1, BCDNum1      ;load the first BCD number
12      LDR      R2, BCDNum2      ;and the second
13      LDRB     R8, Length        ;init counter
14      ADD      R0, R0, #3        ;adjust for offset
15      MOV      R5, #0            ;carry
16
17  Loop
18      MOV      R3, R1            ;copy what is left in the data register
19      MOV      R4, R2            ;and the other number
20      AND      R3, R3, #Mask     ;mask out everything except low order nibble
21      AND      R4, R4, #Mask     ;mask out everything except low order nibble
22      MOV      R1, R1, LSR #4    ;shift the original number one nibble
23      MOV      R2, R2, LSR #4    ;shift the original number one nibble
24      ADD      R6, R3, R4        ;add the digits
25      ADD      R6, R6, R5        ;and the carry
26      CMP      R6, #0xA          ;is it over 10?
27      BLT      RCarry1           ;if not, reset the carry to 0
28      MOV      R5, #1            ;otherwise set the carry
29      SUB      R6, R6, #0xA      ;and subtract 10
30      B        Next
31  RCarry1
32      MOV      R5, #0            ;carry reset to 0
33
34  Next
35      MOV      R3, R1            ;copy what is left in the data register
36      MOV      R4, R2            ;and the other number
37      AND      R3, R3, #Mask     ;mask out everything except low order nibble
38      AND      R4, R4, #Mask     ;mask out everything except low order nibble
39      MOV      R1, R1, LSR #4    ;shift the original number one nibble
40      MOV      R2, R2, LSR #4    ;shift the original number one nibble
41      ADD      R7, R3, R4        ;add the digits
42      ADD      R7, R7, R5        ;and the carry
43      CMP      R7, #0xA          ;is it over 10?
44      BLT      RCarry2           ;if not, reset the carry to 0
45      MOV      R5, #1            ;otherwise set the carry
46      SUB      R7, R7, #0xA      ;and subtract 10
47      B        Loopend
48
49  RCarry2
50      MOV      R5, #0            ;carry reset to 0
51  Loopend
52      MOV      R7, R7, LSL #4    ;shift the second digit processed to the left
53      ORR      R6, R6, R7        ;and OR in the first digit to the 1s nibble
54      STRB     R6, [R0], #-1     ;store the byte, and decrement address
55      SUBS     R8, R8, #1        ;decrement loop counter
56      BNE     Loop              ;loop while > 0
57      SWI     &11
58
59      AREA     Data1, DATA
60  Length DCB    &04
61      ALIGN
62  BCDNum1 DCB    &36, &70, &19, &85 ;an 8 digit packed BCD number
63
64      AREA     Data2, DATA
65  BCDNum2 DCB    &12, &66, &34, &59 ;another 8 digit packed BCD number
66
67      AREA     Data3, DATA
68  Result DCD    0                ;storage for result
69
70      END

```

Program 10.1d: Ch8Ex4.s — *Multiply two 32 bit number to give a 64 bit result (corrupts R0 and R1)*

```

1  *      multiply two 32 bit number to give a 64 bit result
2  *      (corrupts R0 and R1)

```

```

3
4      TTL      Ch8Ex4
5      AREA     Program, CODE, READONLY
6      ENTRY
7
8  Main
9      LDR      R0, Number1      ;load first number
10     LDR      R1, Number2      ;and second
11     LDR      R6, =Result      ;load the address of result
12     MOV      R5, R0, LSR #16  ;top half of R0
13     MOV      R3, R1, LSR #16  ;top half of R1
14     BIC      R0, R0, R5, LSL #16 ;bottom half of R0
15     BIC      R1, R1, R3, LSL #16 ;bottom half of R1
16     MUL      R2, R0, R1      ;partial result
17     MUL      R0, R3, R0      ;partial result
18     MUL      R1, R5, R1      ;partial result
19     MUL      R3, R5, R3      ;partial result
20     ADDS     R0, R1, R0      ;add middle parts
21     ADDCS    R3, R3, #&10000  ;add in any carry from above
22     ADDS     R2, R2, R0, LSL #16 ;LSB 32 bits
23     ADC      R3, R3, R0, LSR #16 ;MSB 32 bits
24
25     STR      R2, [R6]        ;store LSB
26     ADD      R6, R6, #4      ;increment pointer
27     STR      R3, [R6]        ;store MSB
28     SWI      &11            ;all done
29
30     AREA     Data1, DATA
31     Number1 DCD &12345678    ;a 16 bit binary number
32     Number2 DCD &ABCDEF01    ;another
33     ALIGN
34
35     AREA     Data2, DATA
36     Result  DCD 0            ;storage for result
37     ALIGN
38
39     END

```

10.2 Problems

10.2.1 Multiple precision Binary subtraction

Subtract one multiple-word number from another. The length in words of both numbers is in the `LENGTH` variable. The numbers themselves are stored (most significant bits First) in the variables `NUM1` and `NUM2` respectively. Subtract the number in `NUM2` from the one in `NUM1`. Store the difference in `NUM1`.

Sample Problem:

Input:	<code>LENGTH</code>	3	(Number of words in each number)
	<code>NUM1</code>	2F5B8568 84C32546 706C9567	(First number is 2F5B856884C32546706C9567 ₁₆)
	<code>NUM2</code>	14DF4098 85B81095 A3BC1284	(The second number is 14DF409885B81095A3BC1284 ₁₆)
Output:	<code>NUM1</code>	1A7C44CF FF0B14B0 CCB082E3	(Difference is 1A7C44CFFF0B14B0CCB082E3 ₁₆)

That is,

$$\begin{array}{r}
 2F5B856884C32546706C9567 \\
 - 14DF409885B81095A3BC1284 \\
 \hline
 1A7C44CFFF0B14B0CCB082E3
 \end{array}$$

10.2.2 Decimal Subtraction

Subtract one packed decimal (BCD) number from another. The length in bytes of both numbers is in the byte variable `LENGTH`. The numbers themselves are in the variables `NUM1` and `NUM2` respectively. Subtract the number contained in `NUM2` from the one contained in `NUM1`. Store the difference in `NUM1`.

Sample Problem:

Input:	<code>LENGTH</code>	4	(Number of bytes in each number)
	<code>NUM1</code>	36	(The first number is 367019857834 ₁₀)
		70	
		19	
		85	
		78	
		34	
	<code>NUM2</code>	12	(The second number is 126634593269 ₁₀)
		66	
		34	
		59	
		32	
		69	
Output:	<code>NUM1</code>	24	(Difference is 240385264565 ₁₀)
		03	
		85	
		26	
		45	
		65	

That is,

$$\begin{array}{r}
 367019857834 \\
 - 126634593269 \\
 \hline
 240385264565
 \end{array}$$

10.2.3 32-Bit by 32-Bit Multiply

Multiply the 32-bit value in the `NUM1` variable by the value in the `NUM2` variable. Use the `MULU` instruction and place the result in the 64-bit variable `PROD1`.

Sample Problem:

Input:	NUM1	0024 68AC	(<i>The first number is 2468AC₁₆</i>)
	NUM2	0328 1088	(<i>The second number is 3281088₁₀</i>)
Output:	PROD1	0000	
		72EC	(<i>MULU product is 72ECB8C25B60₁₆</i>)
		B8C2 5B60	
	PROD2	0000	
		72EC	(<i>Shift product is 72ECB8C25B60₁₆</i>)
		B8C2 5B60	

11 Tables and Lists

Tables and lists are two of the basic data structures used with all computers. We have already seen tables used to perform code conversions and arithmetic. Tables may also be used to identify or respond to commands and instructions, provide access to files or records, define the meaning of keys or switches, and choose among alternate programs. Lists are usually less structured than tables. Lists may record tasks that the processor must perform, messages or data that the processor must record, or conditions that have changed or should be monitored.

11.1 Program Examples

Program 11.1a: Ch9Ex1.s — *Examine a table for a match - store a new entry at the end if no match found*

```
1  *      examine a table for a match - store a new entry at
2  *      the end if no match found
3
4          TTL      Ch9Ex1
5          AREA     Program, CODE, READONLY
6          ENTRY
7
8  Main
9          LDR      R0, List           ;load the start address of the list
10         LDR      R1, NewItem        ;load the new item
11         LDR      R3, [R0]           ;copy the list counter
12         LDR      R2, [R0], #4       ;init counter and increment pointer
13         LDR      R4, [R0], #4
14  Loop
15         CMP      R1, R4             ;does the item match the list?
16         BEQ      Done              ;found it - finished
17         SUBS     R2, R2, #1          ;no - get the next item
18         LDR      R4, [R0], #4       ;get the next item
19         BNE      Loop              ;and loop
20
21         SUB      R0, R0, #4         ;adjust the pointer
22         ADD      R3, R3, #1         ;increment the number of items
23         STR      R3, Start          ;and store it back
24         STR      R1, [R0]          ;store the new item at the end of the list
25
26  Done    SWI      &11
27
28         AREA     Data1, DATA
29  Start   DCD      &4                ;length of list
30         DCD      &5376              ;items
31         DCD      &7615
32         DCD      &138A
33         DCD      &21DC
34  Store   %        &20              ;reserve 20 bytes of storage
35
36         AREA     Data2, DATA
37 NewItem  DCD      &16FA
```

```

38 List    DCD    Start
39
40         END

```

Program 11.1b: Ch9Ex2.s — *Examine a table for a match - store a new entry if no match found extends Ch9Ex1*

```

1  *      examine a table for a match - store a new entry if no match found
2  *      extends Ch9Ex1
3
4          TTL      Ch9Ex2
5          AREA     Program, CODE, READONLY
6          ENTRY
7
8  Main
9          LDR      R0, List           ;load the start address of the list
10         LDR      R1, NewItem        ;load the new item
11         LDR      R3, [R0]           ;copy the list counter
12         LDR      R2, [R0], #4       ;init counter and increment pointer
13         CMP      R3, #0             ;it's an empty list
14         BEQ      Insert             ;so store it
15         LDR      R4, [R0], #4       ;not empty - move to 1st item
16
17 Loop
18         CMP      R1, R4             ;does the item match the list?
19         BEQ      Done              ;found it - finished
20         SUBS     R2, R2, #1          ;no - get the next item
21         LDR      R4, [R0], #4       ;get the next item
22         BNE      Loop              ;and loop
23
24 Insert
25         SUB      R0, R0, #4          ;adjust the pointer
26         ADD      R3, R3, #1          ;incr list count
27         STR      R3, Start           ;and store it
28         STR      R1, [R0]           ;store new item at the end
29
30 Done
31         SWI      &11                ;all done
32
33         AREA     Data1, DATA
34         Start   DCD      &4          ;length of list
35         DCD      &5376              ;items
36         DCD      &7615
37         DCD      &138A
38         DCD      &21DC
39         Store   %      &20          ;reserve 20 bytes of storage
40
41         AREA     Data2, DATA
42         NewItem DCD      &16FA
43         List    DCD      Start
44
45         END

```

Program 11.1c: Ch9Ex3.s — *Examine an ordered table for a match*

```

1  *      examine an ordered table for a match
2
3          TTL      Ch9Ex3
4          AREA     Program, CODE, READONLY
5          ENTRY
6
7  Main
8          LDR      R0, =NewItem       ;load the address past the list
9          SUB      R0, R0, #4         ;adjust pointer to point at last element of list
10         LDR      R1, NewItem        ;load the item to test
11         LDR      R3, Start           ;init counter by reading index from list

```

```

12      CMP     R3, #0                ;are there zero items
13      BEQ     Missing              ;zero items in list - error condition
14      LDR     R4, [R0], #-4
15  Loop
16      CMP     R1, R4                ;does the item match the list?
17      BEQ     Done                  ;found it - finished
18      BHI     Missing              ;if the one to test is higher, it's not in the list
19      SUBS    R3, R3, #1            ;no - decr counter
20      LDR     R4, [R0], #-4        ;get the next item
21      BNE     Loop                  ;and loop
22                                      ;if we get to here, it's not there either
23  Missing MOV   R3, #0xFFFFFFFF    ;flag it as missing
24
25  Done   STR    R3, Index            ;store the index (either index or -1)
26      SWI     &11                  ;all done
27
28      AREA    Data1, DATA
29  Start  DCD    &4                  ;length of list
30      DCD    &0000138A              ;items
31      DCD    &000A21DC
32      DCD    &001F5376
33      DCD    &09018613
34
35      AREA    Data2, DATA
36 NewItem DCD    &001F5376
37  Index  DCW    0
38  List   DCD    Start
39
40      END

```

Program 11.1d: Ch9Ex4.s — *Remove the first element of a queue*

```

1  *      remove the first element of a queue
2
3      TTL      Ch9Ex4
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R0, Queue              ;load the head of the queue
9      STR      R1, Pointer            ;and save it in 'Pointer'
10     CMP      R0, #0                 ;is it NULL?
11     BEQ      Done                   ;if so, nothing to do
12
13     LDR      R1, [R0]                ;otherwise get the ptr to next
14     STR      R1, Queue              ;and make it the start of the queue
15
16  Done   SWI     &11
17
18     AREA     Data1, DATA
19  Queue  DCD     Item1                ;pointer to the start of the queue
20  Pointer DCD     0                    ;space to save the pointer
21
22  DArea  %      20                    ;space for new entries
23
24  * each item consists of a pointer to the next item, and some data
25  Item1  DCD     Item2                ;pointer
26      DCB      30, 20                ;data
27
28  Item2  DCD     Item3                ;pointer
29      DCB      30, 0xFF               ;data
30
31  Item3  DCD     0                    ;pointer (NULL)
32      DCB      30,&87,&65              ;data
33
34      END

```

 Program 11.1e: Ch9Ex5.s — *Sort a list of values - simple bubble sort*

```

1  *      sort a list of values - simple bubble sort
2
3      TTL      Ch9Ex5
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R6, List          ;pointer to start of list
9      MOV      R0, #0           ;clear register
10     LDRB     R0, [R6]          ;get the length of list
11     MOV      R8, R6           ;make a copy of start of list
12  Sort
13     ADD      R7, R6, R0        ;get address of last element
14     MOV      R1, #0           ;zero flag for changes
15     ADD      R8, R8, #1        ;move 1 byte up the list each
16  Next
17     LDRB     R2, [R7], #-1     ;load the first byte
18     LDRB     R3, [R7]         ;and the second
19     CMP      R2, R3           ;compare them
20     BCC      NoSwitch          ;branch if r2 less than r3
21
22     STRB     R2, [R7], #1      ;otherwise swap the bytes
23     STRB     R3, [R7]         ;like this
24     ADD      R1, R1, #1        ;flag that changes made
25     SUB      R7, R7, #1        ;decrement address to check
26  NoSwitch
27     CMP      R7, R8           ;have we checked enough bytes?
28     BHI      Next             ;if not, do inner loop
29     CMP      R1, #0           ;did we mke changes
30     BNE      Sort             ;if so check again - outer loop
31
32  Done   SWI      &11           ;all done
33
34     AREA     Data1, DATA
35  Start  DCB      6
36         DCB      &2A, &5B, &60, &3F, &D1, &19
37
38     AREA     Data2, DATA
39  List   DCD      Start
40
41     END

```

11.2 Problems

11.2.1 Remove Entry from List

Remove the value in the variable ITEM at a list if the value is present. The address of the list is in the LIST variable. The first entry in the list is the number (in words) of elements remaining in the list. Move entries below the one removed up one position and reduce the length of the list by one.

Sample Problems:

	Test A		Test B	
	Input	Output	Input	Output
ITEM	D010257		D0102596	
LIST	Table		Table	
Table	00000004	No change	00000004	0003
	2946C121	since item	C1212546	C1212546
	2054A346	not in list	D0102596	3A64422B
	05723A64		3A64422B	6C20432E
	12576C20		6C20432E	—

11.2.2 Add Entry to Ordered List

Insert the value in the variable **ITEM** into an ordered list if it is not already there. The address of the list is in the **LIST** variable. The first entry in the list is the list's length in words. The list itself consists of unsigned binary numbers in increasing order. Place the new entry in the correct position in the list, adjust the element below it down, and increase the length of the list by one.

Sample Problems

	Test A		Test B	
	Input	Output	Input	Output
ITEM	7A35B310		7A35B310	
LIST	<i>Table</i>		<i>Table</i>	
<i>Table</i>	00000004	<i>0005</i>	00000005	<i>No change</i>
	09250037	09250037	09250037	<i>since ITEM</i>
	29567322	29567322	29567322	<i>already in</i>
	A356A101	<i>7A35B310</i>	7A35B310	<i>list.</i>
	E235C203	<i>A356A101</i>	A356A101	
	—	<i>E235C203</i>	E235C203	

11.2.3 Add Element to Queue

Add the value in the variable **ITEM** to a queue. The address of the first element in the queue is in the variable **QUEUE**. Each element in the queue contains a structure of two items (*value* and *next*) where *next* is either the address of the next element in the queue or zero if there is no next element. The new element is placed at the end (tail) of the queue; the new element's address will be in the element that *was* at the end of the queue. The *next* entry of the new element will contain zero to indicate that it is now the end of the queue.

Sample Problem:

	Input		Output	
	Value	Next	Value	Next
ITEM	23854760		23854760	
QUEUE	00000001	item1	00000001	item1
item1	00000123	item2	00000123	item2
item2	00123456	00000000	00123456	item3
item3	—	—	23854760	00000000

11.2.4 4-Byte Sort

Sort a list of 4-byte entries into descending order. The first three bytes in each entry are an unsigned key with the first byte being the most significant. The fourth byte is additional information and should not be used to determine the sort order, but should be moved along with its key.

The number of entries in the list is defined by the word variable `LENGTH`. The list itself begins at location `LIST`.

Sample Problem:

	Input		Output	
<code>LENGTH</code>	00000004			
<code>LIST</code>	414243 07	("ABC")	<i>4A4B4C 13</i>	("JKL")
	4A4B4C 13	("JKL")	<i>4A4B41 37</i>	("JKA")
	4A4B41 37	("JKA")	<i>444B41 3F</i>	("DKA")
	444B41 3F	("DKA")	<i>414243 07</i>	("ABC")

11.2.5 Using a Jump Table with a Key

Using the value in the variable `INDEX` as a key to a jump table (`TABLE`). Each entry in the jump table contains a 32-bit identifier followed by a 32-bit address to which the program should transfer control if the key is equal to that identifier.

Sample Problem:

<code>INDEX</code>	00000010	
<code>TABLE</code>	00000001	<code>Proc1</code>
	00000010	<code>Proc2</code>
	0000001E	<code>Proc3</code>
<code>Proc1</code>	NOP	
<code>Proc2</code>	NOP	
<code>Proc3</code>	NOP	

Control should be transferred to `Proc2`, the second entry in the table.

12 *Subroutines*

None of the examples that we have shown thus far is a typical program that would stand by itself. Most real programs perform a series of tasks, many of which may be used a number of times or be common to other programs.

The standard method of producing programs which can be used in this manner is to write subroutines that perform particular tasks. The resulting sequences of instructions can be written once, tested once, and then used repeatedly.

There are special instructions for transferring control to subroutines and restoring control to the main program. We often refer to the special instruction that transfers control to a subroutine as Call, Jump, or Branch to a Subroutine. The special instruction that restores control to the main program is usually called Return.

In the ARM the Branch-and-Link instruction (BL) is used to Branch to a Subroutine. This saves the current value of the program counter (PC or R15) in the Link Register (LR or R14) before placing the starting address of the subroutine in the program counter. The ARM does not have a standard Return from Subroutine instruction like other processors, rather the programmer should copy the value in the Link Register into the Program Counter in order to return to the instruction after the Branch-and-Link instruction. Thus, to return from a subroutine you should use the instruction:

```
MOV    PC, LR
```

Should the subroutine wish to call another subroutine it will have to save the value of the Link Register before calling the nested subroutine.

12.1 Types of Subroutines

Sometimes a subroutine must have special characteristics.

Relocatable

The code can be placed anywhere in memory. You can use such a subroutine easily, regardless of other programs or the arrangement of the memory. A relocating loader is necessary to place the program in memory properly; the loader will start the program after other programs and will add the starting address or relocation constant to all addresses in the program.

Position Independent

The code does not require a relocating loader — all program addresses are expressed relative to the program counter's current value. Data addresses are held in registers at all times. We will discuss the writing of position independent code later in this chapter.

Reentrant

The subroutine can be interrupted and called by the interrupting program, giving the correct results for both the interrupting and interrupted programs. Reentrant subroutines are required for good for event based systems such as a multitasking operating system (Windows or Unix) and embedded real time environments. It is not difficult to make a subroutine reentrant. The only requirement is that the subroutine uses just registers and the stack for its data storage, and the subroutine is self contained in that it does not use any value defined outside of the routine (global values).

Recursive

The subroutine can call itself. Such a subroutine clearly must also be reentrant.

12.2 Subroutine Documentation

Most programs consist of a main program and several subroutines. This is useful as you can use known pre-written routines when available and you can debug and test the other subroutines properly and remember their exact effects on registers and memory locations.

You should provide sufficient documentation such that users need not examine the subroutine's internal structure. Among necessary specifications are:

- A description of the purpose of the subroutine
- A list of input and output parameters
- Registers and memory locations used
- A sample case, perhaps including a sample calling sequence

The subroutine will be easy to use if you follow these guidelines.

12.3 Parameter Passing Techniques

In order to be really useful, a subroutine must be general. For example, a subroutine that can perform only a specialized task, such as looking for a particular letter in an input string of fixed length, will not be very useful. If, on the other hand, the subroutine can look for any letter, in strings of any length, it will be far more helpful.

In order to provide subroutines with this flexibility, it is necessary to provide them with the ability to receive various kinds of information. We call data or addresses that we provide the subroutine parameters. An important part of writing subroutines is providing for transferring the parameters to the subroutine. This process is called Parameter Passing.

There are three general approaches to passing parameters:

1. Place the parameters in registers.
2. Place the parameters in a block of memory.
3. Transfer the parameters and results on the hardware stack.

The registers often provide a fast, convenient way of passing parameters and returning results. The limitations of this method are that it cannot be expanded beyond the number of available registers; it often results in unforeseen side effects; and it lacks generality.

The trade-off here is between fast execution time and a more general approach. Such a trade-off is common in computer applications at all levels. General approaches are easy to learn and consistent; they can be automated through the use of macros. On the other hand, approaches that take advantage of the specific features of a particular task require less time and memory. The choice of one approach over the other depends on your application, but you should take the general approach (saving programming time and simplifying documentation and maintenance) unless time or memory constraints force you to do otherwise.

12.3.1 Passing Parameters In Registers

The first and simplest method of passing parameters to a subroutine is via the registers. After calling a subroutine, the calling program can load memory addresses, counters, and other data into registers. For example, suppose a subroutine operates on two data buffers of equal length. The subroutine might specify that the length of the two data buffers be in the register *R0* while the starting address of the two data buffers are in the registers *R1* and *R2*. The calling program would then call the subroutine as follows:

```

MOV  R0, #BufferLen    ; Length of Buffer in R0
LDR  R1, =BufferA      ; Buffer A beginning address in R1
LDR  R2, =BufferB      ; Buffer B beginning address in R2
BL   Subr               ; Call subroutine

```

Using this method of parameter passing, the subroutine can simply assume that the parameters are there. Results can also be returned in registers, or the addresses of locations for results can be passed as parameters via the registers. Of course, this technique is limited by the number of registers available.

Processor features such as register indirect addressing, indexed addressing, and the ability to use any register as a stack pointer allow far more powerful and general ways of passing parameters.

12.3.2 Passing Parameters In A Parameter Block

Parameters that are to be passed to a subroutine can also be placed into memory in a parameter block. The location of this parameter block can be passed to the subroutine via a register.

```
LDR    R0, =Params    ; R0 Points to Parameter Block
BL     Subr           ; Call the subroutine
```

If you place the parameter block immediately after the subroutine call the address of the parameter block is automatically placed into the Link Register by the Branch and Link instruction. The subroutine must modify the return address in the Link Register in addition to fetching the parameters. Using this technique, our example would be modified as follows:

```
BL     Subr
DCD    BufferLen      ;Buffer Length
DCD    BufferA        ;Buffer A starting address
DCD    BufferB        ;Buffer B starting address
```

The subroutine saves' prior contents of CPU registers, then loads parameters and adjusts the return address as follows:

```
Subr   LDR    R0, [LR], #4    ; Read BuufferLen
        LDR    R1, [LR], #4    ; Read address of Buffer A
        LDR    R2, [LR], #4    ; Read address of Buffer B
                                ; LR points to next instruction
```

The addressing mode [LR], #4 will read the value at the address pointed to by the Link Register and then move the register on by four bytes. Thus at the end of this sequence the value of LR has been updated to point to the next instruction after the parameter block.

This parameter passing technique has the advantage of being easy to read. It has, however, the disadvantage of requiring parameters to be *fixed* when the program is written. Passing the address of the parameter block in via a register allows the parameters to be changed as the program is running.

12.3.3 Passing Parameters On The Stack

Another common method of passing parameters to a subroutine is to push the parameters onto the stack. Using this parameter passing technique, the subroutine call illustrated above would occur as follows:

```
MOV    R0, #BufferLen    ; Read Buffer Length
STR    R0, [SP, #-4]!     ; Save on the stack
LDR    R0, =BufferA      ; Read Address of Buffer A
STR    R0, [SP, #-4]!     ; Save on the stack
LDR    R0, =BufferA      ; Read Address of Buffer B
STR    R0, [SP, #-4]!     ; Save on the stack
BL     Subr
```

The subroutine must begin by loading parameters into CPU registers as follows:

```
Subr   STMIA   R12, {R0, R1, R2, R12, R14}    ; save working registers to stack
        LDR    R0, [R12, #0]                  ; Buffer Length in D0
        LDR    R1, [R12, #4]                  ; Buffer A starting address
        LDR    R2, [R12, #8]                  ; Buffer B starting address
        ...                                     ; Main function of subroutine
        LDMIA   R12, {R0, R1, R2, R12, R14}    ; Recover working registers
        MOV     PC, LR                        ; Return to caller
```

In this approach, all parameters are passed and results are returned on the stack.

The stack grows downward (toward lower addresses). This occurs because elements are pushed onto the stack using the pre-decrement address mode. The use of the pre-decrement mode causes the stack pointer to always contain the address of the last occupied location, rather than the next empty one as on some other microprocessors. This implies that you must initialise the stack pointer to a value higher than the largest address in the stack area.

When passing parameters on the stack, the programmer must implement this approach as follows:

1. Decrement the system stack pointer to make room for parameters on the system stack, and store them using offsets from the stack pointer, or simply push the parameters on the stack.
2. Access the parameters by means of offsets from the system stack pointer.
3. Store the results on the stack by means of offsets from the systems stack pointer.
4. Clean up the stack before or after returning from the subroutine, so that the parameters are removed and the results are handled appropriately.

12.4 Types Of Parameters

Regardless of our approach to passing parameters, we can specify the parameters in a variety of ways. For example, we can:

pass-by-value

Where the actual values are placed in the parameter list. The name comes from the fact that it is only the value of the parameter that is passed into the subroutine rather than the parameter itself. This is the method used by most high level programming languages.

pass-by-reference

The address of the parameters are placed in the parameter list. The subroutine can access the value directly rather than a copy of the parameter. This is much more dangerous as the subroutine can change a value you don't want it to.

pass-by-name

Rather than passing either the value or a reference to the value a string containing the name of the parameter is passed. This is used by very high level languages or scripting languages. This is very flexible but rather time consuming as we need to look up the value associated with the variable name every time we wish to access the variable.

12.5 Program Examples

Program 12.1a: Ch10Ex1.s — *Initiate a simple stack*

```

1  *      initiate a simple stack
2
3      TTL      Ch10Ex1
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R1, Value1          ;put some data into registers
9      LDR      R2, Value2
10     LDR      R3, Value3
11     LDR      R4, Value4
12
13     LDR      R7, =Data2          ;load the top of stack
14     STMFD    R7, {R1 - R4}      ;push the data onto the stack
15
16     SWI      &11                ;all done

```

```

17
18         AREA      Stack1, DATA
19 Value1 DCD      0xFFFF
20 Value2 DCD      0xDDDD
21 Value3 DCD      0AAAA
22 Value4 DCD      0x3333
23
24         AREA      Data2, DATA
25 Stack  %        40                ;reserve 40 bytes of memory for the stack
26 StackEnd
27         DCD      0
28
29         END

```

Program 12.1b: Ch10Ex2.s — *Initiate a simple stack*

```

1  *        initiate a simple stack
2
3          TTL      Ch10Ex2
4          AREA      Program, CODE, READONLY
5          ENTRY
6
7 Main
8          LDR      R1, Value1        ;put some data into registers
9          LDR      R2, Value2
10         LDR      R3, Value3
11         LDR      R4, Value4
12
13         LDR      R7, =Data2
14         STMDB    R7, {R1 - R4}
15
16         SWI      &11                ;all done
17
18         AREA      Stack1, DATA
19 Value1 DCD      0xFFFF
20 Value2 DCD      0xDDDD
21 Value3 DCD      0AAAA
22 Value4 DCD      0x3333
23
24         AREA      Data2, DATA
25 Stack  %        40                ;reserve 40 bytes of memory for the stack
26 StackEnd
27         DCD      0
28
29         END

```

Program 12.1c: Ch10Ex3.s — *Initiate a simple stack*

```

1  *        initiate a simple stack
2
3          TTL      Ch10Ex3
4          AREA      Program, CODE, READONLY
5          ENTRY
6
7 StackStart EQU      0x9000
8 Main
9          LDR      R1, Value1        ;put some data into registers
10         LDR      R2, Value2
11         LDR      R3, Value3
12         LDR      R4, Value4
13
14         LDR      R7, =StackStart    ;Top of stack = 9000
15         STMDB    R7, {R1 - R4}      ;push R1-R4 onto stack
16

```

```

17      SWI      &11                      ;all done
18
19      AREA     Data1, DATA
20  Value1 DCD    0xFFFF                  ;some data to put on stack
21  Value2 DCD    0xDDDD
22  Value3 DCD    0xAAAA
23  Value4 DCD    0x3333
24
25      AREA     Data2, DATA
26      ~       StackStart                ;reserve 40 bytes of memory for the stack
27  Stack1 DCD    0
28
29      END

```

Program 12.1d: Ch10Ex3a.s — *Initiate a simple stack*

```

1  *      initiate a simple stack
2
3      TTL      Ch10Ex4
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  StackStart EQU    0x9000
8  start
9      LDR      R1, Value1                ;put some data into registers
10     LDR      R2, Value2
11     LDR      R3, Value3
12     LDR      R4, Value4
13
14     LDR      R7, =StackStart            ;Top of stack = 9000
15     STMDB    R7, {R1 - R4}             ;push R1-R4 onto stack
16
17     SWI      &11                      ;all done
18
19     AREA     Data1, DATA
20  Value1 DCD    0xFFFF
21  Value2 DCD    0xDDDD
22  Value3 DCD    0xAAAA
23  Value4 DCD    0x3333
24
25     AREA     Data2, DATA
26     ~       StackStart                ;reserve 40 bytes of memory for the stack
27  Stack1 DCD    0
28
29     END

```

Program 12.1e: Ch10Ex4.s — *A simple subroutine example program passes a variable to the routine in a register*

```

1  *      a simple subroutine example
2  *      program passes a variable to the routine in a register
3
4      TTL      Ch10Ex4
5      AREA     Program, CODE, READONLY
6      ENTRY
7
8  StackStart EQU    0x9000
9  Main
10     LDRB     R0, HDigit                ;variable stored to register
11     BL       Hexdigit                  ;branch/link
12     STRB     R0, AChar                  ;store the result of the subroutine
13     SWI      &0                        ;output to console
14     SWI      &11                      ;all done
15

```

```

16 *      =====
17 *      Hexdigit subroutine
18 *      =====
19
20 *      Purpose
21 *      Hexdigit subroutine converts a Hex digit to an ASCII character
22 *
23 *      Initial Condition
24 *      R0 contains a value in the range 00 ... 0F
25 *
26 *      Final Condition
27 *      R0 contains ASCII character in the range '0' ... '9' or 'A' ... 'F'
28 *
29 *      Registers changed
30 *      R0 only
31 *
32 *      Sample case
33 *      Initial condition      R0 = 6
34 *      Final condition       R0 = 36 ('6')
35
36 Hexdigit
37     CMP     R0, #0xA           ;is it > 9
38     BLE     Addz              ;if not skip the next
39     ADD     R0, R0, #"A" - "0" - 0xA      ;adjust for A .. F
40
41 Addz
42     ADD     R0, R0, #"0"       ;convert to ASCII
43     MOV     PC, LR            ;return from subroutine
44
45     AREA    Data1, DATA
46 HDigit DCB 6                  ;digit to convert
47 AChar  DCB 0                  ;storage for ASCII character
48
49     END

```

Program 12.1f: Ch10Ex5.s — *A more complex subroutine example program passes variables to the routine using the stack*

```

1 *      a more complex subroutine example
2 *      program passes variables to the routine using the stack
3
4     TTL     Ch10Ex5
5     AREA    Program, CODE, READONLY
6     ENTRY
7
8 StackStart EQU 0x9000          ;declare where top of stack will be
9 Mask       EQU 0x0000000F      ;bit mask for masking out lower nibble
10
11 Main
12     LDR     R7, =StackStart     ;Top of stack = 9000
13     LDR     R0, Number          ;Load number to register
14     LDR     R1, =String         ;load address of string
15     STR     R1, [R7], #-4       ;and store it
16     STR     R0, [R7], #-4       ;and store number to stack
17     BL      Binhex              ;branch/link
18     SWI     &11                 ;all done
19
20 *      =====
21 *      Binhex subroutine
22 *      =====
23
24 *      Purpose
25 *      Binhex subroutine converts a 16 bit value to an ASCII string
26 *
27 *      Initial Condition
28 *      First parameter on the stack is the value

```

```

29 *      Second parameter is the address of the string
30 *
31 *      Final Condition
32 *      the HEX string occupies 4 bytes beginning with
33 *      the address passed as the second parameter
34 *
35 *      Registers changed
36 *      No registers are affected
37 *
38 *      Sample case
39 *      Initial condition      top of stack : 4CD0
40 *                          Address of string
41 *      Final condition      The string '4''C''D''0' in ASCII
42 *                          occupies memory
43
44 Binhex
45      MOV     R8, R7                ;save stack pointer for later
46      STMDA   R7, {R0-R6,R14}      ;push contents of R0 to R6, and LR onto the stack
47      MOV     R1, #4                ;init counter
48      ADD     R7, R7, #4            ;adjust pointer
49      LDR     R2, [R7], #4          ;get the number
50      LDR     R4, [R7]              ;get the address of the string
51      ADD     R4, R4, #4            ;move past the end of where the string is to be stored
52
53 Loop
54      MOV     R0, R2                ;copy the number
55      AND     R0, R0, #Mask         ;get the low nibble
56      BL      Hexdigit             ;convert to ASCII
57      STRB    R0, [R4], #-1         ;store it
58      MOV     R2, R2, LSR #4        ;shift to next nibble
59      SUBS    R1, R1, #1            ;decr counter
60      BNE     Loop                 ;loop while still elements left
61
62      LDMDA   R8, {R0-R6,R14}      ;restore the registers
63      MOV     PC, LR               ;return from subroutine
64
65 *      =====
66 *      Hexdigit subroutine
67 *      =====
68
69 *      Purpose
70 *      Hexdigit subroutine converts a Hex digit to an ASCII character
71 *
72 *      Initial Condition
73 *      R0 contains a value in the range 00 ... 0F
74 *
75 *      Final Condition
76 *      R0 contains ASCII character in the range '0' ... '9' or 'A' ... 'F'
77 *
78 *      Registers changed
79 *      R0 only
80 *
81 *      Sample case
82 *      Initial condition      R0 = 6
83 *      Final condition      R0 = 36 ('6')
84
85 Hexdigit
86      CMP     R0, #0xA              ;is the number 0 ... 9?
87      BLE     Addz                  ;if so, branch
88      ADD     R0, R0, #"A" - "0" - 0xA ;adjust for A ... F
89
90 Addz
91      ADD     R0, R0, #"0"          ;change to ASCII
92      MOV     PC, LR               ;return from subroutine
93
94      AREA    Data1, DATA
95      Number DCD    &4CD0          ;number to convert

```

```

96 String DCB      4, 0                ;counted string for result
97
98      END

```

Program 12.1g: Ch10Ex6.s — A 64 bit addition subroutine

```

1  *      a 64 bit addition subroutine
2
3      TTL      Ch10Ex6
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7 Main
8      BL       Add64                  ;branch/link
9      DCD      Value1                 ;address of parameter 1
10     DCD      Value2                 ;address of parameter 2
11
12     SWI      &11                    ;all done
13
14
15 *      =====
16 *      Add64 subroutine
17 *      =====
18
19 *      Purpose
20 *      Add two 64 bit values
21 *
22 *      Initial Condition
23 *      The two parameter values are passed immediately
24 *      following the subroutine call
25 *
26 *      Final Condition
27 *      The sum of the two values is returned in R0 and R1
28 *
29 *      Registers changed
30 *      R0 and R1 only
31 *
32 *      Sample case
33 *      Initial condition
34 *      para 1 =  = &0420147AEB529CB8
35 *      para 2 =  = &3020EB8520473118
36 *
37 *      Final condition
38 *      R0 = &34410000
39 *      R1 = &0B99CDD0
40
41 Add64
42     STMIA    R12, {R2, R3, R14}      ;save registers to stack
43     MOV      R7, R12                 ;copy stack pointer
44     SUB      R7, R7, #4               ;adjust to point at LSB of 2nd value
45     LDR      R3, [R7], #-4            ;load successive bytes
46     LDR      R2, [R7], #-4
47     LDR      R1, [R7], #-4
48     LDR      R0, [R7], #-4
49
50     ADDS     R1, R1, R3                ;add LS bytes & set carry flag
51     BCC      Next                    ;branch if carry bit not set
52     ADD      R0, R0, #1                ;otherwise add the carry
53 Next
54     ADD      R0, R0, R2                ;add MS bytes
55     LDMIA    R12, {R2, R3, R14}      ;pop from stack
56     MOV      PC, LR                  ;and return
57
58     AREA     Data1, DATA
59 Value1 DCD      &0420147A, &EB529CB8 ;number1 to add
60 Value2 DCD      &3020EB85, &20473118 ;number2 to add

```

61 END

 Program 12.1h: Ch10Ex7.s — *A subroutine to find the factorial of a number*

```

1  *      a subroutine to find the factorial of a number
2
3      TTL      Ch10Ex6
4      AREA     Program, CODE, READONLY
5      ENTRY
6
7  Main
8      LDR      R0, Number          ;get number
9      BL       Factor             ;branch/link
10     STR      R0, FNum           ;store the factorial
11
12     SWI      &11                ;all done
13
14
15 *      =====
16 *      Factor subroutine
17 *      =====
18
19 *      Purpose
20 *      Recursively find the factorial of a number
21 *
22 *      Initial Condition
23 *      R0 contains the number to factorial
24 *
25 *      Final Condition
26 *      R0 = factorial of number
27 *
28 *      Registers changed
29 *      R0 and R1 only
30 *
31 *      Sample case
32 *      Initial condition
33 *      Number = 5
34 *
35 *      Final condition
36 *      FNum = 120 = 0x78
37
38  Factor
39     STR      R0, [R12], #4        ;push to stack
40     STR      R14, [R12], #4       ;push the return address
41     SUBS     R0, R0, #1           ;subtract 1 from number
42     BNE      F_Cont              ;not finished
43
44     MOV      R0, #1               ;Factorial == 1
45     SUB      R12, R12, #4         ;adjust stack pointer
46     B        Return              ;done
47
48  F_Cont
49     BL       Factor              ;if not done, call again
50
51  Return
52     LDR      R14, [R12], #-4      ;return address
53     LDR      R1, [R12], #-4      ;load to R1 (can't do MUL R0, R0, xxx)
54     MUL      R0, R1, R0          ;multiply the result
55     MOV      PC, LR              ;and return
56
57     AREA     Data1, DATA
58  Number    DCD      5            ;number
59  FNum       DCD      0            ;factorial
60     END

```

12.6 Problems

Write both a calling program for the sample problem and at least one properly documented subroutine for each problem.

12.6.1 ASCII Hex to Binary

Write a subroutine to convert the least significant eight bits in register *R0* from the ASCII representation of a hexadecimal digit to the 4-bit binary representation of the digit. Place the result back into *R0*.

Sample Problems:

		<i>Test A</i>	<i>Test B</i>
Input:	<i>R0</i>	<u>43 'C'</u>	<u>36 '6'</u>
Output:	<i>R0</i>	0C	06

12.6.2 ASCII Hex String to Binary Word

Write a subroutine that takes the address of a string of eight ASCII characters in *R0*. It should convert the hexadecimal string into a 32-bit binary number, which it return is *R0*.

Sample Problem:

Input:	<i>R0</i>	<i>String</i>
	STRING	42 'B'
		32 '2'
		46 'F'
		30 '0'
Output:	<i>R0</i>	0000B2F0

12.6.3 Test for Alphabetic Character

Write a subroutine that checks the character in register *R0* to see if it is alphabetic (upper- or lower-case). It should set the Zero flag if the character is alphabetic, and reset the flag if it is not.

Sample Problems:

		<i>Test A</i>	<i>Test B</i>	<i>Test C</i>
Input:	<i>R0</i>	<u>47 'G'</u>	<u>36 '6'</u>	<u>6A 'j'</u>
Output:	<i>Z</i>	FF	00	FF

12.6.4 Scan to Next Non-alphabetic

Write a subroutien that takes the address of the start of a text string in register *R1* and returns the address of the first non-alphabetic character in the string in register *R1*. You should consider using the *isalpha* subroutine you have just define.

Sample Problems:

		<i>Test A</i>	<i>B</i>
Input:	<i>R1</i>	<u><i>String</i></u>	<u>6100</u>
	<i>String</i>	43 'C'	32 '2'
		61 'a'	50 'P'
		74 't'	49 'I'
		0D CR	0D CR
Output:	<i>R1</i>	<i>String</i> + 4	<i>String</i> + 0
		(CR)	(2)

12.6.5 Check Even Parity

Write a subroutine that takes the address of a counted string in the register **R0**. It should check for an even number of set bits in each character of the string. If all the bytes have an even parity then it should set the **Z**-flag, if one or more bytes have an odd parity it should clear the **Z**-flag.

Sample Problems:

Input:	R0 <i>String</i>	<i>Test A</i>	<i>Test B</i>
		<i>String</i>	<i>String</i>
		03	03
		47	47
		AF	AF
		18	19
Output:	Z	00	FF

Note that 19_{16} is $0001\ 1001_2$ which has three 1 bits and is thus has an odd parity.

12.6.6 Check the Checksum of a String

Write a subroutine to calculate the 8-bit checksum of the counted string pointed to by the register **R0** and compares the calculated checksum with the 8-bit checksum at the end of the string. It should set the **Z**-flag if the checksums are equal, and reset the flag if they are not.

Sample Problems:

Input:	R0 <i>String</i>	<i>Test A</i>	<i>Test B</i>
		<i>String</i>	<i>String</i>
		03 (Length)	03 (Length)
		41 ('A')	61 ('a')
		42 ('B')	62 ('b')
		43 ('C')	63 ('c')
		C6 (Checksum)	C6 (<i>Checksum should be 26j/em₆</i>)
Output:	Z	Set	Clear

12.6.7 Compare Two Counted Strings

Write a subroutine to compare two ASCII strings. The first byte in each string is its length. Return the result in the condition codes; i.e., the **N**-flag will be set if the first string is lexically less than (prior to) the second, the **Z**-flag will be set if the strings are equal, no flags are set if the second is prior to the first. Note that "ABCD" is lexically greater than "ABC".

A ARM Instructions

This chapter describes the syntax and usage of every ARM instruction, in the sections:

- Alphabetical list of ARM instructions
- ARM instructions and architecture versions

A.1 Alphabetical list of ARM instructions

Every ARM instruction is listed on the following pages. Each instruction description shows:

- the instruction encoding
- the instruction syntax
- the version of the ARM architecture where the instruction is valid
- any exceptions that apply
- an example in pseudo-code of how the instruction operates
- notes on usage and special cases.

A.1.1 General notes

These notes explain the types of information and abbreviations used on the instruction pages.

Syntax abbreviations

The following abbreviations are used in the instruction pages:

⟨immed n ⟩ This is an immediate value, where n is the number of bits. For example, an 8-bit immediate value is represented by:

⟨immed 8⟩

⟨offset n ⟩ This is an offset value, where n is the number of bits. For example, an 8-bit offset value is represented by:

⟨offset 8⟩

The same construction is used for signed offsets. For example, an 8-bit signed offset is represented by:

⟨signed offset 8⟩

Encoding diagram and assembler syntax

For the conventions used, see *Assembler syntax descriptions*.

Architecture versions

This gives details of architecture versions where the instruction is valid. For details, see *Architecture versions and variants*.

Exceptions

This gives details of which exceptions can occur during the execution of the instruction. Prefetch Abort is not listed in general, both because it can occur for any instruction and because if an abort occurred during instruction fetch, the instruction bit pattern is not known. (Prefetch Abort is however listed for BKPT, since it can generate a Prefetch Abort exception without these considerations applying.)

Operation

This gives a pseudo-code description of what the instruction does. For details of conventions used in this pseudo-code, see *Pseudo-code descriptions of instructions*.)

Information on usage

Usage sections are included where appropriate to supply suggestions and other information about how to use the instruction effectively.

ADC	Add with Carry
-----	----------------

Operation	$\langle cc \rangle: Rd \leftarrow Rn + \langle op1 \rangle + \text{CPSR}(C)$ $\langle cc \rangle \langle S \rangle: \text{CPSR} \leftarrow \text{ALU}(\text{Flags})$
Syntax	<code>ADC</code> $\langle cc \rangle \langle S \rangle$ <i>Rd</i> , <i>Rn</i> , $\langle op1 \rangle$
Description	The ADC (Add with Carry) instruction adds the value of $\langle op1 \rangle$ and the Carry flag to the value of <i>Rn</i> and stores the result in <i>Rd</i> . The condition code flags are optionally updated, based on the result.
Usage	ADC is used to synthesize multi-word addition. If register pairs <i>R0</i> , <i>R1</i> and <i>R2</i> , <i>R3</i> hold 64-bit values (where <i>R0</i> and <i>R2</i> hold the least significant words) the following instructions leave the 64-bit sum in <i>R4</i> , <i>R5</i> :

```

        ADDS  R4,R0,R2
        ADC   R5,R1,R3

```

If the second instruction is changed from:

```

        ADC   R5,R1,R3

```

to:

```

        ADCS  R5,R1,R3

```

the resulting values of the flags indicate:

N The 64-bit addition produced a negative result.

C An unsigned overflow occurred.

V A signed overflow occurred.

Z The most significant 32 bits are all zero.

The following instruction produces a single-bit Rotate Left with Extend operation (33-bit rotate through the Carry flag) on *R0*:

```

        ADCS  R0,R0,R0

```

See *Data-processing operands - Rotate right with extend* for information on how to perform a similar rotation to the right.

Condition Codes

The N and Z flags are set according to the result of the addition, and the C and V flags are set according to whether the addition generated a carry (unsigned overflow) and a signed overflow, respectively.

ADD	Add
Operation	$\langle cc \rangle: Rd \leftarrow Rn + \langle op1 \rangle$ $\langle cc \rangle \langle S \rangle: CPSR \leftarrow ALU(Flags)$
Syntax	ADD $\langle cc \rangle \langle S \rangle$ Rd, Rn, $\langle op1 \rangle$
Description	Adds the value of $\langle op1 \rangle$ to the value of register Rn, and stores the result in the destination register Rd. The condition code flags are optionally updated, based on the result.
Usage	The ADD instruction is used to add two values together to produce a third. To increment a register value in Rx use: <pre>ADD Rx, Rx, #1</pre> Constant multiplication of Rx by $2^n + 1$ into Rd can be performed with: <pre>ADD Rd, Rx, Rx, LSL #n</pre> To form a PC-relative address use: <pre>ADD Rs, PC, #offset</pre> where the $\langle offset \rangle$ must be the difference between the required address and the address held in the PC, where the PC is the address of the ADD instruction itself plus 8 bytes.

Condition Codes

The N and Z flags are set according to the result of the addition, and the C and V flags are set according to whether the addition generated a carry (unsigned overflow) and a signed overflow, respectively.

AND	Bitwise AND
Operation	$\langle cc \rangle: Rd \leftarrow Rn \wedge \langle op1 \rangle$ $\langle cc \rangle \langle S \rangle: CPSR \leftarrow ALU(Flags)$
Syntax	AND $\langle cc \rangle \langle S \rangle$ Rd, Rn, $\langle op1 \rangle$
Description	The AND instruction performs a bitwise AND of the value of register Rn with the value of $\langle op1 \rangle$, and stores the result in the destination register Rd. The condition code flags are optionally updated, based on the result.
Usage	AND is most useful for extracting a field from a register, by ANDing the register with a mask value that has 1s in the field to be extracted, and 0s elsewhere.

Condition Codes

The N and Z flags are set according to the result of the operation, and the C flag is set to the carry output generated by $\langle op1 \rangle$ (see 5.1 on page 59) The V flag is unaffected.

B, BL	Branch, Branch and Link
Operation	$\langle cc \rangle \langle L \rangle: LR \leftarrow PC + 8$ $\langle cc \rangle: PC \leftarrow PC + \langle offset \rangle$
Syntax	B $\langle L \rangle \langle cc \rangle$ $\langle offset \rangle$

Description The B (Branch) and BL (Branch and Link) instructions cause a branch to a target address, and provide both conditional and unconditional changes to program flow.

The BL (Branch and Link) instruction stores a return address in the link register (LR or R14).

The $\langle offset \rangle$ specifies the target address of the branch. The address of the next instruction is calculated by adding the offset to the program counter (PC) which contains the address of the branch instruction plus 8.

The branch instructions can specify a branch of approximately $\pm 32\text{MB}$.

Usage The BL instruction is used to perform a subroutine call. The return from subroutine is achieved by copying the LR to the PC. Typically, this is done by one of the following methods:

- Executing a MOV PC,R14 instruction.
- Storing a group of registers and R14 to the stack on subroutine entry, using an instruction of the form:

STMTD R13!, { $\langle registers \rangle$, R14}

and then restoring the register values and returning with an instruction of the form:

LDMFD R13!, { $\langle registers \rangle$, PC}

Condition Codes

The condition codes are not effected by this instruction.

Notes Branching backwards past location zero and forwards over the end of the 32-bit address space is UNPREDICTABLE.

CMP	Compare
Operation	$\langle cc \rangle: \text{ALU}(0) \leftarrow Rn - \langle op1 \rangle$ $\langle cc \rangle: \text{CSPR} \leftarrow \text{ALU}(\text{Flags})$
Syntax	CMP $\langle cc \rangle$ Rn, $\langle op1 \rangle$
Description	The CMP (Compare) instruction compares a register value with another arithmetic value. The condition flags are updated, based on the result of subtracting $\langle op1 \rangle$ from Rn, so that subsequent instructions can be conditionally executed.
Condition Codes	
The N and Z flags are set according to the result of the subtraction, and the C and V flags are set according to whether the subtraction generated a borrow (unsinged underflow) and a signed overflow, respectively.	

EOR	Exclusive OR
Operation	$\langle cc \rangle: Rd \leftarrow Rn \oplus \langle op1 \rangle$ $\langle cc \rangle \langle S \rangle: \text{CPSR} \leftarrow \text{ALU}(\text{Flags})$
Syntax	EOR $\langle cc \rangle \langle S \rangle$ Rd, Rn, $\langle op1 \rangle$
Description	The EOR (Exclusive OR) instruction performs a bitwise Exclusive-OR of the value of register Rn with the value of $\langle op1 \rangle$, and stores the result in the destination register Rd. The condition code flags are optionally updated, based on the result.
Usage	EOR can be used to invert selected bits in a register. For each bit, EOR with 1 inverts that bit, and EOR with 0 leaves it unchanged.
Condition Codes	
The N and Z flags are set according to the result of the operation, and the C flag is set to the carry output bit generated by the shifter. The V flag is unaffected.	

LDM	Load Multiple
-----	---------------

Operation	if $\langle cc \rangle$ IA: $\text{addr} \leftarrow Rn$ IB: $\text{addr} \leftarrow Rn + 4$ DA: $\text{addr} \leftarrow Rn - (\# \langle registers \rangle * 4) + 4$ DB: $\text{addr} \leftarrow Rn - (\# \langle registers \rangle * 4)$ for each register Ri in $\langle registers \rangle$ IB: $\text{addr} \leftarrow \text{addr} + 4$ DB: $\text{addr} \leftarrow \text{addr} - 4$ $Ri \leftarrow M(\text{addr})$ IA: $\text{addr} \leftarrow \text{addr} + 4$ DA: $\text{addr} \leftarrow \text{addr} - 1$ $\langle ! \rangle: Rn \leftarrow \text{addr}$								
Syntax	$\text{LDM} \langle cc \rangle \langle mode \rangle Rn \langle ! \rangle, \{ \langle registers \rangle \}$								
Description	The LDM (Load Multiple) instruction is useful for block loads, stack operations and procedure exit sequences. It loads a subset, or possibly all, of the general-purpose registers from sequential memory locations. The general-purpose registers loaded can include the PC. If they do, the word loaded for the PC is treated as an address and a branch occurs to that address. The register Rn points to the memory local to load the values from. Each of the registers listed in $\langle registers \rangle$ is loaded in turn, reading each value from the next memory address as directed by $\langle mode \rangle$, one of: <table> <tr> <td>IB</td><td>Increment Before</td></tr> <tr> <td>DB</td><td>Decrement Before</td></tr> <tr> <td>IA</td><td>Increment After</td></tr> <tr> <td>DA</td><td>Decrement After</td></tr> </table> The base register writeback option ($\langle ! \rangle$) causes the base register to be modified to hold the address of the final valued loaded. The register are loaded in sequence, the lowest-numbered register from the lowest memory address, through to the highest-numbered register from the highest memory address. If the PC ($R15$) is specified in the register list, the instruction causes a branch to the address loaded into the PC.	IB	Increment Before	DB	Decrement Before	IA	Increment After	DA	Decrement After
IB	Increment Before								
DB	Decrement Before								
IA	Increment After								
DA	Decrement After								
Exceptions	Data Abort								
Condition Codes	The condition codes are not effected by this instruction.								
Notes	If the base register Rn is specified in $\langle registers \rangle$, and base register writeback is specified ($\langle ! \rangle$), the final value of Rn is UNPREDICTABLE.								

LDR	Load Register
-----	---------------

Operation	$\langle cc \rangle: Rd \leftarrow M(\langle op2 \rangle)$
Syntax	$\text{LDR} \langle cc \rangle Rd, \langle op2 \rangle$
Description	The LDR (Load Register) instruction loads a word from the memory address calculated by $\langle op1 \rangle$ and writes it to register Rd. If the PC is specified as register Rd, the instruction loads a data word which it treats as an address, then branches to that address.
Exceptions	Data Abort

Usage Using the PC as the base register allows PC-relative addressing, which facilitates position-independent code. Combined with a suitable addressing mode, **LDR** allows 32-bit memory data to be loaded into a general-purpose register where its value can be manipulated. If the destination register is the PC, this instruction loads a 32-bit address from memory and branches to that address.

To synthesize a Branch with Link, precede the **LDR** instruction with **MOV LR, PC**.

Condition Codes

The condition codes are not effected by this instruction.

Notes If $\langle op2 \rangle$ specifies an address that is not word-aligned, the instruction attempts to load a byte. The result is UNPREDICTABLE and the **LDRB** instruction should be used.

If $\langle op2 \rangle$ specifies base register writeback (!), and the same register is specified for Rd and Rn , the results are UNPREDICTABLE.

If the PC ($R15$) is specified for Rd , the value must be word aligned otherwise the result is UNPREDICTABLE.

LDRB	Load Register Byte
Operation	$\langle cc \rangle: Rd(7:0) \leftarrow M(\langle op2 \rangle)$ $\langle cc \rangle: Rd(31:8) \leftarrow 0$
Syntax	LDR $\langle cc \rangle$ B Rd , $\langle op2 \rangle$
Description	The LDRB (Load Register Byte) instruction loads a byte from the memory address calculated by $\langle op2 \rangle$, zero-extends the byte to a 32-bit word, and writes the word to register Rd .
Exceptions	Data Abort
Usage	LDRB allows 8-bit memory data to be loaded into a general-purpose register where it can be manipulated. Using the PC as the base register allows PC-relative addressing, to facilitate position-independent code.
Condition Codes	The condition codes are not effected by this instruction.
Notes	If the PC ($R15$) is specified for Rd, the result is UNPREDICTABLE. If $\langle op2 \rangle$ specifies base register writeback (!), and the same register is specified for Rd and Rn, the results are UNPREDICTABLE.

MOV	Move
Operation	$\langle cc \rangle: Rd \leftarrow \langle op1 \rangle$ $\langle cc \rangle \langle S \rangle: CPSR \leftarrow ALU(Flags)$
Syntax	MOV $\langle cc \rangle \langle S \rangle$ Rd , $\langle op1 \rangle$
Description	The MOV (Move) instruction moves the value of $\langle op1 \rangle$ to the destination register Rd . The condition code flags are optionally updated, based on the result.
Usage	MOV is used to: • Move a value from one register to another. • Put a constant value into a register. • Perform a shift without any other arithmetic or logical operation. A left shift by n can be used to multiply by 2^n. • When the PC is the destination of the instruction, a branch occurs. The instruction: MOV PC, LR can therefore be used to return from a subroutine (see instructions <i>B</i>, and <i>BL</i> on page 137).

Condition Codes

The N and Z flags are set according to the value moved (post-shift if a shift is specified), and the C flag is set to the carry output bit generated by the shifter (see 5.1 on page 59). The V flag is unaffected.

MVN	Move Negative
Operation	$\langle cc \rangle: Rd \leftarrow \overline{\langle op1 \rangle}$ $\langle cc \rangle \langle S \rangle: CPSR \leftarrow ALU(Flags)$
Syntax	MVN $\langle cc \rangle \langle S \rangle$ Rd, $\langle op1 \rangle$
Description	The MVN (Move Negative) instruction moves the logical one's complement of the value of $\langle op1 \rangle$ to the destination register Rd. The condition code flags are optionally updated, based on the result.
Usage	MVN is used to: • Write a negative value into a register. • Form a bit mask. • Take the one's complement of a value.

Condition Codes

The N and Z flags are set according to the result of the operation, and the C flag is set to the carry output bit generated by the shifter (see 5.1 on page 59). The V flag is unaffected.

ORR	Bitwise OR
Operation	$\langle cc \rangle: Rd \leftarrow Rn \vee \langle op1 \rangle$ $\langle cc \rangle \langle S \rangle: CPSR \leftarrow ALU(Flags)$
Syntax	ORR $\langle cc \rangle \langle S \rangle$ Rd, Rn, $\langle op1 \rangle$
Description	The ORR (Logical OR) instruction performs a bitwise (inclusive) OR of the value of register Rn with the value of $\langle op1 \rangle$, and stores the result in the destination register Rd. The condition code flags are optionally updated, based on the result.
Usage	ORR can be used to set selected bits in a register. For each bit, OR with 1 sets the bit, and OR with 0 leaves it unchanged.

Condition Codes

The N and Z flags are set according to the result of the operation, and the C flag is set to the carry output bit generated by the shifter (see 5.1 on page 59). The V flag is unaffected.

SBC	Subtract with Carry
Operation	$\langle cc \rangle: Rd \leftarrow Rn - \langle op1 \rangle - \text{NOT}(CPSR(C))$ $\langle cc \rangle \langle S \rangle: CPSR \leftarrow ALU(Flags)$
Syntax	SBC $\langle cc \rangle \langle S \rangle$ Rd, Rn, $\langle op1 \rangle$
Description	The SBC (Subtract with Carry) instruction is used to synthesize multi-word subtraction. SBC subtracts the value of $\langle op1 \rangle$ and the value of NOT(Carry flag) from the value of register Rn, and stores the result in the destination register Rd. The condition code flags are optionally updated, based on the result.
Usage	If register pairs R0,R1 and R2,R3 hold 64-bit values (R0 and R2 hold the least significant words), the following instructions leave the 64-bit difference in R4,R5: SUBS R4,R0,R2 SBC R5,R1,R3

Condition Codes

The N and Z flags are set according to the result of the subtraction, and the C and V flags are set according to whether the subtraction generated a borrow (unsigned underflow) and a signed overflow, respectively.

Notes

If $\langle S \rangle$ is specified, the C flag is set to:

- 0 if no borrow occurs
- 1 if a borrow does occur

In other words, the C flag is used as a NOT(borrow) flag. This inversion of the borrow condition is usually compensated for by subsequent instructions. For example:

- The SBC and RSC instructions use the C flag as a NOT(borrow) operand, performing a normal subtraction if $C == 1$ and subtracting one more than usual if $C == 0$.
- The HS (unsigned higher or same) and LO (unsigned lower) conditions are equivalent to CS (carry set) and CC (carry clear) respectively.

STM	Store Multiple
-----	----------------

Operation

```

if  $\langle cc \rangle$ 
  IA:  $addr \leftarrow Rn$ 
  IB:  $addr \leftarrow Rn + 4$ 
  DA:  $addr \leftarrow Rn - (\# \langle registers \rangle * 4) + 4$ 
  DB:  $addr \leftarrow Rn - (\# \langle registers \rangle * 4)$ 

  for each register  $Ri$  in  $\langle registers \rangle$ 
    IB:  $addr \leftarrow addr + 4$ 
    DB:  $addr \leftarrow addr - 4$ 
    M(addr)  $\leftarrow Ri$ 
    IA:  $addr \leftarrow addr + 4$ 
    DA:  $addr \leftarrow addr - 4$ 

 $\langle ! \rangle$ :  $Rn \leftarrow addr$ 

```

Syntax

STM $\langle cc \rangle \langle mode \rangle$ Rn $\langle ! \rangle$, { $\langle registers \rangle$ }

Description

The STM (Store Multiple) instruction stores a subset (or possibly all) of the general-purpose registers to sequential memory locations.

The register Rn specifies the base register used to store the registers. Each register given in Rregisters is stored in turn, storing each register in the next memory address as directed by $\langle mode \rangle$, which can be one of:

IB	Increment Before
DB	Decrement Before
IA	Increment After
DA	Decrement After

If the base register writeback option ($\langle ! \rangle$) is specified, the base register (Rn) is modified with the new base address.

$\langle registers \rangle$ is a list of registers, separated by commas and specifies the set of registers to be stored. The registers are stored in sequence, the lowest-numbered register to the lowest memory address, through to the highest-numbered register to the highest memory address.

If R15 (PC) is specified in $\langle registers \rangle$, the value stored is UNKNOWN.

Exceptions

Data Abort

Usage

STM is useful as a block store instruction (combined with LDM it allows efficient block copy) and for stack operations. A single STM used in the sequence of a procedure can push the return address and general-purpose register values on to the stack, updating the stack pointer in the process.

Condition Codes

The condition codes are not effected by this instruction.

Notes

If **R15** (PC) is given as the base register (**Rn**), the result is UNPREDICTABLE.

If **Rn** is specified as $\langle registers \rangle$ and base register writeback ($\langle ! \rangle$) is specified:

- If **Rn** is the lowest-numbered register specified in $\langle registers \rangle$, the original value of **Rn** is stored.
- Otherwise, the stored value of **Rn** is UNPREDICTABLE.

The value of **Rn** should be word aligned.

STR	Store Register
Operation	$\langle cc \rangle: M(\langle op2 \rangle) \leftarrow Rd$
Syntax	STR $\langle cc \rangle$ Rd, $\langle op2 \rangle$
Description	The STR (Store Register) instruction stores a word from register Rd to the memory address calculated by $\langle op2 \rangle$.
Exceptions	Data Abort
Usage	Combined with a suitable addressing mode, STR stores 32-bit data from a general-purpose register into memory. Using the PC as the base register allows PC-relative addressing, which facilitates position-independent code.
Condition Codes	The condition codes are not effected by this instruction.
Notes	Using the PC as the source register (Rd) will cause an UNKNOWN value to be written. If $\langle op2 \rangle$ specifies base register writeback (!), and the same register is specified for Rd and Rn , the results are UNPREDICTABLE. The address calculated by $\langle op2 \rangle$ must be word-aligned. The result of a store to a non-word-aligned address is UNPREDICTABLE.
STRB	Store Register Byte

Operation	$\langle cc \rangle: M(\langle op2 \rangle) \leftarrow Rd(7:0)$
Syntax	STR $\langle cc \rangle$ B Rd, $\langle op2 \rangle$
Description	The STRB (Store Register Byte) instruction stores a byte from the least significant byte of register Rd to the memory address calculated by $\langle op2 \rangle$.
Exceptions	Data Abort
Usage	Combined with a suitable addressing mode, STRB writes the least significant byte of a general-purpose register to memory. Using the PC as the base register allows PC-relative addressing, which facilitates position-independent code.
Condition Codes	The condition codes are not effected by this instruction.
Notes	Specifying the PC as the source register (Rd) is UNPREDICTABLE. If $\langle op2 \rangle$ specifies base register writeback (!), and the same register is specified for Rd and Rn , the results are UNPREDICTABLE.

SUB	Subtract
Operation	$\langle cc \rangle: Rd \leftarrow Rn - \langle op1 \rangle$ $\langle cc \rangle \langle S \rangle: CPSR \leftarrow ALU(Flags)$
Syntax	<code>SUB</code> $\langle cc \rangle \langle S \rangle$ <i>Rd</i> , <i>Rn</i> , $\langle op1 \rangle$
Description	Subtracts the value of $\langle op1 \rangle$ from the value of register <i>Rn</i> , and stores the result in the destination register <i>Rd</i> . The condition code flags are optionally updated, based on the result.
Usage	SUB is used to subtract one value from another to produce a third. To decrement a register value (in <i>Rx</i>) use: <pre>SUBS Rx, Rx, #1</pre> SUBS is useful as a loop counter decrement, as the loop branch can test the flags for the appropriate termination condition, without the need for a compare instruction: <pre>CMP Rx, #0</pre> This both decrements the loop counter in <i>Rx</i> and checks whether it has reached zero.
Condition Codes	The N and Z flags are set according to the result of the subtraction, and the C and V flags are set according to whether the subtraction generated a borrow (unsigned underflow) and a signed overflow, respectively.
Notes	If $\langle S \rangle$ is specified, the C flag is set to: 1 if no borrow occurs 0 if a borrow does occur In other words, the C flag is used as a NOT(borrow) flag. This inversion of the borrow condition is usually compensated for by subsequent instructions. For example: • The SBC and RSC instructions use the C flag as a NOT(borrow) operand, performing a normal subtraction if C == 1 and subtracting one more than usual if C == 0. • The HS (unsigned higher or same) and LO (unsigned lower) conditions are equivalent to CS (carry set) and CC (carry clear) respectively.
SWI	Software Interrupt
Operation	$\langle cc \rangle: R14_svc \leftarrow PC + 8$ $\langle cc \rangle: SPSR_svc \leftarrow CPSR$ $\langle cc \rangle: CPSR(mode) \leftarrow \text{Supervisor}$ $\langle cc \rangle: CPSR(I) \leftarrow 1$ (Disable Interrupts) $\langle cc \rangle: PC \leftarrow 0x00000008$
Syntax	<code>SWI</code> $\langle cc \rangle$ $\langle value \rangle$
Description	Causes a SWI exception (see 3.4 on page 29).
Exceptions	Software interrupt
Usage	The SWI instruction is used as an operating system service call. The method used to select which operating system service is required is specified by the operating system, and the SWI exception handler for the operating system determines and provides the requested service. Two typical methods are: • $\langle value \rangle$ specifies which service is required, and any parameters needed by the selected service are passed in general-purpose registers. • $\langle value \rangle$ is ignored, general-purpose register <i>R0</i> is used to select which service is wanted, and any parameters needed by the selected service are passed in other general-purpose registers.

Condition Codes

The flags will be effected by the operation of the software interrupt. It is not possible to say how they will be effected. The status of the condition code flags is unknown after a software interrupt is UNKNOWN.

SWP	Swap
Operation	$\langle cc \rangle: \text{ALU}(0) \leftarrow M(Rn)$ $\langle cc \rangle: M(Rn) \leftarrow Rm$ $\langle cc \rangle: Rd \leftarrow \text{ALU}(0)$
Syntax	$\text{SWP} \langle cc \rangle \text{ } Rd, Rm, [Rn]$
Description	Swaps a word between registers and memory. SWP loads a word from the memory address given by the value of register Rn . The value of register Rm is then stored to the memory address given by the value of Rn , and the original loaded value is written to register Rd . If the same register is specified for Rd and Rm , this instruction swaps the value of the register and the value at the memory address.
Exceptions	Data Abort
Usage	The SWP instruction can be used to implement semaphores. For sample code, see <i>Semaphore instructions</i> .
Condition Codes	The condition codes are not effected by this instruction.
Notes	If the address contained in Rn is non word-aligned the effect is UNPREDICTABLE. If the PC is specified as the destination (Rd), address (Rn) or the value (Rm), the result is UNPREDICTABLE. If the same register is specified as $\langle Rn \rangle$ and $\langle Rm \rangle$, or $\langle Rn \rangle$ and $\langle Rd \rangle$, the result is UNPREDICTABLE. If a data abort is signaled on either the load access or the store access, the loaded value is not written to $\langle Rd \rangle$. If a data abort is signaled on the load access, the store access does not occur.

SWPB	Swap Byte
Operation	$\langle cc \rangle: \text{ALU}(0) \leftarrow M(Rn)$ $\langle cc \rangle: M(Rn) \leftarrow Rm(7:0)$ $\langle cc \rangle: Rd(7:0) \leftarrow \text{ALU}(0)$
Syntax	$\text{SWPB} \langle cc \rangle B \text{ } Rd, Rm, [Rn]$
Description	Swaps a byte between registers and memory. SWPB loads a byte from the memory address given by the value of register Rn . The value of the least significant byte of register Rm is stored to the memory address given by Rn , the original loaded value is zero-extended to a 32-bit word, and the word is written to register Rd . If the same register is specified for Rd and Rm , this instruction swaps the value of the least significant byte of the register and the byte value at the memory address.
Exceptions	Data Abort
Usage	The SWPB instruction can be used to implement semaphores, in a similar manner to that shown for SWP instructions in <i>Semaphore instructions</i> .
Condition Codes	The condition codes are not effected by this instruction.
Notes	If the PC is specified for Rd, Rn, or Rm, the result is UNPREDICTABLE. If the same register is specified as Rn and Rm, or Rn and Rd, the result is UNPREDICTABLE. If a data abort is signaled on either the load access or the store access, the loaded value is not written to $\langle Rd \rangle$. If a data abort is signaled on the load access, the store access does not occur.

B ARM Instruction Summary

<i>cc: Condition Codes</i>		
	<i>Generic</i>	<i>Unsigned</i>
CS	Carry Set	HI Higer Than
CC	Carry Clear	HS Higer or Same
EQ	Equal (Zero Set)	LO Lower Than
NE	Not Equal (Zero Clear)	LS Lower Than or Same
VS	Overflow Set	
VC	Overflow Clear	
		<i>Signed</i>
		GT Greater Than
		GE Greater Than or Equal
		LT Less Than
		LE Less Than or Equal
		MI Minus (Negative)
		PL Plus (Positive)

<i>op1: Data Access</i>		
Immediate	$\# \langle value \rangle$	$\langle op1 \rangle \leftarrow \text{IR}(\text{value})$
Register	Rm	$\langle op1 \rangle \leftarrow Rm$
Logical Shift Left Immediate	$Rm, \text{LSL } \# \langle value \rangle$	$\langle op1 \rangle \leftarrow Rm \ll \text{IR}(\text{value})$
Logical Shift Left Register	$Rm, \text{LSL } Rs$	$\langle op1 \rangle \leftarrow Rm \ll Rs(7:0)$
Logical Shift Right Immediate	$Rm, \text{LSR } \# \langle value \rangle$	$\langle op1 \rangle \leftarrow Rm \gg \text{IR}(\text{value})$
Logical Shift Right Register	$Rm, \text{LSR } Rs$	$\langle op1 \rangle \leftarrow Rm \gg Rs(7:0)$
Arithmetic Shift Right Immediate	$Rm, \text{ASR } \# \langle value \rangle$	$\langle op1 \rangle \leftarrow Rm \ggg \text{IR}(\text{value})$
Arithmetic Shift Right Register	$Rm, \text{ASR } Rs$	$\langle op1 \rangle \leftarrow Rm \ggg Rs(7:0)$
Rotate Right Immediate	$Rm, \text{ROR } \# \langle value \rangle$	$\langle op1 \rangle \leftarrow Rm \ggg \langle value \rangle$
Rotate Right Register	$Rm, \text{ROR } Rs$	$\langle op1 \rangle \leftarrow Rm \ggg Rs(4:0)$
Rotate Right with Extend	Rm, RRX	$\langle op1 \rangle \leftarrow \text{CPSR}(C) \ggg Rm \ggg \text{CPSR}(C)$

<i>op2: Memory Access</i>		
Immediate Offset	$[Rn, \# \pm \langle value \rangle]$	$\langle op2 \rangle \leftarrow Rn + \text{IR}(\text{value})$
Register Offset	$[Rn, Rm]$	$\langle op2 \rangle \leftarrow Rn + Rm$
Scaled Register Offset	$[\langle Rn \rangle, Rm, \langle shift \rangle \# \langle value \rangle]$	$\langle op2 \rangle \leftarrow Rn + (Rm \text{ shift } \text{IR}(\text{value}))$
Immediate Pre-indexed	$[Rn, \# \pm \langle value \rangle]!$	$\langle op2 \rangle \leftarrow Rn + \text{IR}(\text{value})$ $Rn \leftarrow \langle op2 \rangle$
Register Pre-indexed	$[Rn, Rm]!$	$\langle op2 \rangle \leftarrow Rn + Rm$ $Rn \leftarrow \langle op2 \rangle$
Scaled Register Pre-indexed	$[Rn, Rm, \langle shift \rangle \# \langle value \rangle]!$	$\langle op2 \rangle \leftarrow Rn + (Rm \text{ shift } \text{IR}(\text{value}))$ $Rn \leftarrow \langle op2 \rangle$
Immediate Post-indexed	$[Rn], \# \pm \langle value \rangle$	$\langle op2 \rangle \leftarrow Rn$ $Rn \leftarrow Rn + \text{IR}(\text{value})$
Register Post-indexed	$[Rn], Rm$	$\langle op2 \rangle \leftarrow Rn$ $Rn \leftarrow Rn + Rm$
Scaled Register Post-indexed	$[Rn], Rm, \langle shift \rangle \# \langle value \rangle$	$\langle op2 \rangle \leftarrow Rn$ $Rn \leftarrow Rn + Rm \text{ shift } \text{IR}(\text{value})$

Where $\langle shift \rangle$ is one of: LSL, LSR, ASR, ROR or RRX and has the same effect as for $\langle op1 \rangle$

<i>ARM Instructions</i>				
Add with Carry	ADC<cc><S>	Rd, Rn, <op1>	<cc>: Rd	$\leftarrow Rn + \langle op1 \rangle + \text{CPSR}(C)$
Add	ADD<cc><S>	Rd, Rn, <op1>	<cc>: Rd	$\leftarrow Rn + \langle op1 \rangle$
Bitwise AND	AND<cc><S>	Rd, Rn, <op1>	<cc>: Rd	$\leftarrow Rn \wedge \langle op1 \rangle$
Branch	B<cc>	<offset>	<cc>: PC	$\leftarrow PC + \langle offset \rangle$
Branch and Link	BL<cc>	<offset>	<cc>: LR	$\leftarrow PC + 8$
			<cc>: PC	$\leftarrow PC + \langle offset \rangle$
Compare	CMP<cc>	Rn, <op1>	<cc>: CPSR	$\leftarrow (Rn - \langle op1 \rangle)$
Exclusive OR	EOR<cc><S>	Rd, Rn, <op1>	<cc>: Rd	$\leftarrow Rn \oplus \langle op1 \rangle$
Load Register	LDR<cc>	Rd, <op2>	<cc>: Rd	$\leftarrow M(\langle op2 \rangle)$
Load Register Byte	LDR<cc>B	Rd, <op2>	<cc>: Rd(7:0)	$\leftarrow M(\langle op2 \rangle)$
			<cc>: Rd(31:8)	$\leftarrow 0$
Move	MOV<cc><S>	Rd, <op1>	<cc>: Rd	$\leftarrow \langle op1 \rangle$
Move Negative	MVN<cc><S>	Rd, <op1>	<cc>: Rd	$\leftarrow \overline{\langle op1 \rangle}$
Bitwise OR	ORR<cc><S>	Rd, Rn, <op1>	<cc>: Rd	$\leftarrow Rn \vee \langle op1 \rangle$
Subtract with Carry	SBC<cc><S>	Rd, Rn, <op1>	<cc>: Rd	$\leftarrow Rn - \langle op1 \rangle - \overline{\text{CPSR}(C)}$
Store Register	STR<cc>	Rd, <op2>	<cc>: M(<op2>)	$\leftarrow Rd$
Store Register Byte	STR<cc><S>	Rd, <op2>	<cc>: M(<op2>)	$\leftarrow Rd(7:0)$
Subtract	SUB<cc><S>	Rd, Rn, <op1>	<cc>: Rd	$\leftarrow Rn - \langle op1 \rangle$
Software Interrupt	SWI<cc>	<value>		
Swap	SWP<cc>	Rd, Rm, [Rn]	<cc>: Rd	$\leftarrow M(Rn)$
			<cc>: M(Rn)	$\leftarrow Rm$
Swap Byte	SWP<cc>B	Rd, Rm, [Rn]	<cc>: Rd(7:0)	$\leftarrow M(Rn)(7:0)$
			<cc>: M(Rn)(7:0)	$\leftarrow Rm(7:0)$

Index

Characters		
ASCII	91	
International	94	
Unicode	94	
Condition Codes	28–29, 31–32	
Carry Flag	29	
Mnemonics	31	
Negative Flag	29	
Overflow Flag	29	
Zero Flag	29	
Exceptions	29–30	
Data Abort	30	
Fast Interrupt	30	
Interrupt	30	
Prefetch Abort	30	
Reset	30	
Software Interrupt	30	
Undefined	30	
Instructions		
ADC	136	
ADD	137	
AND	137	
B, BL	137	
CMP	138	
EOR	138	
LDM	139	
LDR	139	
LDRB	140	
MOV	140	
MVN	141	
ORR	141	
SBC	141	
STM	142	
STR	143	
STRB	143	
SUB	144	
SWI	144	
SWP	145	
SWPB	145	
Programs		
16bitadd-2.s	74–75	
16bitadd.s	74	
16bitdatatrans.s	72	
64bitadd.s	78	
Ch10Ex1.s	126–127	
Ch10Ex2.s	127	
Ch10Ex3.s	127–128	
Ch10Ex3a.s	128	
Ch10Ex4.s	128–129	
Ch10Ex5.s	129–131	
Ch10Ex6.s	131–132	
Ch10Ex7.s	132	
Ch5Ex1.s	84	
Ch5Ex2.s	84–85	
Ch5Ex3.s	85–86	
Ch5Ex4.s	86	
Ch5Ex5.s	86–87	
Ch5Ex6.s	87–88	
Ch6Ex1.s	94–95	
Ch6Ex2.s	95	
Ch6Ex3.s	95–96	
Ch6Ex4.s	96–97	
Ch6Ex5.s	97–98	
Ch6Ex6.s	98	
Ch6Ex7.s	98–99	
Ch7Ex1.s	103–104	
Ch7Ex2.s	104	
Ch7Ex3.s	104–105	
Ch7Ex4.s	105	
Ch7Ex5.s	105–106	
Ch7Ex6.s	106–107	
Ch7Ex7.s	107	
Ch8Ex1.s	111–112	
Ch8Ex2.s	112	
Ch8Ex3.s	112–113	
Ch8Ex4.s	113–114	
Ch9Ex1.s	117–118	
Ch9Ex2.s	118	
Ch9Ex3.s	118–119	
Ch9Ex4.s	119–120	
Ch9Ex5.s	120	
comparenum.s	77	
factorial.s	79	
onescomp.s	73	
shiftright.s	75–76	
splitbyte.s	76	
Registers	25–28	
General Purpose (R0–R12)	25	
Link Register (LR/R14)	27	
Program Counter (PC/R15)	27	
Stack Pointer (SP/R13)	26	
Status Register (CPSR/SPSR)	28	
Strings	92–94	

Counted.....	94
Fixed Length.....	93
Terminated.....	93